

Contents

[PS2 Overview](#)
[Memory Map](#)
[I/O Maps](#)
[Emotion Engine \(EE\)](#)
[EE Timers](#)
[Graphics Interface \(GIF\)](#)
[DMA Controller \(DMAC\)](#)
[Graphics Synthesizer \(GS\)](#)
[Vector Interface \(VIF\)](#)
[Vector Unit \(VU\)](#)
[Image Processing Unit \(IPU\)](#)
[EE Interrupt Controller \(INTC\)](#)
[IOP Hardware and Peripherals](#)
[Subsystem Interface \(SIF\)](#)
[BIOS](#)

PS2 Overview

This document, inspired by nocash's websites for various consoles, aims to be a comprehensive reference for the PlayStation 2 hardware. While it is far from complete, it is usable as a reference for what information is documented.

PS2 Hardware

Emotion Engine (EE) - Main CPU.

DMAC - Intelligent DMA controller. Used for accessing most peripherals on the EE.

EE Timers - Four 16-bit timers.

Graphics Interface (GIF) - Transmits graphical data to the GS.

Vector Interface (VIF) - Decompresses vector data, uploads microprograms to the VUs, and sends graphical data to the GIF.

Vector Units (VUs) - Custom DSPs used to process vertex data, physics calculations, and other related tasks.

Image Processing Unit (IPU) - MPEG1/MPEG2 video decoder.

Scratchpad - 16 KB of fast memory.

Graphics Synthesizer (GS) - Fixed-function GPU. Designed to draw polygons very, very fast.

PCRTC - Renders GS output to a television screen. Can be programmed to work on a variety of TVs, including NTSC and PAL.

Input/Output Processor (IOP) - Duplicate of the PSX's CPU. Used for slower input/output tasks and for PSX backwards compatibility.

IOP DMA - Same channels as on the PSX, along with additional channels exclusive to PS2 mode.

IOP Timers - Same timers as on the PSX, with three additional 32-bit counters.

CDVD Drive - Reads disc media. Also responsible for MagicGate decryption.

SI02 - Serial ports used to read controllers and memory cards.

SPU2 - Sound processor. Similar to the PSX SPU with added features.

DEV9 - Expansion port. Used mainly for the PS2 HDD and Network Adapter.

USB

FireWire

Subsystem Interface (SIF) - Allows the EE and IOP to communicate with each other.

Memory Map

EE Virtual/Physical Memory Map

KUSEG: 00000000h-7FFFFFFFh User segment

KSEG0: 80000000h-9FFFFFFFh Kernel segment 0

KSEG1: A0000000h-BFFFFFFFh Kernel segment 1

KSSEG: C0000000h-DFFFFFFFh Supervisor segment

KSEG3: E0000000h-FFFFFFFh Kernel segment 3

Virtual	Physical		
00000000h	00000000h	32 MB	Main RAM (first 1 MB reserved for kernel)
20000000h	00000000h	32 MB	Main RAM, uncached
30100000h	00100000h	31 MB	Main RAM, uncached and accelerated
10000000h	10000000h	64 KB	I/O registers
11000000h	11000000h	4 KB	VU0 code memory
11004000h	11004000h	4 KB	VU0 data memory
11008000h	11008000h	16 KB	VU1 code memory
1100C000h	1100C000h	16 KB	VU1 data memory
12000000h	12000000h	8 KB	GS privileged registers
1C000000h	1C000000h	2 MB	IOP RAM
1FC00000h	1FC00000h	4 MB	BIOS, uncached (rom0)
9FC00000h	1FC00000h	4 MB	BIOS, cached (rom09)
BFC00000h	1FC00000h	4 MB	BIOS, uncached (rom0b)
70000000h	-----	16 KB	Scratchpad RAM (only accessible via virtual addressing)

EE RAM is reportedly expandable up to 256 MB. However, the maximum seen in practice is 128 MB, for special TOOL consoles.

IOP Physical Memory Map

KUSEG: 00000000h-7FFFFFFFh User segment

KSEG0: 80000000h-9FFFFFFFh Kernel segment 0

KSEG1: A0000000h-BFFFFFFFh Kernel segment 1

Physical		
00000000h	2 MB	Main RAM (same as on PSX)
1D000000h		SIF registers
1F800000h	64 KB	Various I/O registers
1F900000h	1 KB	SPU2 registers
1FC00000h	4 MB	BIOS (rom0) - Same as EE BIOS

FFFE0000h (KSEG2) Cache control

Additional Memory

4 MB GS VRAM (used for framebuffer, textures, zbuffer, etc)
 2 MB SPU2 work RAM - quadrupled from PSX's SPU
 8 MB Memory card

I/O Maps

EE Map

EE Timers

100000xxh	Timer 0
100008xxh	Timer 1
100010xxh	Timer 2
100018xxh	Timer 3

Image Processing Unit (IPU)

10002000h 8h	IPU Command
10002010h 4h	IPU Control
10002020h 4h	IPU bit pointer control
10002030h 8h	Top of bitstream
10007000h 10h	Out FIFO (read)
10007010h 10h	In FIFO (write)

Graphics Interface (GIF)

10003000h 4h	GIF_CTRL - Control register
10003010h 4h	GIF_MODE - Mode setting
10003020h 4h	GIF_STAT - Status
10003040h 4h	GIF_TAG0 - Bits 0-31 of tag before
10003050h 4h	GIF_TAG1 - Bits 32-63 of tag before
10003060h 4h	GIF_TAG2 - Bits 64-95 of tag before
10003070h 4h	GIF_TAG3 - Bits 96-127 of tag before
10003080h 4h	GIF_CNT - Transfer status counter
10003090h 4h	GIF_P3CNT - PATH3 transfer status counter
100030A0h 4h	GIF_P3TAG - Bits 0-31 of PATH3 tag when interrupted
10006000h 10h	GIF_FIFO

DMA Controller (DMAC)

100080xxh	VIF0 - channel 0
100090xxh	VIF1 - channel 1
1000A0xxh	GIF - channel 2
1000B0xxh	IPU_FROM - channel 3
1000B4xxh	IPU_TO - channel 4
1000C0xxh	SIF0 - channel 5
1000C4xxh	SIF1 - channel 6
1000C8xxh	SIF2 - channel 7
1000D0xxh	SPR_FROM - channel 8
1000D4xxh	SPR_TO - channel 9
1000E000h 4h	D_CTRL - DMAC control
1000E010h 4h	D_STAT - DMAC interrupt status
1000E020h 4h	D_PCR - DMAC priority control
1000E030h 4h	D_SQWC - DMAC skip quadword
1000E040h 4h	D_RBSR - DMAC ringbuffer size
1000E050h 4h	D_RBOR - DMAC ringbuffer offset
1000E060h 4h	D_STADR - DMAC stall address
1000F520h 4h	D_ENABLER - DMAC disabled status
1000F590h 4h	D_ENABLEW - DMAC disable

Interrupt Controller (INTC)

1000F000h 4h	INTC_STAT - Interrupt status
1000F010h 4h	INTC_MASK - Interrupt mask

Subsystem Interface (SIF)

1000F200h 4h	MSCOM - EE->IOP communication
1000F210h 4h	SMCOM - IOP->EE communication
1000F220h 4h	MSFLAG - EE->IOP flags
1000F230h 4h	SMFLAG - IOP->EE flags
1000F240h 4h	Control register

Privileged GS registers

12000000h 8h	PMODE - various PCRTC controls
12000010h 8h	S.MODE1
12000020h 8h	S.MODE2
12000030h 8h	S.RFSH
12000040h 8h	S.SYNCH1
12000050h 8h	S.SYNCH2
12000060h 8h	S.SYNCV
12000070h 8h	DISPFB1 - display buffer for output circuit 1
12000080h 8h	DISPLAY1 - output circuit 1 control
12000090h 8h	DISPFB2 - display buffer for output circuit 2
120000A0h 8h	DISPLAY2 - output circuit 2 control
120000B0h 8h	EXTBUF
120000C0h 8h	EXTDATA
120000D0h 8h	EXTWRITE
120000E0h 8h	BGCOLR - background color
12001000h 8h	GS_CSR - control register
12001010h 8h	GS_IMR - GS interrupt control
12001040h 8h	BUSDIR - transfer direction
12001080h 8h	SIGLBLID - signal

Misc registers

1000F180h 1h	KPUTCHAR - Console output
1000F430h 4h	MCH_DRD - RDRAM initialization
1000F440h 4h	MCH_RICM

IOP Map

Subsystem Interface (SIF)

1D000000h 4h	MSCOM - EE->IOP communication
1D000010h 4h	SMCOM - IOP->EE communication
1D000020h 4h	MSFLAG - EE->IOP flags
1D000030h 4h	SMFLAG - IOP->EE flags
1D000040h 4h	Control register

CDVD Drive

1F402004h 1h	Current N command
1F402005h 1h	N command status (R)
1F402005h 1h	N command params (W)
1F402006h 1h	Error
1F402007h 1h	Send BREAK command
1F402008h 1h	CDVD_I_STAT - interrupt register
1F40200Ah 1h	Drive status
1F40200Fh 1h	Disk type

```

1F402016h 1h    Current S command
1F402017h 1h    S command status
1F402018h 1h    S command params

```

Interrupt Control

```

1F801070h 4h    I_STAT - Interrupt status
1F801074h 4h    I_MASK - Interrupt mask
1F801078h 1h    I_CTRL - Global interrupt disable

```

DMA registers

```

1F80108xh      MDECin - channel 0
1F80109xh      MDECout - channel 1
1F8010Axh      SIF2 (GPU) - channel 2
1F8010Bxh      CDVD - channel 3
1F8010Cxh      SPU2 Core0 - channel 4
1F8010Dxh      PIO - channel 5
1F8010Exh      OTC - channel 6
1F80150xh      SPU2 Core1 - channel 7
1F80151xh      DEV9 - channel 8
1F80152xh      SIF0 - channel 9
1F80153xh      SIF1 - channel 10
1F80154xh      SIO2in - channel 11
1F80155xh      SIO2out - channel 12

```

```

1F8010F0h 4h    DPCR - DMA priority control
1F8010F4h 4h    DICR - DMA interrupt control
1F801570h 4h    DPCR2 - DMA priority control 2
1F801574h 4h    DICR2 - DMA priority control 2
1F801578h 4h    DMACEN - DMA global enable
1F80157Ch 4h    DMACINTEN - DMA global interrupt control

```

IOP Timers

```

1F80110xh      Timer 0
1F80111xh      Timer 1
1F80112xh      Timer 2
1F80148xh      Timer 3
1F80149xh      Timer 4
1F8014Axh      Timer 5

```

Serial Interface (SIO2)

```

1F808200h 40h   SEND3 buffer
1F808240h 20h   SEND1/2 buffers
1F808260h 1h    In FIFO
1F808264h 1h    Out FIFO
1F808268h 4h    SIO2 control
1F80826Ch 4h    RECV1
1F808270h 4h    RECV2
1F808274h 4h    RECV3

```

Sound Processing Unit (SPU2)

```

1F900000h 180h  Core0 Voice 0-23 registers
1F900190h 4h    Key ON 0/1
1F900194h 4h    Key OFF 0/1
1F90019Ah 2h    Core attributes
1F90019Ch 4h    Interrupt address H/L
1F9001A8h 4h    DMA transfer address H/L
1F9001ACh 2h    Internal transfer FIFO
1F9001B0h 2h    AutoDMA status
1F9001C0h 120h  Core0 Voice 0-23 start/loop/next addresses
1F900340h 4h    ENDX 0/1
1F900344h 2h    Status register

```

... above addresses repeat for Core1 starting at 1F900400h ...

```

1F900760h 2h    Master Volume Left
1F900762h 2h    Master Volume Right
1F900764h 2h    Effect Volume Left
1F900766h 2h    Effect Volume Right
1F900768h 2h    Core1 External Input Volume Left
1F90076Ah 2h    Core1 External Input Volume Right

```

Emotion Engine (EE)

The EE is the main CPU of the PS2. Also known as the R5900, it is a custom MIPS core designed by Sony and Toshiba.

EE Reference

[EE Architecture](#)

[EE Registers](#)

[EE Instruction Decoding](#)

[EE RDRAM initialization](#)

[EE COP0 Registers](#)

[EE COP0 Exception Handling](#)

[EE COP0 Memory Management](#)

[EE COP0 Caches](#)

[EE COP0 Timer](#)

[EE COP1 Registers](#)

[EE COP1 Floating Point Format](#)

EE Architecture

On real hardware, the EE itself contains many different hardware units along with the main CPU, such as the DMAC and IPU. This document will consider the EE as only the MIPS CPU and its coprocessors.

Mix of MIPS III and MIPS IV with dozens of custom instructions

Speed: 294.912 MHz

Superscalar, able to issue two instructions per cycle in ideal conditions

Support for custom integer SIMD instructions known as MMI (MultiMedia Instructions)

16 KB instruction cache

8 KB data cache

16 KB scratchpad. Fast memory directly accessible by program and DMAC

COP0 system coprocessor. Responsible for handling memory management, exceptions, caches, and performance counters

COP1 FPU. Fast single-precision unit

COP2 VU0. SIMD floating-point processor that can either run as a separate CPU or be used by the EE core for COP2 instructions

See [Vector Unit \(VU\)](#) for a general discussion on the VUs, including VU0.

EE Registers

General-Purpose Registers (GPRs)

As per MIPS tradition, the EE has 32 GPRs. Notably, they are all 128-bit, though the full 128 bits are only used in certain instructions.

Name	Convention
zero	Hardwired to 0, writes are ignored
at	Temporary register used for pseudo-instructions
v0-v1	Return register, holds values returned by functions
a0-a3	Argument registers, holds first four parameters passed to a function
t0-t7	Temporary registers. t0-t3 may also be used as additional argument registers
s0-s7	Saved registers. Functions must save and restore these before using them
t8-t9	Temporary registers
k0-k1	Reserved for use by kernels
gp	Global pointer
sp	Stack pointer
fp	Frame pointer
ra	Return address. Used by JAL and (usually) JALR to store the address to return to after a function

Aside from zero, all GPRs may be freely accessed if convention rules are respected.

Special Registers

Name	Purpose
pc	Program counter, address of currently-executing instruction (32-bit)
hi/lo	Stores multiplication and division results (64-bit)
hi1/lo1	Used by MULT1/DIV1 type instructions, same as above (64-bit)
sa	Shift amount used by QFSRV instruction

EE Instruction Decoding

Normal instructions

31-----26-----0	
opcode	
-----6-----	
--000-- --001-- --010-- --011-- --100-- --101-- --110-- --111-- lo	
000	*1 *2 J JAL BEQ BNE BLEZ BGTZ
001	ADDI ADDIU SLTI SLTIU ANDI ORI XORI LUI
010	*3 *4 *5 --- BEQL BNEL BLEZL BGTZL
011	DADDI DADDIU LDL LDR *6 --- LQ SQ
100	LB LH LWL LW LBU LHU LWR LWU
101	SB SH SWL SW SDL SDR SWR CACHE
110	--- LWC1 --- PREF --- --- LQC2 LD
111	--- SWC1 --- --- --- --- SQC2 SD
hi	-----
*1 = SPECIAL, see SPECIAL list *2 = REGIMM, see REGIMM list	
*3 = COP0 *4 = COP1	
*5 = COP2 *6 = MMI table	

SPECIAL

31-----26-----5-----0	
= SPECIAL function	
-----6-----	
--000-- --001-- --010-- --011-- --100-- --101-- --110-- --111-- lo	
000	SLL --- SRL SRA SLLV --- SRLV SRAV
001	JR JALR MOVZ MOVN SYSCALL BREAK --- SYNC
010	MFHI MTHI MFLO MTLO DSLLV --- DSRLV DSRAV
011	MULT MULTU DIV DIVU --- --- ---
100	ADD ADDU SUB SUBU AND OR XOR NOR
101	MFSA MTSa SLT SLTU DADD DADDU DSUB DSUBU
110	TGE TGEU TLT TLTU TEQ --- TNE ---
111	DSLL --- DSRL DSRA DSLL32 --- DSRL32 DSRA32
hi	-----

REGIMM

31-----26-----20-----16-----0	
= REGIMM rt	
-----6-----5-----	
--000-- --001-- --010-- --011-- --100-- --101-- --110-- --111-- lo	
00	BLTZ BGEZ BLTZL BGEZL --- --- ---
01	TGEI TGEIU TLT TLTU TEQI --- TNEI ---
10	BLTZAL BGEZAL BLTZALL BGEZALL --- --- ---
11	MTSAB MTSAL --- --- --- --- ---
hi	-----

MMI

31-----26-----5-----0	
= MMI function	
-----6-----	
--000-- --001-- --010-- --011-- --100-- --101-- --110-- --111-- lo	
000	MADD MADDU --- --- PLZCW --- --- ---
001	*1 *2 --- --- --- --- ---
010	MFHI1 MTHI1 MFLO1 MTLO1 --- --- ---
011	MULT1 MULTU1 DIV1 DIVU1 --- --- ---
100	MADD1 MADDU1 --- --- --- --- ---
101	*3 *4 --- --- --- --- ---
110	PMFHL PMTHL --- --- PSLLH --- PSRLH PSRAH
111	--- --- --- --- PSLLW --- PSRLW PSRAW
hi	-----
*1 = MMI0 list *2 = MMI2 list	
*3 = MMI1 list *4 = MMI3 list	

MMI0

	31-----26-----10-----6-5-----0
	----- ----- function MMI0
	-----6-----
	---00--- ---01--- ---10--- ---11--- lo
000	PADDW PSUBW PCGTW PMAXW
001	PADDH PSUBH PCGTH PMAXH
010	PADDB PSUBB PCGTB ---
011	--- --- --- ---
100	PADDSW PSUBSW PEXTLW PPACW
101	PADDSH PSUBSH PEXTLH PPACH
110	PADDSB PSUBSB PEXTLB PPACB
111	--- --- PEXT5 PPAC5
hi	-----

MMI1

	31-----26-----5-----0
	----- ----- function MMI1
	-----6-----
	---00--- ---01--- ---10--- ---11--- lo
000	--- PABSW PCEQW PMINW
001	PADSBH PABSH PCEQH PMINH
010	--- --- PCEQB ---
011	--- --- --- ---
100	PADDUW PSUBUW PEXTUW ---
101	PADDUH PSUBUH PEXTUH ---
110	PADDUB PSUBUB PEXTUB QFSRV
111	--- --- --- ---
hi	-----

MMI2

	31-----26-----5-----0
	----- ----- function MMI2
	-----6-----
	---00--- ---01--- ---10--- ---11--- lo
000	PMADDW --- PSLLVW PSRLVW
001	PMSUBW --- --- ---
010	PMFHI PMFLO PINTH ---
011	PMULTW PDIVW PCPYLD ---
100	PMADDH PHMADH PAND PXOR
101	PMSUBH PHMSBH --- ---
110	--- --- PEXEH PREVH
111	PMULTH PDIVBW PEXEW PROT3W
hi	-----

MMI3

	31-----26-----5-----0
	----- ----- function MMI3
	-----6-----
	---00--- ---01--- ---10--- ---11--- lo
000	PMADDUW --- --- PSRAVW
001	--- --- --- ---
010	PMTHI PMTLO PINTEH ---
011	PMULTUW PDIVUW PCPYUD ---
100	--- --- POR PNOR
101	--- --- --- ---
110	--- --- PEXCH PCPYH
111	--- --- PEXCW ---
hi	-----

COP0

	31-----26-25-----21-----0
	= COP0 fmt
	-----6-----5-----
	---000--- ---001--- ---010--- ---011--- ---100--- ---101--- ---110--- ---111--- lo
00	MFC0 --- --- MTC0
01	*1 --- --- ---
10	*2 --- --- ---
11	--- --- --- ---
hi	-----

*1=BC See BC0 list *2 = TLB instr, see TLB list

BC0

	31-----26-25-----21-20-----16-----0
	= COP0 BC0 fmt
	-----6-----5-----5-----
	---000--- ---001--- ---010--- ---011--- ---100--- ---101--- ---110--- ---111--- lo
00	BC0F BC0T BC0FL BC0TL
01	--- --- --- ---
10	--- --- --- ---
11	--- --- --- ---
hi	-----

TLB/Exception

	31-----26-25-----21-----5-----0
	= COP0 TLB fmt
	-----6-----5-----
	---000--- ---001--- ---010--- ---011--- ---100--- ---101--- ---110--- ---111--- lo
000	--- TLBR TLBWI
001	TLBP --- ---
010	--- --- --- ---
011	ERET --- ---
100	--- --- --- ---
101	--- --- --- ---
110	--- --- --- ---
111	EI DI --- ---
hi	-----

COP1

31-----26-25-----21-----0		= COP1		fmt		-----0				
-----6-----5-----		-----6-----5-----		-----6-----5-----		-----6-----5-----				
00	hi	000	001	010	011	100	101	110	111	lo
01		MFC1	---	CFC1	---	MTC1	---	CTC1	---	
10		*1	---	---	---	*3	---	---	---	
11		*2	---	---	---	---	---	---	---	
hi		---	---	---	---	---	---	---	---	

*1 = BC instructions, see BC1 list *2 = S instr, see FPU list
 *3 = W instr, see FPU list

BC1

31-----26-25-----21-20-----16-----0											
= COP1		BC1		fmt							
-----6-----5-----		-----5-----		-----5-----							
00	hi	000	001	010	011	100	101	110	111	lo	
01		BC1F	BC1T	BC1FL	BC1TL	---	---	---	---		
10		---	---	---	---	---	---	---	---		
11		---	---	---	---	---	---	---	---		
hi		---	---	---	---	---	---	---	---		

FPU.S

31-----26-25-----21-----5-----0		= COP1		= S		-----5-----		function		
-----6-----5-----		-----6-----5-----		-----6-----5-----		-----6-----5-----		-----6-----5-----		
000	hi	000	001	010	011	100	101	110	111	lo
001		ADD.S	SUB.S	MUL.S	DIV.S	SQRT.S	ABS.S	MOV.S	NEG.S	
010		---	---	---	---	---	---	---	---	
011		ADDA.S	SUBA.S	MULA.S	---	MADD.S	MSUB.S	MADDA.S	MSUBA.S	
100		---	---	---	---	CVT.W	---	---	---	
101		MAX.S	MIN.S	---	---	---	---	---	---	
110		C.F	---	C.EQ	---	C.LT	---	C.LE	---	
111		---	---	---	---	---	---	---	---	
hi		---	---	---	---	---	---	---	---	

FPU.W

31-----26-25-----21-----5-----0		= COP1		= W		-----5-----		function		
-----6-----5-----		-----6-----5-----		-----6-----5-----		-----6-----5-----		-----6-----5-----		
000	hi	000	001	010	011	100	101	110	111	lo
001		---	---	---	---	---	---	---	---	
010		---	---	---	---	---	---	---	---	
011		---	---	---	---	---	---	---	---	
100	CVT.S	---	---	---	---	---	---	---	---	
101		---	---	---	---	---	---	---	---	
110		---	---	---	---	---	---	---	---	
111		---	---	---	---	---	---	---	---	
hi		---	---	---	---	---	---	---	---	

COP2

31-----26-25-----21-----0		= COP2		fmt		-----0				
-----6-----5-----		-----6-----5-----		-----6-----5-----		-----6-----5-----				
00	hi	000	001	010	011	100	101	110	111	lo
01		---	QMF2	CFC2	---	---	QMT2	CTC2	---	
10		*1	---	---	---	---	---	---	---	
11		*2	*2	*2	*2	*2	*2	*2	*2	
hi		*2	*2	*2	*2	*2	*2	*2	*2	

*1 = BC instructions, see BC2 list *2 =see special1 table

BC2

31-----26-25-----21-20-----16-----0		= COP0		BC2		fmt		-----0		
-----6-----5-----		-----6-----5-----		-----6-----5-----		-----6-----5-----		-----6-----5-----		
00	hi	000	001	010	011	100	101	110	111	lo
01		BC2F	BC2T	BC2FL	BC2TL	---	---	---	---	
10		---	---	---	---	---	---	---	---	
11		---	---	---	---	---	---	---	---	
hi		---	---	---	---	---	---	---	---	

COP2 Special1

31-----26-25-----21-20-----5-----0											
=COP2		Special1						function			
-----6-----		-----5-----		-----6-----		-----5-----		-----6-----		-----5-----	
000	hi	000	001	010	011	100	101	110	111	lo	
000	VADDx	VADDy	VADDz	VADDw	VSUBx	VSUBy	VSUBz	VSUBw			
001	VMADDx	VMADDy	VMADDz	VMADDw	VMSUBx	VMSUBy	VMSUBz	VMSUBw			
010	VMAXx	VMAXy	VMAXz	VMAXw	VMINix	VMINIy	VMINIZ	VMINIw			
011	VMULx	VMULy	VMULz	VMULw	VMULq	VMAXi	VMULi	VMINIi			
100	VADDq	VMADDq	VADDi	VMADDi	VSUBq	VMSUBq	VSUBi	VMSUBi			
101	VADD	VMADD	VMUL	VMAX	VSUB	VMSUB	VOPMSUB	VMINI			
110	VIADD	VISUB	VIADDI	---	VIAND	VIOR	---	---			
111	VCALLMS	CALLMSR	---	---	*1	*1	*1	*1			
hi											

*1=see special2 table

COP2 Special2

31-----26-25-----21-20-----11-10-----6-5-2-1--0		=COP2		Special2		-----6-----5-----		fhi 1111 flo	
Note: opcode is flo (fhi * 4).									
	0000	0001	0010	0011	0100	0101	0110	0111	10
0000	VADDax	VADDay	VADDaz	VADDaw	VSUBax	VSUBay	VSUBaz	VSUBaw	
0001	VMADDax	VMADDay	VMADDaz	VMADDaw	VMSUBax	VMSUBay	VMSUBaz	VMSUBaw	
0010	VITOF0	VITOF4	VITOF12	VITOF15	VFTOI0	VFTOI4	VFTOI12	VFTOI15	

0011	VMULAx	VMULAY	VMULAZ	VMULAw	VMULAq	VABS	VMULAi	VCLIPw
0100	VADDAq	VMADDAq	VADDAi	VMADDAi	VSUBAq	VMSUBAq	VSUBAi	VMSUBAi
0101	VADDA	VMADDA	VMULA	---	VSUBA	VMSUBA	VOPMULA	VNOP
0110	VMOVE	VMR32	---	---	VLQI	VSQI	VLQD	VSQD
0111	VDIV	VSQRT	VRSQRT	VWAITQ	VMTIR	VMFIR	VILWR	VISWR
1000	VRNEXT	VRGET	VRINIT	VRXOR	---	---	---	---
1001	---	---	---	---	---	---	---	---
1010	---	---	---	---	---	---	---	---
1011	---	---	---	---	---	---	---	---
1100	---	---	---	---	---	---	---	---
1101	---	---	---	---	---	---	---	---
1110	---	---	---	---	---	---	---	---
1111	---	---	---	---	---	---	---	---
hi	-----	-----	-----	-----	-----	-----	-----	-----

EE RDRAM initialization

During the early boot stages, the BIOS writes to a pair of registers; MCH_DRD and MCH_RICM used to initialize RDRAM. Not much is known about these registers. The following logic used in emulators:

Read

```
case 0x1000f430:
{
    return 0;
}
case 0x1000f440:
{
    uint8_t SOP = (MCH_RICM >> 6) & 0xF;
    uint8_t SA = (MCH_RICM >> 16) & 0xFF;
    if (!SOP)
    {
        switch (SA)
        {
            case 0x21:
                if (rdram_sdevid < 2)
                {
                    rdram_sdevid++;
                    return 0x1F;
                }
                return 0;
            case 0x23:
                return 0x0D0D;
            case 0x24:
                return 0x0090;
            case 0x40:
                return MCH_RICM & 0x1F;
        }
    }
    return 0;
}
```

Write

```
case 0x1000f430:
{
    uint8_t SA = (data >> 16) & 0xFF;
    uint8_t SBC = (data >> 6) & 0xF;

    if (SA == 0x21 && SBC == 0x1 && ((MCH_DRD >> 7) & 1) == 0)
        rdram_sdevid = 0;

    MCH_RICM = data & ~0x80000000;
    break;
}
case 0x1000f440:
{
    MCH_DRD = data;
    break;
}
```

EE COP0 Registers

Num	Name
\$0	Index
\$1	Random
\$2	EntryLo0
\$3	EntryLo1
\$4	Context
\$5	PageMask
\$6	Wired
\$8	BadVAddr
\$9	Count
\$10	EntryHi
\$11	Compare
\$12	Status
\$13	Cause
\$14	EPC
\$15	PRid
\$16	Config
\$23	BadPAddr
\$24	Debug
\$25	Perf
\$28	TagLo
\$29	TagHi
\$30	ErrorEPC

EE COP0 Exception Handling

Exceptions

An exception is an *exceptional* event that stops the CPU's current task and forces it to address the event, usually with kernel privileges.

On the EE, there are exceptions for interrupts, TLB management, syscalls, and various errors. When an exception occurs, the EE jumps to one of the following vectors:

Name	Normal	Bootstrap	Level
Reset/NMI	BFC00000h	BFC00000h	2
TLB Refill	80000000h	BFC00200h	1
Perf. Counter	80000080h	BFC00280h	2
Debug	80000100h	BFC00300h	2
All others	80000180h	BFC00380h	1
Interrupt	80000200h	BFC00400h	1

The full exception handling works as follows.

```
void handle_exception_level1(u32 vector, u8 cause)
{
    Cause.ExcCode = cause;
    if (in_branch_delay)
    {
        EPC = PC - 4;
        Cause.BD = true;
    }
    else
    {
        EPC = PC;
        Cause.BD = false;
    }
    Status.EXL = true;
    PC = vector;
}

void handle_exception_level2(u32 vector, u8 cause)
{
    Cause.ExcCode = cause;
    if (in_branch_delay)
    {
        ErrorEPC = PC - 4;
        Cause.BD2 = true;
    }
    else
    {
        ErrorEPC = PC;
        Cause.BD2 = false;
    }
    Status.ERL = true;
    PC = vector;
}
```

\$08 - COP0.BadVAddr

0-31 Virtual address that caused an exception BadVAddr is written to when a TLB Refill, TLB Modified, TLB Invalid, or Address Error exception occurs.

\$12 - COP0.Status

```
0    IE - Interrupt enable
1    EXL - Exception level (set when a level 1 exception occurs)
2    ERL - Error level (set when a level 2 exception occurs)
3-4  KSU - Privilege level
      0=Kernel
      1=Supervisor
      2=User
10   INT0 enable (INTC)
11   INT1 enable (DMAC)
12   Bus error mask - when set, bus errors are disabled
15   INT5 enable (COP0 timer)
16   EIE - Master interrupt enable
17   EDI - If not set, EI/DI only works in kernel mode
18   CH - Status of most recent Cache Hit instruction
      0=miss
      1=hit
22   BEV - If set, level 1 exceptions go to "bootstrap" vectors in BFC00xxx
23   DEV - If set, level 2 exceptions go to "bootstrap" vectors in BFC00xxx
28-31 Usability of coprocessors 0-3
      If not set, using a coprocessor raises an exception
```

All interrupts are disabled unless the following condition is true.

```
Status.IE && Status.EIE && !Status.EXL && !Status.ERL
```

When Status.ERL or Status.EXL is set, the EE operates in kernel mode regardless of Status.KSU.

\$13 - COP0.Cause

```
2-6  Exception code
      00h=Interrupt
      01h=TLB Modified
      02h=TLB Refill (instruction fetch or load)
      03h=TLB Refill (store)
      04h=Address Error (instruction fetch or load)
      05h=Address Error (store)
      06h=Bus Error (instruction)
      07h=Bus Error (data)
      08h=Syscall
      09h=Breakpoint
      0Ah=Reserved Instruction
      0Bh=Coprocessor Unusable
      0Ch=Overflow
      0Dh=Trap
10   INTC interrupt pending
11   DMAC interrupt pending
15   COP0 timer interrupt pending
```

- 16-18 Error code
 - 00h=Reset
 - 01h=NMI
 - 02h=Performance counter
 - 03h=Debug
- 28-29 Coprocessor that triggered a CU exception
- 30 BD2 - Set when a level 2 exception occurs in a delay slot
- 31 BD - Set when a level 1 exception occurs in a delay slot

Cause is used to determine what caused an exception.

The BD/BD2 bits are important as when they are set, EPC/ErrorEPC is set to the address of the branch instruction, not its delay slot. This allows the EE to re-execute the branch when it returns from an exception.

\$14/\$30 - COP0.EPC/COP0.ErrorEPC

- 0-31 Address to return to after an exception

EPC and ErrorEPC are used by level 1 and 2 exceptions respectively. When the ERET instruction is executed, it jumps to the address in EPC or ErrorEPC, depending on if Status.EXL or Status.ERL is set.

\$23 - COP0.BadPAddr

- 0-31 Physical address that caused an exception

BadPAddr is written to when a bus error occurs and Status.BEM is not set.

EE COP0 Memory Management

Virtual Memory Basics

Virtual memory is a mechanism that allows processors to arbitrarily map a virtual address to a physical address. This is accomplished using a Memory Management Unit (MMU). For example, a program might access address 1000h. With virtual memory, this address could map to any physical address, such as 1000h, 20000h, 424000h, and so on. Virtual address 1000h might also not be mapped to anything, in which case accessing it causes an error. The PS2 and most modern OSes do not map addresses near NULL, so that if a program uses a NULL pointer, it can crash immediately rather than corrupt program data.

MIPS Memory Segments

The MIPS architecture splits the memory space in half for the user and kernel, and kernelspace is further partitioned into various segments.

Name	Range	Description
kuseg	00000000h-7FFFFFFFh	User, TLB-mapped
kseg0	80000000h-9FFFFFFFh	Kernel, directly-mapped, cached
kseg1	A0000000h-BFFFFFFFh	Kernel, directly-mapped, uncached
ksseg	C0000000h-DFFFFFFFh	Supervisor, TLB-mapped
kseg3	E0000000h-FFFFFFFh	Kernel, TLB-mapped

Accessing a privileged region without the necessary privileges raises an Address Error exception.

For kseg0 and kseg1, the physical address retrieved is simply (address & 1FFFFFFFh).

Translation-Lookaside Buffer (TLB)

The EE uses a TLB, a cache of virtual->physical mappings, for memory management. The TLB contains 48 entries split into even and odd pages. A TLB entry has the following format:

- 1 V0 - Even page valid. When not set, the memory referenced in this entry is not mapped.
- 2 D0 - Even page dirty. When not set, writes cause an exception.
- 3-5 C0 - Even page cache mode.
 - 2=Uncached
 - 3=Cached
 - 7=Uncached accelerated
- 6-25 PFN0 - Even page frame number.
- 33 V1 - Odd page valid.
- 34 D1 - Odd page dirty.
- 35 C1 - Odd page cache mode.
- 38-57 PFN1 - Odd page frame number.
- 63 S - Scratchpad. When set, the virtual mapping goes to scratchpad instead of main memory.
- 64-71 ASID - Address Space ID.
- 76 G - Global. When set, ASID is ignored.
- 77-95 VPN2 - Virtual page number / 2.
 - Even pages have a VPN of (VPN2 * 2) and odd pages have a VPN of (VPN2 * 2) + 1
- 109-120 MASK - Size of an even/odd page.

The units of PFN0/PFN1 and VPN2 depend on MASK. For example, if MASK indicates a size of 4 KB, then if PFN0/PFN1 are 10h/11h and VPN2 is 8h, the virtual address of the even page is (8h * 2 * 4096) or 10000h, and the virtual address of the odd page is ((8h * 2) + 1) * 4096 or 11000h.

The physical addresses of the even/odd pages are also 10000h/11000h respectively. Therefore, an access to 10500h will touch physical address 10500h.

\$0 - COP0.Index

- 0-5 TLB entry to access with TLBR/TLBWI

\$1 - COP0.Random

- 0-5 TLB entry to access with TLBWR

Random decrements after every instruction that is executed. It has an upper bound of 47 and a lower bound of the value in Wired.

\$2/\$3 - COP0.EntryLo0/EntryLo1

- 0 G - Global.
- 1 V - Page valid. When not set, the memory referenced in this entry is not mapped.
- 2 D - Page dirty. When not set, writes cause an exception.
- 3-5 C - Page cache mode.
 - 2=Uncached
 - 3=Cached
 - 7=Uncached accelerated
- 6-25 PFN - Page frame number.
- 31 S - Scratchpad. Only applicable for EntryLo0.

EntryLo0/EntryLo1 modify the lower part of the TLB as given for the even page and odd page respectively.

The Global bit in the TLB entry is only set if both EntryLo0 and EntryLo1 have their G bit set.

\$5 - COP0.PageMask

- 13-24 Page size
 - 000h=4 KB
 - 003h=16 KB

```

00Fh=64 KB
03Fh=256 KB
0FFh=1 MB
3FFh=4 MB
FFFh=16 MB

```

\$6 - COP0.Wired

0-5 Lower bound of Random

\$10 - COP0.EntryHi

0-7 ASID
13-31 VP2

EE COP0 Caches

Cache Basics

As technology has progressed, CPU speeds have far outpaced memory speeds. An access to main RAM on the PS2 takes around 40 EE cycles, and on a modern computer, it can be hundreds of cycles! CPU caches alleviate this problem. They are smaller but faster buffers of memory that the CPU attempts to access when fetching instructions or loading/storing data.

If the CPU finds what it is looking for in the cache, it is called a cache hit. If the CPU has to access main memory, it is called a cache miss.

Modern caches are very complex and have multiple levels, but the EE's caches are rather simple, having only a single level and two ways.

EE Cache Structure

The instruction cache and data cache are split into 128 and 64 cache lines respectively. Each line has two 64-byte (4-quadword) ways, which means the icache and dcache are 16 KB and 8 KB respectively.

Each way has a tag which stores metadata about the way, including, for instance, if the way has valid data in it.

The structure of the tags for the icache and dcache is as follows.

```

-----
|V|R|                PFN                |
-----

```

```

-----
|D|V|R|L|            PFN                |
-----

```

V=Valid. If set, the way contains valid data.

R=LRF bit. Used for the LRF cache replacement algorithm.

D=Dirty. Set on writes. If set when the way is reloaded, the data is first stored to main memory before loading new data.

L=Locked. Prevents the way from ever being reloaded. Only allowed for a single way in a line.

Each cache line is indexed by virtual address. Here is how a virtual address is used:

```

-----
31-----14-13-----6-5-----0
|                PFN                | index | offset |
-----

```

```

-----
31-----13-12-----6-5-----0
|                PFN                | index | offset |
-----

```

Note how the dcache index is 6-bit, whereas the icache index is 7-bit.

The index is used to get the cache line to access. The offset is used to, of course, offset into the 64-byte region.

Cache Misses and Replacement Algorithm

When accessing the cache, the EE checks both ways in a line by comparing the address's PFN with the PFNs in the ways' tags. If both lookups fail, a cache miss has occurred, and new data needs to be loaded into the cache.

The EE uses a Least Recently Filled (LRF) algorithm to determine which way to load data into. If either way does not have the Valid bit set, the EE sets the Valid bit in the first way that does not have it and loads data into it. If both ways are full, the EE takes the XOR of the LRF bits in the tags. The result is the index of the way it uses. The EE then flips the LRF bit in the selected way.

If a way in the data cache has the Locked bit set, the EE will always reload the other way. This allows important data to remain in the cache indefinitely.

Four quadwords are loaded into a way at a time. Since the EE bus is 128-bit, this means a cache miss has a penalty of a minimum of four bus cycles (eight EE cycles). However, main memory is much slower - ~40 EE cycles is closer to reality, though the exact number is nondeterministic due to the nature of RDRAM.

Cache Coherency

Because the caches are separate from main memory, it is possible for the contents of the caches to be different from the contents of main memory.

This is not an issue when simply accessing memory. However, other hardware components such as the DMAC cannot read from the cache. When transferring or receiving data from hardware peripherals, the program is expected to maintain cache coherency by flushing the cache, which forces the cache to synchronize its contents with main memory.

Sony's SDK correctly maintains cache coherency in most cases, so most games are none the wiser. Certain games are so buggy, however, that they require cache emulation to work.

Some examples:

- Dead or Alive 2: Re-uses the same buffer for sending and receiving data in a library that communicates with a custom sound module on the IOP. Without cache emulation, the IOP will overwrite data in the send/receive buffer, preventing the game from booting.
- Ice Age 2: The Meltdown: Function returns a pointer to a stack array used to send data to the IOP. Astute C programmers will recognize this as undefined behavior. The game immediately flushes the cache after calling this function, so the ensuing corruption only occurs in the cache, and the data in main memory is protected.
- Savage Skies: Similar to IA2. Allocates a VIF DMA buffer on the stack, then starts a transfer without waiting for it to end and returns from a function. Without cache emulation, the buffer is corrupted and the game is unable to go in-game.
- WRC 4: Notably requires icache emulation. Main executable is a small decompressor that decompresses a larger executable. The decompressed ELF is large enough to overlap with the decompressor in memory. Without icache emulation, the decompressor is destroyed.

EE COP0 Timer

\$09 - COP0.Count

0-31 Timer count value

Count increments automatically every EE cycle.

\$11 - COP0.Compare

0-31 Timer compare value

When Count==Compare, a COP0 timer interrupt is signaled. Writing to Compare acknowledges and clears the pending interrupt.

EE COP1 Registers

Floating-point Registers (FPRs)

Name	Convention
f0-f3	Return values
f4-f11	Temporary registers
f12-f19	Argument registers
f20-f31	Saved registers

Special Registers (accessed through CFC1/CTC1)

Name	Purpose
fcr0	Reports implementation and revision of FPU
fcr31	Control register, stores status flags

EE COP1 Floating Point Format

COP1, along with the rest of the PS2, only supports single-precision floats. The PS2 uses IEEE 754 floats with the following format:

31-30	-----	23-22	-----	0
si	exponent		mantissa	

"si" is the sign bit, and the mantissa is the fractional component. The float is calculated as follows.

$$\text{num} = (\text{si} ? -1 : 1) * 2^{(\text{exponent} - 127)} * (1.\text{mantissa})$$

1.mantissa means the number 1 and all the fractional bits following the decimal.

For example, 3F800000h becomes 1.0f.

IEEE 754 Differences

There are major differences between the PS2 floating-point format and standard IEEE 754, used in most modern computers.

- NaNs and Infinities do not exist on the PS2. In IEEE 754, any number with the exponent field set to 255 is a NaN/Infinity. On the PS2, it is treated as a normal value. This means the upper range of PS2 floats is greater than on IEEE 754.
- Denormals, numbers with the exponent field set to 0, do not exist on the PS2. For every arithmetic operation, a denormal is automatically truncated to zero.
- On the PS2, the rounding mode is always forced to round-towards-zero. The PS2 uses a quirky version where the least-significant-bit may vary after a round, and the details on this are not well understood.

EE Timers

The EE has four hardware timers. They are all functionally equivalent, though T0 and T1 have an additional register used when an SBUS interrupt occurs. T3 is reserved by the BIOS for alarm functions. All other timers may be used freely.

10000000h + N*800h TN_COUNT

0-15 Current counter

16-bit counter incremented according to the settings in TN_MODE. Writes set the counter to the given value.

10000010h + N*800h TN_MODE

0-1	Clock
	0=Bus clock (~147 MHz)
	1=Bus clock / 16
	2=Bus clock / 256
	3=HBLANK
2	Gate enable
3	Gate type
	0=HBLANK
	1=VBLANK
4-5	Gate mode
	0=Count while gate not active
	1=Reset counter when gate goes from low to high
	2=Reset counter when gate goes from high to low
	3=Reset counter for high->low gate transitions
6	Clear counter when it reaches compare value
7	Timer enable
8	Compare interrupt enable - trigger interrupt when COUNT==COMP
9	Overflow interrupt enable - trigger interrupt when COUNT goes from FFFFh to 0000h
10	Compare interrupt flag
	W: Clears flag
11	Overflow interrupt flag
	W: Clears flag

Timer interrupts are edge-triggered: an interrupt will only be sent to the EE if either interrupt flag goes from 0 to 1.

Harry Potter and the Prisoner of Azkaban relies on this behavior, as it does not clear the interrupt flags in the timer handler but rather in a different part of the code.

Not emulating this will cause the game to crash by reading from NULL.

10000020h + N*800h TN_COMP

0-15 Compare value

10000030h + N*800h TN_HOLD

0-15 Counter value when an SBUS interrupt occurs

Only exists for T0 and T1.

Vertical and horizontal timings are presented below.

Vertical Timings

PAL: 312 scanlines per frame (VB0FF: 286 | VB0N: 26)
 NTSC: 262 scanlines per frame (VB0FF: 240 | VB0N: 22)

Horizontal Timings

PAL: 9436 BUSCLK cycles per scanline
 NTSC: 9370 BUSCLK cycles per scanline

Graphics Interface (GIF)

The GIF sends textures and geometry to the GS. It takes data from three different PATHs:

- PATH1: VU1 via XGKICK instruction. Highest priority.
- PATH2: VIF1 via DIRECT/DIRECTHL. Medium priority.
- PATH3: GIF DMAC channel (channel 2). Lowest priority.

Only one PATH may run at a time. Under normal conditions, when one PATH stops, the highest priority queued PATH will begin transfer.

GIF Reference

[GIF I/O](#)

[GIFtags](#)

[GIF Data Formats](#)

[GIF PATH3 Masking](#)

GIF I/O

10003000h GIF_CTRL - Control register (W)

- 0 Reset GIF
- 1-2 Unused
- 3 Temporary stop (1=stop transfers, 0=restart transfers)
- 4-31 Unused

10003010h GIF_MODE - Mode of operation (W)

- 0 Mask PATH3 (1=Mask)
- 1 Unused
- 2 Intermittent mode

When PATH3 is masked by this register and GIF DMA is ongoing, the mask applies once GIF DMA ends its transfer.

10003020h GIF_STAT - Status register (R)

- 0 PATH3 masked by GIF_MODE
- 1 PATH3 masked by VIF1 MASKP3 register
- 2 Intermittent mode activated
- 3 Temporary stop
- 4 Unused
- 5 PATH3 interrupted (by intermittent mode?)
- 6 PATH3 queued
- 7 PATH2 queued
- 8 PATH1 queued
- 9 Output path (1=transfer ongoing)
- 10-11 Active path
 - 0=Idle
 - 1=PATH1
 - 2=PATH2
 - 3=PATH3
- 12 Transfer direction (0=EE->GS, 1=GS->EE)
- 13-23 Unused
- 24-28 Data in GIF FIFO (in quadwords, max 16)
- 29-31 Unused

10003040h GIF_TAG0 (R)

- 0-31 Bits 0-31 of most recently read GIFtag

10003050h GIF_TAG1 (R)

- 0-31 Bits 32-63 of most recently read GIFtag

10003060h GIF_TAG2 (R)

- 0-31 Bits 64-95 of most recently read GIFtag

10003070h GIF_TAG3 (R)

- 0-31 Bits 96-127 of most recently read GIFtag

These registers are only readable when the GIF has been paused by GIF_CTRL.

For details on the GIFtag format, see

[GIFtags](#)

10003080h GIF_CNT (R)

- 0-14 Backwards loop counter from NL00P
 - Decrements to zero
- 15 Unused
- 16-19 Register descriptor in progress
 - 0=highest
 - 1=lowest
 - 2=2nd lowest
 - ...
 - 15=15th lowest
- 20-29 VU data address being transferred
- 30-31 Unused

Only accessible when GIF is paused by GIF_CTRL.

10003090h GIF_P3CNT (R)

- 0-14 Backwards loop counter from PATH3 NL00P when PATH3 is interrupted
- 15-31 Unused

Only accessible when GIF is paused by GIF_CTRL.

100030A0h GIF_P3TAG (R)

- 0-31 Bits 0-31 of PATH3 GIFtag when PATH3 is interrupted

Only accessible when GIF is paused by GIF_CTRL.

GIFtags

The basic unit of GIF transfer data is the GIF packet. Each packet is split into one or more primitives. Every primitive must be preceded by a GIFtag.

GIFtag Format

0-14	NLOOP - Data per register to transfer
15	EOP - End of packet
16-45	Unused
46	Enable PRIM field
47-57	Data to be sent to GS PRIM register if GIFtag.46 == 1
58-59	Data format
	0=PACKED
	1=REGLIST
	2=IMAGE
	3=IMAGE
60-63	NREGS - Number of registers
	0=16 registers
64-127	Register field, 4 bits each

When NLOOP == 0, all fields are ignored except EOP and no further processing is done.

When GIFtag.46 == 0, an idle cycle is inserted before processing begins.

Registers are handled in little-endian order; i.e. bits 64-67 are processed first, then 68-71, and so on.

NOTE: The GS Q register is initialized to 1.0f when reading a GIFtag.

GIF Data Formats

The data format after a GIFtag varies depending on the value of GIFtag.58-59. The GIF can transfer to the GS in PACKED, REGLIST, or IMAGE formats.

PACKED Format

Data is transferred in units of quadwords (16 bytes). Total amount of data in the GIF primitive = NLOOP * NREGS.

All registers not specified below output the lower 64-bits of the quadword to their GS register directly. The upper 64-bits are discarded.

Current reg=0h PRIM

0-10	Data to write to GS PRIM
11-127	Unused

Current reg=1h RGBA

Writes to RGBAQ register (Q is unchanged)

0-7	R
8-31	Unused
32-39	G
40-63	Unused
64-71	B
72-95	Unused
96-103	A
104-127	Unused

R, G, B, and A are 8-bit values. Q is set by the STQ command.

Current reg=2h STQ

Writes to ST register and Q component of RGBAQ.

0-31	S
32-63	T
64-95	Q
96-127	Unused

S, T, and Q are single-precision (32-bit) floats in IEEE 754 format.

Current reg=3h UV

0-13	U
14-31	Unused
32-45	V
46-127	Unused

U and V are 14-bit unsigned fixed-point integers with a 4-bit fractional component.

Current reg=4h XYZ2F/XYZ3F

0-15	X
16-31	Unused
32-47	Y
48-67	Unused
68-91	Z
92-99	Unused
100-107	F
108-110	Unused
111	Disable drawing (1=write to XYZ3F, 0=write to XYZ2F)
112-127	Unused

X and Y are signed 16-bit fixed-point integers with a 4-bit fractional component.

Z is a 24-bit integer, and F is an 8-bit integer.

The "disable drawing" bit controls whether the data is written to XYZ2F or XYZ3F.

Current reg=5h XYZ2/XYZ3

0-15	X
16-31	Unused
32-47	Y
48-63	Unused
64-95	Z
96-110	Unused
111	Disable drawing (1=write to XYZ3F, 0=write to XYZ2F)
112-127	Unused

Similar to XYZ2F/XYZ3F above, except there is no F register and Z is 32-bit.

Current reg=Ah FOG

0-99	Unused
100-107	F
108-127	Unused

Current reg=Eh A+D (output data to specified address)

0-63 Data
 64-71 Register address
 72-127 Unused

Outputs data to the given register address.

WARNING: Do not use this register descriptor on PATH3 when PATH1 is also running, or the GS may lock up!

Current reg=Fh NOP

Data not output to GS.

REGLIST Format

Total data in GIF primitive = NREGS * NLOOP, in units of doublewords. (64-bits)

In REGLIST, a doubleword is sent directly to the register descriptor. Every quadword contains data for two registers. When the data is prepared properly, this can multiply data density by up to a factor of 2.

When NREGS * NLOOP is odd, the last doubleword in a primitive is discarded.

IMAGE Format

Total data in GIF primitive = NLOOP only, in units of quadwords.

IMAGE is a shortcut for writing to the GS HWREG register, which transfers textures and other data to VRAM. Each quadword has enough data for two writes to HWREG.

GIF PATH3 Masking

PATH1 and PATH2 transfer to the GS without any internal buffering. However, PATH3 has a 16-quadword GIF FIFO.

When PATH3 masking is enabled, data sent by PATH3 will reside in the FIFO until the mask is lifted. This is useful for queueing texture transfers while other paths are sending geometry. GTA: San Andreas, Lemmings, and some other games rely on this.

Furthermore, Wallace and Gromit at Project Zoo will enable the mask and start a GIF DMA transfer, expecting it to finish. Emulating PATH3 masking without emulating the GIF FIFO will cause it to hang on a black screen, as the DMA channel is unable to write any data.

DMA Controller (DMAC)

The DMAC gives an interface for the EE to access the rest of the system. It can transfer one quadword (16 bytes) at bus speed (half the rate of the EE's clock).

DMAC Reference

[DMAC I/O](#)
[DMAC Chain Mode](#)
[DMAC MFIFO](#)
[DMAC Interrupts](#)

DMAC I/O

Channels

100080xxh VIF0
 100090xxh VIF1 (can act as PATH2 for GIF)
 1000A0xxh GIF (PATH3)
 1000B0xxh IPU_FROM
 1000B4xxh IPU_TO
 1000C0xxh SIF0 (from IOP)
 1000C4xxh SIF1 (to IOP)
 1000C8xxh SIF2 (bidirectional, used for PSX mode and debugging)
 1000D0xxh SPR_FROM
 1000D4xxh SPR_TO

1000xx00h Dn_CHCR - Channel control (R/W)

0 DIR - Direction (0=to memory, 1=from memory)
 Only effective for VIF1 and SIF2
 1 Unused
 2-3 MOD - Mode (0=normal, 1=chain, 2=interleave)
 4-5 ASP - Address stack pointer
 6 TTE - Transfer DMA tag (only in source chain mode)
 7 TIE - Enable IRQ bit in DMA tag
 8 STR - Start/busy
 9-15 Unused
 16-31 TAG - Bits 16-31 of most recently read DMA tag

1000xx10h Dn_MADR - Channel address (R/W)**1000xx30h Dn_TADR - Channel tag address (R/W)**

0-30 Address (lower 4 bits must be zero)
 31 Memory selection (0=RAM, 1=scratchpad)

The channel will read from/write to MADR directly during a transfer. TADR is only used for chain mode.

MADR and TADR update while a transfer is ongoing, i.e., the DMAC has no internal channels for memory addresses.

Certain games expect MADR and TADR to increment during a transfer.

1000xx20h Dn_QWC - Quadword count (R/W)

0-15 Quadwords
 16-31 Unused

In normal and interleaved mode, the transfer ends when QWC reaches zero. Chain mode behaves differently; see

[DMAC Chain Mode](#)

1000xx40h Dn_ASR0 - Channel saved tag address (R/W)**1000xx50h Dn_ASR1 - Channel saved tag address (R/W)**

0-30 Tag address (lower 4 bits must be zero)
 31 Memory selection (0=RAM, 1=scratchpad)

ASR0/ASR1 can only be used by the VIF0, VIF1, and GIF channels.

1000xx80h Dn_SADR - Channel scratchpad address (R/W)

- 0-13 Address (lower 4 bits must be zero)
- 14-31 Unused

SADR is only used by SPR_FROM and SPR_TO.

1000E000h D_CTRL - DMAC control (R/W)

- 0 DMA enable
- 1 Cycle stealing on
- 2-3 MFIFO drain channel
 - 0=None
 - 1=Reserved
 - 2=VIF1
 - 3=GIF
- 4-5 Stall control channel
 - 0=None
 - 1=SIF0
 - 2=SPR_FROM
 - 3=IPU_FROM
- 6-7 Stall control drain channel
 - 0=None
 - 1=VIF1
 - 2=GIF
 - 3=SIF1
- 8-10 Release cycle period
 - 0=8
 - 1=16
 - 2=32
 - 3=64
 - 4=128
 - 5=256

When cycle stealing is on, the release cycle period controls how long the EE has control of the bus.

1000E010h D_STAT - DMAC interrupt status (R/W)

- 0-9 Channel interrupt status (1=IRQ, write 1 to clear)
- 10-12 Unused
- 13 DMA stall interrupt status
- 14 MFIFO empty interrupt status
- 15 BUSERR interrupt status
- 16-25 Channel interrupt mask (1=enabled, write 1 to reverse)
- 26-28 Unused
- 29 Stall interrupt mask
- 30 MFIFO empty mask

INT1 is asserted when (status & mask) != 0.

1000E020h D_PCR - DMAC priority control (R/W)

- 0-9 COP0 condition control
- 10-15 Unused
- 16-25 Channel priority (0=Channel disabled, only if D_PCR.31 is true)
- 26-30 Unused
- 31 Priority enable

COP0 control determines whether or not CPCOND0 is activated. If ($\sim$ control | D_STAT.0-9) == 0x3FF, CPCOND0 is set. This is useful for knowing when multiple DMA transfers have finished without using interrupts.

1000E030h D_SQWC - DMAC skip quadword (R/W)

- 0-7 Quadwords to skip
- 8-15 Unused
- 16-23 Quadwords to transfer
- 24-31 Unused

In interleave mode, every time D_SQWC.16-23 quadwords have been transferred, D_SQWC.0-7 quadwords are skipped.

1000E040h D_RBSR - DMAC ringbuffer size (R/W)

- 4-30 Size of MFIFO buffer in quadwords, minus 1

1000E050h D_RBOR - DMAC ringbuffer offset (R/W)

- 0-30 Starting address of MFIFO buffer. Lower 4 bits are 0

1000F520h D_ENABLER - DMAC disabled status (R)

1000F590h D_ENABLEW - DMAC disable (W)

- 16 DMAC disabled
- All other bits appear to be garbage, but writable
 SCPH-39001 (and other BIOSes?) seems to expect D_ENABLER to be set to 1201h upon boot

DMAC Chain Mode

The DMAC can intelligently chain DMA transfers together. When a transfer starts in chain mode, the channel will transfer QWC quadwords and then read a DMAtag.

In source chain mode, the DMAtag comes from TADR. In destination chain mode, the DMAtag comes from the peripheral the channel is reading from.

DMAtag

- 0-15 QWC to transfer
- 16-25 Unused
- 26-27 Priority control
 - 0=No effect
 - 1=Reserved
 - 2=Priority control disabled (D_PCR.31 = 0)
 - 3=Priority control enabled (D_PCR.31 = 1)
- 28-30 Tag ID
- 31 IRQ
- 32-62 ADDR field (lower 4 bits must be zero)
- 63 Memory selection for ADDR (0=RAM, 1=scratchpad)
- 64-127 Data to transfer (only if Dn_CHCR.TTE==1)

When both IRQ and Dn_CHCR.TIE are set, the transfer ends after QWC has been transferred.

When Dn_CHCR.TTE is on, bits 64-127 are transferred BEFORE QWC.

The effects of the tag ID vary depending on if the channel is in source chain or dest chain mode.

Source Chain Tag ID

0	refe	MADR=DMAtag.ADDR TADR+=16 tag_end=true
1	cnt	MADR=TADR+16 (next to DMAtag) TADR=MADR (next to transfer data)
2	next	MADR=TADR+16 TADR=DMAtag.ADDR
3	ref	MADR=DMAtag.ADDR TADR+=16
4	refs	MADR=DMAtag.ADDR TADR+=16
5	call	MADR=TADR+16 if (CHCR.ASP == 0) ASR0=MADR+(QWC*16) else if (CHCR.ASP == 1) ASR1=MADR+(QWC*16) TADR=DMAtag.ADDR CHCR.ASP++
6	ret	MADR=TADR+16 if (CHCR.ASP == 2) TADR=ASR1 CHCR.ASP-- else if (CHCR.ASP == 1) TADR=ASR0 CHCR.ASP-- else tag_end=true
7	end	MADR=TADR+16 tag_end=true

When tag_end=true, the transfer ends after QWC has been transferred.

Dest Chain Tag ID

0	cnt	MADR=DMAtag.ADDR
1	cnts	MADR=DMAtag.ADDR
7	end	MADR=DMAtag.ADDR tag_end=true

DMA Resuming

If a transfer starts in source chain mode and $QWC > 0$, the DMAC assumes that the TAG field of CHCR was the last read DMAtag. This means that if the tag ID is REFE or END, the channel will stop after QWC has been transferred.

Sony's movie-playing library requires this behavior, as it uses MADR to determine the current movie buffer position.

DMAC MFIFO

Scratchpad memory is much faster than main memory, but the scratchpad is a measly 16 KB, only enough for 1024 quadwords of data.

The Memory FIFO (MFIFO) works around this issue by automatically transferring data sent via SPR_FROM to a user-defined buffer in main memory. The D_RBSR and D_RBOR registers define the size and location of this buffer. SPR_FROM's MADR is automatically set to the next read position in the buffer, wrapping around when the end is reached.

Either the VIF1 DMA channel or GIF DMA channel can be the drain channel, reading from the buffer when SPR_FROM transfers data to it. When a new tag is read, the drain channel's TADR is set to $D_RBOR | (TADR \& D_RBSR)$, as is MADR if QWC is non-zero and the tag ID is not REF/REFE/REFS.

When TADR equals SPR_FROM's MADR, the drain channel is stalled, and an MFIFO empty interrupt is triggered. If the tag ID is REF/REFE/REFS and QWC is non-zero, an interrupt is still sent but the channel is not stalled.

At first, it may seem odd to use MFIFO, because the extra transfer is slower than just interacting with VIF1/GIF DMA directly. However, VIF1 and the GIF have a tendency to stall for long periods of time when transferring data to the GS because of VU1 transferring data to PATH1. MFIFO allows the EE to queue up data without worrying about GIF stalls and managing DMA buffers, which in turn allows the EE to do other things more efficiently.

DMAC Interrupts

Upon any transfer completion, the DMAC will raise the channel's stat flag in D_STAT. When $(stat \& mask) \neq 0$, the DMAC asserts an INT1 signal to the EE.

INT1 may also be asserted when MFIFO is empty, a stall occurs, or a bus error occurs. The bus error is not maskable via D_STAT.

NOTE: INT0 (INTC) has priority over INT1. If INT0 and INT1 are asserted at the same time, the INT0 interrupt will be processed first.

Graphics Synthesizer (GS)

GS Reference

[GS Register List](#)

[GS Primitives](#)

[GS Vertex Attributes](#)

[GS Frame and Z Buffers](#)

[GS Transfers](#)

[GS Textures](#)

[GS Fog](#)

[GS Alpha Blending](#)

[GS Tests and Pixel Control](#)

GS Register List

Internal registers (accessible via GIF)

00h	PRIM
01h	RGBAQ
02h	ST
03h	UV
04h	XYZF2
05h	XYZ2
06h/07h	TEX0_1/2
08h/09h	CLAMP_1/2
0Ah	FOG
0Ch	XYZF3
0Dh	XYZ3
14h/15h	TEX1_1/2
16h/17h	TEX2_1/2
18h/19h	XYOFFSET_1/2
1Ah	PRMODECONT
1Bh	PRMODE
1Ch	TEXCLUT
22h	SCANMSK
34h/35h	MIPTBP1_1/2
36h/37h	MIPTBP2_1/2
3Bh	TEXA
3Dh	FOGCOL
3Fh	TEXFLUSH
40h/41h	SCISSOR_1/2
42h/43h	ALPHA_1/2
44h	DIMX
45h	DTHE
46h	COLCLAMP
47h/48h	TEST_1/2
49h	PABE
4Ah/4Bh	FBA_1/2
4Ch/4Dh	FRAME_1/2
4Eh/4Fh	ZBUF_1/2
50h	BITBLTBUF
51h	TRXPOS
52h	TRXREG
53h	TRXDIR
54h	HWREG
60h	SIGNAL
61h	FINISH
62h	LABEL

Privileged registers (accessible via EE)

12000000h	Pmode
12000020h	Smode2
12000070h	DISPFB1
12000080h	DISPLAY1
12000090h	DISPFB2
120000A0h	DISPLAY2
120000B0h	EXTBUF
120000C0h	EXTDATA
120000D0h	EXTWRITE
120000E0h	BGcolor
12001000h	CSR
12001010h	IMR
12001040h	BUSDIR
12001080h	SIGLBLID

GS Primitives

The GS supports point, line, triangle, and sprite (2D rectangle with two points) primitives.

00h PRIM

0-2	Primitive type
0	Point
1	Line
2	LineStrip
3	Triangle
4	TriangleStrip
5	TriangleFan
6	Sprite
7	Reserved
3	Gourand shading
4	Texture mapping
5	Fog
6	Alpha blending
7	Antialiasing
8	Use UV for texture coordinates (0=Use STQ)
9	Use Context 2's registers for drawing (0=Use Context 1)
10	Fix fragment value (related to antialiasing?)

1Ah PRMODECONT

When this register is set to 1, bits 3-10 in PRIM are used when rendering primitives. Otherwise, bits 3-10 in PRMODE are used.

1Bh PRMODE

Same as PRIM, except bits 0-2 are unused.

PRMODE can be useful when you want to draw multiple primitives with different attributes but the same type.

GS Vertex Attributes

01h RGBAQ

0-7 Red
 8-15 Green
 16-23 Blue
 24-31 Alpha
 32-63 Q

These attributes are applied to newly created vertices.

04h/0Ch XYZF2/XYZF3

0-15 X
 16-31 Y
 32-55 Z
 56-63 Fog coefficient (placed in FOG register)

This defines a new vertex to be placed in the vertex queue, a process known as the vertex kick. X and Y are 12-bit fixed-point integers with a 4-bit fractional component, ranging from 0 to 4095.9375.

When 1, 2, or 3 vertices have been defined as per the primitive type, a "drawing kick" occurs, where the GS begins drawing the primitive.

NOTE: Drawing kicks can only occur on XYZF2! XYZF3 will clear the queue as needed but will not draw any primitives. XYZF3 exists to allow the programmer to easily cull polygons, as the GS has no native support for culling.

See [GS Fog](#) for details on fog.

05h/0Dh XYZ2/XYZ3

0-15 X
 16-31 Y
 32-63 Z

Same as XYZF2/XYZF3, except Z is a 32-bit unsigned integer rather than a 24-bit unsigned integer.

XYZ2 can also result in a drawing kick, whereas XYZ3 cannot.

18h/19h XYOFFSET_1/2

0-15 X
 32-47 Y

When a vertex kick occurs, XYOFFSET is subtracted from the vertex's X and Y coordinates.

GS Frame and Z Buffers

4Ch/4Dh FRAME_1/2

0-8 Base pointer in words/2048
 16-21 Buffer width in pixels/64
 24-29 Format
 00h=PSMCT32
 01h=PSMCT24
 02h=PSMCT16
 0Ah=PSMCT16S
 30h=PSMZ32
 31h=PSMZ24
 32h=PSMZ16
 3Ah=PSMZ16S
 32-63 Framebuffer mask

Bits 32-63 prevent the specified bits in the framebuffer from being updated by the following formula:

$\text{final_color} = (\text{final_color} \& \sim\text{mask}) \mid (\text{frame_color} \& \text{mask})$

This mask is applied before format conversions, e.g., bit 63 of FRAME will affect bit 15 (alpha bit) of a 16-bit color.

4Eh/4Fh ZBUF_1/2

0-8 Base pointer in words/2048
 24-27 Format
 00h=PSMZ32
 01h=PSMZ24
 02h=PSMZ16
 0Ah=PSMZ16S
 32 Buffer mask (1=do not update zbuffer)

The zbuffer's width is the same as the framebuffer's, specified in FRAME.

GS Transfers

The GS supports GIF->VRAM, VRAM->VRAM, and VRAM->GIF data transfers.

50h BITBLTBUF

0-13 Source base pointer in words/64
 16-21 Source buffer width in pixels/64
 24-29 Source format
 00h=PSMCT32
 01h=PSMCT24
 02h=PSMCT16
 0Ah=PSMCT16S
 13h=PSMCT8
 14h=PSMCT4
 18h=PSMCT8H
 24h=PSMCT4HL
 2Ch=PSMCT4HH
 30h=PSMZ32
 31h=PSMZ24
 32h=PSMZ16
 3Ah=PSMZ16S
 32-45 Destination base pointer in words/64
 48-53 Destination buffer width in pixels/64
 56-61 Destination format (same as source format)

In VRAM->VRAM transfers, the source and destination formats must have the same bits per pixel.

51h TRXPOS

0-10 X for source rectangle
 16-26 Y for source rectangle
 32-42 X for destination rectangle

48-58 Y for destination rectangle
 59-60 Transmission order for VRAM->VRAM transfers
 0=Upper-left->lower-right
 1=Lower-left->upper-right
 2=Upper-right->lower-left
 3=Lower-right->upper-left

X and Y are in units of pixels and define the upper-left corner of their respective rectangle.

NOTE: During transfer, X and Y wrap around if they exceed 2048, e.g., by the following formula:

$X = (TRXPOS.X + TRXREG.width) \% 2048$

52h TRXREG

0-11 Width in pixels of transmission area
 32-43 Height in pixels of transmission area

53h TRXDIR

0-1 Transmission direction
 0=GIF->VRAM
 1=VRAM->GIF
 2=VRAM->VRAM
 3=Deactivated

Note that the privileged register BUSDIR must be set appropriately for GIF->VRAM and VRAM->GIF.

54h HWREG

0-63 Data to be transferred for GIF->VRAM

The "IMAGE" GIFtag format is a shortcut for writing to this register. Data is packed according to the format. For example, PSMCT4 will have 16 4-bit pixels per doubleword.

GS Textures

06h/07h TEX0_1/2

0-13 Base pointer in words/64
 14-19 Buffer width in pixels/64
 20-25 Texture format
 00h=PSMCT32
 01h=PSMCT24
 02h=PSMCT16
 0Ah=PSMCT16S
 13h=PSMCT8
 14h=PSMCT4
 1Bh=PSMCT8H
 24h=PSMCT4HL
 2Ch=PSMCT4HH
 30h=PSMZ32
 31h=PSMZ24
 32h=PSMZ16
 3Ah=PSMZ16S
 26-29 Texture width (width = min(2^{value} , 1024))
 30-33 Texture height (height = min(2^{value} , 1024))
 34 Alpha control (0=texture is RGB, 1=texture is RGBA)
 35-36 Color function
 0=Modulate
 1=Decal
 2=Highlight
 3=Highlight2
 37-50 CLUT base pointer in words/64
 51-54 CLUT format
 00h=PSMCT32
 02h=PSMCT16
 0Ah=PSMCT16S
 55 CLUT uses CSM2 (0=CSM1)
 56-60 CLUT entry offset/16 (In CSM2, this value must be 0)
 61-63 CLUT cache control
 0=Do not reload cache
 1=Reload cache
 2=Reload cache and copy CLUT base pointer to CBP0
 3=Reload cache and copy CLUT base pointer to CBP1
 4=IF CLUT base pointer != CBP0, reload cache and copy pointer to CBP0
 5=IF CLUT base pointer != CBP1, reload cache and copy pointer to CBP1

14h/15h TEX1_1/2

0 LOD (level of detail) calculation method
 0=LOD=($\log_2(1/\text{abs}(Q)) \leq 0$ (reduced texture)
 0=nearest
 1=bilinear
 2=nearest_mipmap_nearest
 3=nearest_mipmap_bilinear
 4=bilinear_mipmap_nearest
 5=bilinear_mipmap_bilinear
 9 Automatic calculation of mipmap levels 1-3 (0=use MIPTBP1)
 19-20 L parameter
 32-43 K parameter (signed fixed-point, 7 bits whole, 4 bits fractional)

16h/17h TEX2_1/2

20-25 Texture format (same as TEX0)
 37-50 CLUT base pointer in words/64
 51-54 CLUT format
 55 CLUT uses CSM2 (0=CSM1)
 56-60 CLUT entry offset/16
 61-63 CLUT cache control (same as TEX0)

TEX2 is a subset of TEX0. This is useful for modifying texture format and CLUT information when the texture base pointer, width, etc. must stay the same.

3Fh TEXFLUSH

Writing any value to this register will invalidate the texture cache. Do this in the following situations:

- Using newly transferred texture data
- Using newly transferred CLUT data or reloading the CLUT cache
- Using framebuffer or zbuffer data as textures

Texture coordinates

The GS supports STQ texture coordinates and UV texel coordinates. The relationship between the two is as follows.

$$U = (S/Q) * \text{TEX0.texwidth}$$

$$V = (T/Q) * \text{TEX0.texheight}$$

Q is defined by RGBAQ and used for perspective correction. It is not possible to use perspective correction when directly using UV coordinates.

02h ST

0-31 S (lower 8 bits are rounded down to zero)

32-63 T (lower 8 bits are rounded down to zero)

S and T are (mostly) IEEE 754-compliant single-precision floating-point values. For both S and T, the range [0.0, 1.0] refers to the whole texture.

03h UV

0-13 U

16-29 V

U and V are unsigned 10-bit fixed-point integers with a 4-bit fractional component. The ranges [0, TEXWIDTH] and [0, TEXHEIGHT] for U and V respectively refer to the whole texture.

Color function TEX0 defines four possible color functions that can be used to blend texture and vertex colors:

	RGB	RGBA
Modulate	$R_v = (R_v * R_t) \gg 7$ $G_v = (G_v * G_t) \gg 7$ $B_v = (B_v * B_t) \gg 7$ $A_v = A_v$	$<- \text{Same}$ $<- \text{Same}$ $<- \text{Same}$ $A_v = (A_v * A_t) \gg 7$
Decal	$R_v = R_t$ $G_v = G_t$ $B_v = B_t$ $A_v = A_v$	$<- \text{Same}$ $<- \text{Same}$ $<- \text{Same}$ $A_v = A_t$
Highlight	$R_v = ((R_v * R_t) \gg 7) + A_v$ $G_v = ((G_v * G_t) \gg 7) + A_v$ $B_v = ((B_v * B_t) \gg 7) + A_v$ $A_v = A_v$	$<- \text{Same}$ $<- \text{Same}$ $<- \text{Same}$ $A_v = A_t + A_v$
Highlight2	Same as Highlight	Same as Highlight, but $A_v = A_t$

GS Fog

3Dh FOGCOL

0-7 R

8-15 G

16-23 B

FOGCOL represents the color of a "distant" object, or one enshrouded in fog. See below for details.

0Ah FOG

0-7 Fog effect

FOG gives a fog effect F to the current vertex. XYZ2F/XYZ3F also modify this register.

During rasterization, if fog is enabled in PRIM/PRMODE, F is linearly interpolated for all vertices. The fog equation is then applied to an outputted texture color after the color function has been applied:

$$\text{Output} = ((F * \text{input}) \gg 8) + (((255 - F) * \text{FOGCOL}) \gg 8)$$

So a value of 0xFF results in no change and a value of 0 completely converts the color to FOGCOL.

You can see that fog equation is just another alpha blending equation.

GS Alpha Blending

42h/43h ALPHA_1/2

0-1 Spec A

2-3 Spec B

4-5 Spec C

6-7 Spec D

8-15 Alpha FIX

The GS's alpha blending formula is fixed but it contains four variables that can be reconfigured:

$$\text{Output} = (((A - B) * C) \gg 7) + D$$

A, B, and D are colors and C is an alpha value. Their specific values come from the ALPHA register:

	A	B	C	D
0	Source RGB	Source RGB	Source alpha	Source RGB
1	Framebuffer RGB	Framebuffer RGB	Framebuffer alpha	Framebuffer RGB
2	0	0	FIX	0
3	Reserved	Reserved	Reserved	Reserved

Internally, alpha-blending treats each color component as 9-bit. The output is then clamped accordingly by COLCLAMP.

46h COLCLAMP

0 8-bit signed clamp (0=8-bit AND)

When COLCLAMP is 1, RGB components will be 0 if negative after alpha-blending or 0xFF if 0x100 or above. Otherwise, each color component will be ANDed with 0xFF.

GS Tests and Pixel Control

40h/41h SCISSOR_1/2

0-10 X0

16-26 X1

32-42 Y0

48-58 Y1

SCISSOR defines a rectangle with 11-bit unsigned integer coordinates, which range from 0 to 2047. During drawing, pixels that fall outside the boundaries of this rectangle fail automatically and are not processed.

47h/48h TEST_1/2

0 Alpha test enabled

1-3 Alpha test method

0=NEVER (all pixels fail)

1=ALWAYS (all pixels pass)

```

2=LESS (pixel alpha < AREF passes)
3=LEQUAL (pixel alpha <= AREF passes)
4=EQUAL (pixel alpha == AREF passes)
5=GEQUAL (pixel alpha >= AREF passes)
6=GREATER (pixel alpha > AREF passes)
7=NEQUAL (pixel alpha != AREF passes)
4-11 AREF
12-13 Alpha test failure processing
0=Neither framebuffer nor zbuffer are updated.
1=Only framebuffer is updated.
2=Only zbuffer is updated.
3=Only RGB in framebuffer is updated.
14 Destination alpha test enabled
15 Destination alpha test method
0=destination alpha bit == 0 passes
1=destination alpha bit == 1 passes
16 Depth test enabled (0 is prohibited?)
17-18 Depth test method
0=NEVER (all pixels fail)
1=ALWAYS (all pixels pass)
2=GEQUAL (pixel Z >= zbuffer Z passes)
3=GREATER (pixel Z > zbuffer Z passes)

```

Note on destination alpha test:

The alpha bit tested depends on the framebuffer format. If the format is PSMCT32, bit 7 of alpha is tested. If the format is PSMCT16, the sole alpha bit is tested. If the format is PSMCT24, all pixels pass due to the lack of alpha.

GS Special Effects and Optimizations

The GS has an absurdly high fillrate of 1.2 gigapixels/second when drawing textured polygons. This advantage is further compounded by free (!) alpha blending and depth buffering. However, the GS only supports single-pass texturing and is entirely fixed-unit. The combination of these factors led developers to find creative ways to make their games look good...

Note: many of these effects don't have an official name, so they are made up.

Fast Screen Draw

Drawing a single sprite that covers the entire screen is relatively slow. First, this causes many DRAM page breaks per scanline drawn. Second, for textured sprites, the texture cache also has to be reloaded many times.

Instead, one can split a screen draw into many 64x32 sprites. This greatly improves cache locality, based upon the GS's nonlinear mmemory.

The fast screen draw is often used as a building block for other advanced techniques.

Double Half Clear

Because the framebuffer and depth buffer both occupy the same memory space, framebuffer clears can be optimized by placing the start of the depth buffer halfway down the screen and halving the height of the clear draw. As depth buffering is free, this will result in the clear being around twice as fast.

Interleaved Clear

Similar to the double half clear in principle, but takes advantage of color and depth formats being swizzled differently. The depth buffer base is set to the framebuffer base (ZBP=FBP), and instead of the clear being split into 64x32 sprites, it is split into 32x32 sprites instead. This results in the clear being interleaved, as the color clear touches the left side of a 64x32 block and the depth clear touches the right side. Because the framebuffer and depth buffer share the same cache, this should be faster than a double half clear.

Powerdrome uses a crazy variant of this, where the framebuffer is given a Z format. Due to quirks in the GS rendering process, this causes the depth clear to draw as if it had a color format. This doesn't really speed anything up, as all that changes is the order of the interleave, but it does make emudevcs cry a lot.

VIS Clear

Absolutely insane clear used in the VIS Games engine. The framebuffer width (FBW) is set to 1 (64 pixels for a 32-bit texture), then a very thin and tall sprite is drawn. In normal cases, the sprite is 64x2048. The clear is essentially if one took all the pages from the framebuffer, stacked them vertically, then drew a sprite large enough to clear the stack. This should avoid the horizontal page breaks that other methods cause, but it is unknown how fast this clear is.

Superman Returns, another VIS game, uses a somehow wilder variant, where the clear sprite is 32x4096. The sprite is so large, it wraps around from the bottom of VRAM to the top.

Recursive Drawing

Draws where the texture buffer and framebuffer overlap, usually done with TBP = FBP. Because of the texture cache, texels do not get overwritten in VRAM until after they are read. Recursive drawing can be used to apply special effects without having to create a temporary buffer in memory.

Channel Shuffle

A sophisticated technique made to overcome the GS's lack of shader units. At a high level, the most basic channel shuffle effect uses a texture's 32-bit RGBA channels as 8-bit palette indices to apply an effect on the texture, such as gamma correction or a brightness effect.

To perform the channel shuffle, draw a series of 8x2 sprites that read the 32-bit texture as 8-bit - normally this is done with recursive drawing, but temporary buffers can also be used. The size of the sprites is important, as 32-bit and 8-bit textures are swizzled differently. The sprites will read from a palette which applies the effect, then FBMASK is used to restrict the draw to a single channel. This must be repeated for every channel the effect is to be applied to, with a total of four shuffles for all four channels.

This effect is extremely difficult to emulate with a modern GPU. A 640x224 framebuffer requires 8960 8x2 sprites for a single shuffle. The 32-bit texture must be deswizzled, converted to 8-bit, and reconverted back to 32-bit for each sprite. Since this is usually done with recursive drawing, texture memory and framebuffer memory on the host GPU also needs to be synchronized. Finally, modern GPUs mask on the byte level, but games can mask on the bit level, which requires a slow fragment shader to process the effect correctly if the game doesn't fully mask on the byte level. All this results in a channel shuffle being more expensive than even every other draw call in the game. Other variants of channel shuffle also exist that involve reading the depth buffer, and some shuffle effects are not well-understood.

Vector Interface (VIF)

VIF0 and VIF1 are DMA interfaces for the Vector Units. Their main purpose is to upload microprograms and decompress vectors into VU micro memory and data memory respectively.

VIF1 can also transfer data to the GIF through PATH2 and mask PATH3.

[VIF I/O Registers](#)

[VIF Commands](#)

[VIF UNPACK](#)

VIF I/O Registers

Most VIF registers are read-only unless specified otherwise.

10003800h/10003C00h VIFn_STAT

- 0-1 VPS - VIF command status
 - 0=Idle
 - 1=Waiting for data following command
 - 2=Decoding command
 - 3=Decompressing/transferring data
- 2 VEW - VU is executing microprogram
- 3 VGW - Stalled waiting for GIF (VIF1 only)
- 6 MRK - MARK detected
- 7 DBF - Double buffer flag (VIF1 only)
 - 0=TOPS = BASE
 - 1=TOPS = BASE + OFST
- 8 VSS - Stalled after STOP was sent to FBRST
- 9 VFS - Stalled after force break was sent to FBRST
- 10 VIS - Stalled on interrupt ibt
- 11 INT - Interrupt bit detected
- 12 ER0 - DMAtag mismatch error (don't know what this means)
- 13 ER1 - Invalid VIF command was sent
- 23 FDR - FIFO direction (VIF1 only)
 - 0=Memory -> VIF FIFO
 - 1=VIF FIFO -> Memory
- 24-28 FQC - Amount of quadwords in FIFO
 - Max 8 for VIF0, 16 for VIF1

10003810h/10003C10h VIFn_FBRST

- 0 RST - Reset VIF including contents of FIFO when written to
- 1 FBK - Force break the VIF, causing an immediate stall
- 2 STP - STOP the VIF, stalling it after it finishes the current command
- 3 STC - Stall cancel. Clears VSS, VFS, VIS, INT, ER0, and ER1 in VIFn_STAT

This is a write-only register used to control VIF resets and stalls.

10003820h/10003C20h VIFn_ERR

- 0 MII - Disable interrupt bit stalls and interrupts if set
- 1 ME0 - Disable DMAtag mismatch error and stall if set
- 2 ME1 - Disable invalid command error and stall if set

ME0 should always be set to 1. ME1 should be set to 1 when doing UNPACK V3-16.

10003830h/10003C30h VIFn_MARK

- 0-15 Most recently set MARK value

This register can be written to by the EE. Writes clear the MRK flag in VIFn_STAT.

10003840h/10003C40h VIFn_CYCLE

- 0-7 CL - Cycle length
- 8-15 WL - Write cycle length

10003850h/10003C50h VIFn_MODE

- 0-1 Addition mode used for UNPACK

10003860h/10003C60h VIFn_NUM

- 0-7 Amount of untransferred data in MPG/UNPACK

10003870h/10003C70h VIFn_MASK

- 0-31 Write mask matrix

10003880h/10003C80h VIFn_CODE

- 0-15 IMMEDIATE value processed most recently
- 16-23 NUM value processed most recently
- 24-31 CMD value processed most recently

This register contains the last processed command or if the VIF was stalled while processing a command, the command currently being processed.

10003890h/10003C90h VIFn_ITOPS

- 0-9 ITOPS value

10003CA0h VIF1_BASE

- 0-9 BASE value

10003CB0h VIF1_OFST

- 0-9 OFST value

10003CC0h VIF1_TOPS

- 0-9 TOPS value

100038D0h/10003CD0h VIFn_ITOP

- 0-9 ITOP value

10003CE0h VIF1_TOP

- 0-9 TOP value

10003900h-10003930h/10003D00h-10003D30 VIFn_RN

Array of 4 32-bit values used for row filling data. Each value is stored at 100039N0h/10003DN0h.

10003940h-10003970h/10003D40h-10003D70 VIFn_CN

Same as VIFn_RN, except for column filling data.

VIF Commands

Command Format

31	24	23	16	15	0
	CMD		NUM		IMMEDIATE

Bit 31 is an interrupt bit - when set, a VIF interrupt is generated after the command executes, and the VIF stalls on the following command except for the MARK command.

NUM and IMMEDIATE are command-specific and are not always used.

Command List**00h NOP**

Does nothing. Mainly used for timing PATH3 masking and aligning the DMA stream for certain commands.

01h STCYCL

Sets the CYCLE register to IMMEDIATE. In particular, CYCLE.CL is set to bits 0-7 and CYCLE.WL is set to bits 8-15. The CYCLE register is used for skipping/filling writes for UNPACK.

02h OFFSET (VIF1)

Sets the OFST register to bits 0-9 of IMMEDIATE. This is used for VIF1 double buffering. DBF (the double buffering flag) is also cleared to 0 in STAT, and BASE is set to TOPS.

03h BASE (VIF1)

Sets the BASE register to bits 0-9 of IMMEDIATE. This is used for VIF1 double buffering.

04h ITOP

Sets the ITOP register to bits 0-9 of IMMEDIATE. ITOP can be read by the XITOP instruction on the VU.

05h STMOD

Sets the MODE register to bits 0-1 of IMMEDIATE. This is used for addition decompression in UNPACK.

06h MSKPATH3 (VIF1)

Sets the VIF-side PATH3 mask to bit 15 of IMMEDIATE. When PATH3 masking is enabled, the mask is applied to the next data block for PATH3.

07h MARK

Sets the MARK register to IMMEDIATE.

10h FLUSHE

Stalls the VIF until the VU is finished executing a microprogram.

11h FLUSH (VIF1)

Stalls VIF1 until the VU is finished executing a microprogram and PATH1 and PATH2 are not active.

13h FLUSHA (VIF1)

Stalls VIF1 until the VU is finished executing a microprogram, PATH1 and PATH2 are not active, and there is no pending transfer request for PATH3.

14h MSCAL

Starts a microprogram on the VU at the given address IMMEDIATE*8. If the VU is currently active, MSCAL stalls until the VU is finished before executing a new microprogram.

15h MSCALF (VIF1)

Same as MSCAL, but also waits for PATH1 and PATH2 to not be active before starting a microprogram.

17h MSCNT

Starts microprogram execution starting at the VU's TPC register - this usually means the instruction right after the end of the previous microprogram. If the VU is currently active, MSCNT stalls like MSCAL.

20h STMASK

Sets the MASK register to the next 32-bit word in the stream. This is used for UNPACK write masking.

30h STROW

Sets the R0-R3 row registers to the next 4 32-bit words in the stream. This is used for UNPACK write filling.

31h STCOL

Sets the C0-C3 column registers to the next 4 32-bit words in the stream. This is used for UNPACK write filling.

4Ah MPG

Loads NUM*8 bytes into VU micro memory, starting at the given address IMMEDIATE*8. If the VU is currently active, MPG stalls until the VU is finished before uploading data.

If NUM is 0, then 2048 bytes are loaded.

50h DIRECT (VIF1)

Transfers IMMEDIATE quadwords to the GIF through PATH2. If PATH2 cannot take control of the GIF, the VIF stalls until PATH2 is activated. If IMMEDIATE is 0, 65,536 quadwords are transferred.

51h DIRECTHL (VIF1)

Same as DIRECT, except DIRECTHL is not able to interrupt PATH3 in IMAGE mode and stalls if PATH3 is transferring in IMAGE mode.

60h-7Fh UNPACK

Decompresses data in various formats to the given address in bits 0-9 of IMMEDIATE multiplied by 16.

If bit 14 of IMMEDIATE is set, the decompressed data is zero-extended. Otherwise, it is sign-extended.

If bit 15 of IMMEDIATE is set, TOPS is added to the starting address. This is only applicable for VIF1.

Bits 0-3 of CMD determine the type of UNPACK that occurs. See [VIF UNPACK](#) for details.

Bit 4 of CMD performs UNPACK write masking if set.

0h/70h UNPACK S-32

Output								
w		z		y		x		
128	-----	96	-----	64	-----	32	-----	0
	S1		S1		S1		S1	
	S2		S2		S2		S2	
	S3		S3		S3		S3	

[illegible]

```

Output
w          z          y          x          0
128-----96-----64-----32-----0
|Indetermin.|Indetermin.| V2      | V1      |
-----

```

```

Output
w          z          y          x          0
128-----96-----64-----32-----0
|Indetermin.| |Indetermin.| ext | V2 | ext | V1 |
-----

```

Output				
w	z	y	x	
128	96	64	32	0
Indetermin.	V3	V2	V1	

```

Output
w          z          y          x          0
128-----96-----64-----32-----0
|Indetermin.| ext | V3 | ext | V2 | ext | V1 |
-----

```

Output

w z y x

128-----96-----64-----32-----0

V4	V3	V2	V1
----	----	----	----

6Dh/7Dh UNPACK V4-16

Input

96	80	64	48	32	0
V4	V3	V2	V1	cmd	

Output

w	z	y	x
128	96	64	32
ext	V4	ext	V3
		ext	V2
			ext
			V1

6Eh/7Eh UNPACK V4-8**6Fh/7Fh UNPACK V4-5**

Vector Unit (VU)

The EE contains two Vector Units (VU0 and VU1), custom SIMD processors designed for fast floating-point manipulation. Both VUs can run concurrently with the EE Core in micro mode. VU0 is also available in macro mode as COP2.

VU Reference[VU Architecture](#)[VU Registers](#)[VU Instruction Format and Decoding](#)[VU Pipelining](#)

VU Architecture

Custom SIMD floating-point processors designed by Sony

Speed: 294.912 MHz (same as EE)

Executes two instructions per cycle through upper and lower pipelines - each pipeline is specialized and can only execute certain instructions

4/16 KB of instruction ("micro") memory for VU0 and VU1 respectively

4/16 KB of data memory for VU0 and VU1 respectively

32 128-bit vector registers, 16 16-bit integer registers, and an assortment of special registers

FDIV unit: Used for division and square root operations

Elementary Function Unit (EFU): Exclusive to VU1. Used for complex calculations such as square of sums, sine, and e^x

MAC/CLIP flags: Processor flags used to compare the results of floating-point arithmetic

XGKICK: Exclusive to VU1. Transfers data directly to the GIF through PATH1

Similar decoding and quirks to standard MIPS, such as branch delay slots

VU Registers

General-purpose registers

Each VU contains 32 vector floating-point registers (vf00-vf31) and 16 16-bit integer registers (vi00-vi15).

A vector register has 4 32-bit single-precision floating-point elements: {x, y, z, w}

vi00 is hardwired to 0. vf00 is hardwired to the vector {0.0, 0.0, 0.0, 1.0}. That is to say, vf00.w = 1.0.

Accumulator (ACC)

The accumulator is a special register with the same format as the 32 general-purpose vector registers.

It is intended to be used as an intermediate result by certain instructions.

Q and P

Q and P are 32-bit floating point registers. Operations that use the FDIV or EFU units store their result in the Q and P registers respectively.

For example, a DIV stores its result in Q, and ESQRT stores its result in P.

MAC Flags

Each FMAC unit (corresponding to a vector field) has four MAC flags, which can be read by various instructions.

The MAC flags are overflow, underflow, sign, and zero. The full 16-bit MAC flag register has this format:

15	0
0x	0y
0z	0w
0x	0y
0z	0w
0x	0y
0z	0w
0x	0y
0z	0w
0x	0y
0z	0w
0x	0y
0z	0w
0x	0y
0z	0w

Zero is set when the result is 0.0f or -0.0f (which also clears overflow and underflow), and sign is set when the result is negative (bit 31 is set).

Overflow and underflow are set when the exponent field is 0xFF or 0x0 respectively.

IMPORTANT: MAC flags are modified in the "writeback" stage in the FMAC pipeline. However, an instruction like FMAND will read the flags in the register read stage, and this does not constitute a hazard.

This means every modification to the MAC flags has a delay of four cycles (not necessarily four instructions).

Clip Flags

The clipping flags register is 24-bit and contains the result of up to four CLIP instructions. Every CLIP instruction produces six flags: -x, +x, -y, +y, -z, +z. They are stored in the following format:

5	0
-z	+z
-y	+y
-x	+x

Like the MAC flags, the clip flags are also pipelined (and thus delayed).

Status Flags

11	0
DS	IS
OS	US
SS	ZS
D	I
0	U
S	Z

The Z, S, U, and O flags are set when any of the field flags for zero, sign, underflow, and overflow are set in the MAC register, respectively.

The I flag is an invalid flag, set to 1 when 0/0 is executed by DIV or when a negative number is used for SQRT/RSQRT.

The D flag is set for division by zero (except for 0/0) on DIV/RSQRT and is always cleared on SQRT.

Bits 6-11 are sticky flags - they do not represent the current state but rather the ORed result of all previous states.

If Z goes from 1 to 0 for instance, ZS will always remain 1 until the status register is written to.

VU Instruction Format and Decoding

Each VU executes two instructions per cycle - one instruction is in the upper pipeline and the other is in the lower pipeline. A full VU instruction can be considered as a 64-bit doubleword, where the lower 32-bit word is the lower instruction and the upper word is the upper instruction.

Each pipeline is specialized and can only execute certain instructions. The upper pipeline mostly executes floating-point arithmetic instructions, and the lower pipeline mostly executes integer calculations, branches, and some specialized floating-point operations.

The upper instructions can also control various parts of execution, such as ending microprograms, loading values into the I register, and in the case of VU0, synchronization with the EE.

Upper Instructions

```

3 3 2 2 2 2 2
1-0-9-8-7-6-5-----0
|I|E|M|D|T|          instr          |
-----

```

Meaning of the various bits:

- I-bit: I register load. Instead of executing a lower instruction, the VU loads the 32-bit value in the lower pipeline into the I register.
- E-bit: Ends microprogram execution. The E-bit has a delay slot, much like branches.
- M-bit: Only applicable for VU0. Ends interlock on a single QMTC2.I/CTC2.I instruction, allowing the EE to continue execution.
- D-bit: Debug break. Halts the VU and sends an interrupt to the EE.
- T-bit: Debug halt. Acts similarly to D-bit.

VU Pipelining

Underneath the surface, the VU is full of quirks. This is partially because its pipeline is exposed to the program.

FMAC Pipeline

```

-----
|M|T|X|Y|Z|S|
 |M|T|X|Y|Z|
  |M|T|X|Y|
   |M|T|X|
-----

```

The FMAC pipeline is split into six stages:

- M: Read instruction from memory
- T: Read VU registers
- X/Y/Z: Execute stages.
- Calculation/flag writeback.

In ideal conditions, one FMAC instruction can be executed every cycle. However, read-after-write hazards - reading a register before its write has completed - induces an FMAC stall for up to three cycles, allowing the previous write to complete.

The FMAC pipeline is used for all instructions except DIV, SQRT, RSQRT, integer calculation, and branching. Integer load/stores also use this pipeline and are subject to hazard checks.

Hazard checks are NOT performed for ACC, VF00, I, Q, P, and R. Hazard checks are performed on each VF field, so writing to VF01y and reading from VF01x does not induce a stall.

Integer Pipeline

```

-----
|M|T|X|Y|Z|S|
 |M|T|X|Y|Z|
  |M|T|X|Y|
   |M|T|X|
-----

```

The integer pipeline is almost the same as the FMAC pipeline, except that all integer instructions have a latency of 1 cycle due to pipeline bypassing.

The other execute and writeback stages are dummy stages to align with the FMAC pipeline.

FDIV Pipeline

```

DIV/SQRT
-----
|M|T|D1|D2|D3|D4|D5|D6|F|
-----

```

```

RSQRT
-----
|M|T|D1|  . . . . .  |D12|F|
-----

```

The FDIV pipeline is used by DIV, SQRT, and RSQRT. It is similar to the FMAC pipeline, though it has more execution stages.

It can execute concurrently with the other pipelines. However, only one FDIV instruction can execute at a time, and trying to execute another FDIV instruction before the first has completed will induce a stall that affects the whole VU. The WAITQ instruction also induces a stall and can be used for synchronization.

Since no hazard checks are performed when reading Q, when Q is read in the middle of an FDIV instruction, the old value of Q is retrieved.

DIV and SQRT have a latency of 7 cycles, and RSQRT has a latency of 13 cycles. The maximum stall time for DIV/SQRT and RSQRT is 6 and 12 cycles, respectively.

EFU Pipeline

```

-----
|M|T|N1|N2|  . . . . .  |Nn|P|
-----

```

The EFU pipeline is used by elementary function instructions exclusive to VU1. It is almost the same as the FDIV pipeline, with the only difference being that hazard checks do not occur for the writeback stage, meaning that a stall lasts for one cycle less than usual. The list of latencies for the EFU instructions is as follows.

EATAN/EATANxy/EATANxz	54
EEXP	44
ELENG	18
ERCPR	12
ERLENG	24
ERSADD	18
ERSQRT	18
ESADD	11
ESIN	29
ESQRT	12
ESUM	12

The WAITP instruction, like WAITQ, induces a stall on the EFU pipeline.

XGKICK Pipeline

XGKICK uses the FMAC pipeline, but the PATH1 transfer it starts occurs concurrently with VU operation. However, if a second XGKICK is executed in the middle of the first, the instruction after the second XGKICK will stall until the PATH1 transfer is complete.

Image Processing Unit (IPU)

The IPU is a hardware-accelerated MPEG1/MPEG2 decoder. It is primarily responsible for video playback, though it can also be used to decompress texture data in some cases.

IPU Reference

[IPU I/O Registers](#)

[IPU Commands](#)

IPU I/O Registers

10002000h IPU_CMD - IPU Send Command

Write

0-27 Option - dependent on command

28-31 Code - the actual command

Read

0-31 Result of FDEC/VDEC command

63 Busy

IPU_CMD is where all commands are sent.

Strange quirk: if data is in the IPU through DMA but no commands have been sent, IPU_CMD will contain the first 32 bits of the bitstream. Certain games, e.g., Theme Park Rollercoaster, rely on this behavior to decode the bitstream before sending the first FDEC command.

10002010h IPU_CTRL - IPU Control/Status

0-3 IFC - Size of data in input FIFO in quadwords

4-7 OFC - Size of data in output FIFO in quadwords

8-13 CBP - Coded block pattern, written to by BDEC/IDEC

14 ECD - Error code detected

15 SCD - Start code detected

16-17 IDP - Intra DC precision

0=8 bits

1=9 bits

2=10 bits

20 AS - Scan pattern for BDEC

0=Zigzag

1=Alternate

21 IVF - Intra VLC format

0=MPEG1-compatible

1=Intra macro block

22 QST - Quantize step for BDEC

0=Linear

1=Nonlinear

23 MP1 - If set, treats bitstream as MPEG1. Otherwise, MPEG2

24-26 Picture type for VDEC

1=I-picture

2=P-picture

3=B-picture

4=D-picture

30 RST - Writing 1 to this resets the whole IPU

31 Busy

When a command is sent, ECD and SCD are cleared to 0. A reset triggers an IPU interrupt if a command is currently executing.

10002020h IPU_BP - Bitstream Position

0-6 BP - Position within the 128-bit quadword being decoded in bits

8-11 IFC - Size of data in input FIFO in quadwords

16-17 FP - Size of data in internal buffer in quadwords

This register is used to determine the amount of unprocessed DMA data currently in the IPU. Sony's FMV library uses this to know what to set MADR to when IPU processing is halted and resumed.

10002030h IPU_BP - Bitstream Position

0-31 Next 32 bits in the bitstream

63 Busy/not enough data

The "busy" bit is set when less than 32 bits are in the FIFO. FMV libraries rely on this undocumented behavior.

IPU Commands

00h BCLR - Clear Input FIFO

0-7 BP - Bitstream start

BCLR clears all data in the input FIFO. The bitstream pointer is set to BP.

01h IDEC - Slice Decode

- 0-5 FB - Bitstream skip
- 16-20 QSC - Quantizer step
- 24 DTD - When set, IDEC decodes DT
- 25 SGN - When set, output RGB is decremented by 128 for each channel. Underflow wraps around
- 26 DTE - Dither enable. Only applicable for RGB16
- 27 OFM - Output format
 - 0=RGB32
 - 1=RGB16

02h BDEC - Macroblock Decode

- 0-5 FB - Bitstream skip
- 16-20 QSC - Quantizer step
- 25 DT - Frame type
 - 0=Frame
 - 1=Field
- 26 DCR - When set, DC prediction value is reset
- 27 MBI - Intra bit
 - 0=Non-intra macroblock
 - 1=Intra macroblock

03h VDEC - VLC Decode

- 0-5 FB - Bitstream skip
- 26-27 TBL - VLC table to read from
 - 0=Macroblock Increment (MBI)
 - 1=Macroblock Type
 - 2=Motion Code
 - 3=DMVector

04h FDEC - Fixed-length Decode

- 0-5 FB - Bitstream skip

05h SETIQ - Set Quantization Table

- 0-5 FB - Bitstream skip
- 27 IQM - Matrix type
 - 0=Intra matrix
 - 1=Non-intra matrix

06h SETIQ - Set Quantization Table

- No option bits

07h CSC - Color Space Conversion

- 0-10 MBC - Macroblocks to decode
- 26 DTE - Dither enable. Only applicable for RGB16
- 27 OFM - Output format
 - 0=RGB32
 - 1=RGB16

08h PACK - 32-bit -> 4-bit/16-bit Color Conversion

- 0-10 MBC - Macroblocks to convert
- 26 DTE - Dither enable
- 27 OFM - Output format
 - 0=INDX4
 - 1=RGB16

09h SETTH - Set Alpha Thresholds

- 0-8 TH0 - Transparent alpha threshold
- 16-24 TH1 - Translucent alpha threshold

EE Interrupt Controller (INTC)

The EE has two separate interrupt signals: INT0 (raised by INTC) and INT1 (raised by DMAC).

1000F000h INTC_STAT - Interrupt status register (R=Status, W=Acknowledge)

1000F010h INTC_MASK - Interrupt mask register (R/W)

Status: Read INTC_STAT (1=IRQ raised)

Acknowledge: Write INTC_STAT (0=No effect 1=Clear bit)

Mask: Write INTC_MASK (0=No effect, 1=Reverse)

- 0 IRQ0 GS interrupt
- 1 IRQ1 SBUS
- 2 IRQ2 VBLANK start
- 3 IRQ3 VBLANK end
- 4 IRQ4 VIF0
- 5 IRQ5 VIF1
- 6 IRQ6 VU0
- 7 IRQ7 VU1
- 8 IRQ8 IPU
- 9 IRQ9 Timer 0
- 10 IRQ10 Timer 1
- 11 IRQ11 Timer 2
- 12 IRQ12 Timer 3
- 13 IRQ13 SFIFO
- 14 IRQ14 VU0 Watchdog

When (INTC_STAT & INTC_MASK), INT0 is asserted on COP0.Cause:8. When COP0.Status:8 is true, an interrupt occurs, and the EE jumps to 80000200h.

IOP Hardware and Peripherals

[CDVD Drive](#)

[Sound Processing Unit \(SPU2\)](#)

[PS2 Serial Port \(SIO2\)](#)

[IOP Interrupts](#)
[IOP DMA](#)
[IOP Timers](#)
[IOP Console](#)

CDVD Drive

Using the CDVD drive, the PS2 has the ability to read CDROMs, single-layer DVDs, and dual-layer DVDs. It also sports backwards compatibility with the PSX's CDROM drive in PSX mode.

CDVD commands are either asynchronous (N commands) or synchronous (S commands). Seeks and reads fall into the former category, and miscellaneous commands, such as RTC access, fall into the latter.

CDVD Reference

[CDVD I/O Ports](#)
[CDVD N Commands](#)
[CDVD Reads and Seeks](#)
[CDVD S Commands](#)

CDVD I/O Ports

1F402004h Current N command (R/W)

Write to this register to send an N command. For a list of N commands, see

[CDVD N Commands](#)

1F402005h N command status (R)

- 0 Error (1=error occurred)
- 1 Unknown/unused
- 2 DEV9 device connected (1=HDD/network adapter connected)
- 3 Unknown/unused
- 4 Test mode
- 5 Power off ready
- 6 Drive status (1=ready)
- 7 Busy executing NCMD

CDVDMAN reads bits 6 and 7 to know when the device is ready to receive a command - bit 6 must be on and bit 7 must be off.

1F402005h N command param (W)

Send parameters for an N command here. This must be done BEFORE the N command has been sent via 1F402004h.

1F402006h CDVD error (R)

Any non-zero value indicates an error? Unknown what kind of errors are possible and what their values are.

1F402007h BREAK

Writing any value to this register sends a BREAK command to the CDVD drive, stopping execution of the current N command.

1F402008h CDVD I_STAT (R=Status, W=Acknowledge)

Status = Read I_STAT (1=Reason for IRQ)

Acknowledge = Write I_STAT (1=Clear bit)

- 0 Data ready?
- 1 (N?) Command complete
- 2 Power off pressed
- 3 Disk ejected
- 4 BS_Power DET?
- 5-7 Unused

When a CDVD IRQ is raised on I_STAT, this register shows the reason for the interrupt. Bit 0 seems to be raised when a read command completes, but I'm not certain about this...

Unknown if bit 1 only applies to N commands, although this appears to be the case.

1F40200Ah CDVD drive status (R)

- 0 Tray status (1=open)
- 1 Spindle spinning (1=spinning)
- 2 Read status (1=reading data sectors)
- 3 Paused
- 4 Seek status (1=seeking)
- 5 Error (1=error occurred)
- 6-7 Unknown

1F40200Bh Sticky drive status (R)

Exact same as 0xA, except the bits are "sticky" - the bits here are ORed with all previous drive states.

CDVDMAN compares 0xA and 0xB to know if the tray status has changed. SCMD 0x5 sets this register to reg 0xA's value.

1F40200Fh CDVD disk type (R)

- 00h No disc
- 01h Detecting
- 02h Detecting CD
- 03h Detecting DVD
- 04h Detecting dual-layer DVD
- 05h Unknown
- 10h PSX CD
- 11h PSX CDDA
- 12h PS2 CD
- 13h PS2 CDDA
- 14h PS2 DVD
- FDh CDDA (Music)
- FEh DVDV (Movie disc)
- FFh Illegal

1F402016h Current S command (R/W)

Write to this register to send an S command. For a list of S commands, see

[CDVD S Commands](#)

1F402017h S command status (R)

```

0-5   Unknown
6     Result data available (0=available, 1=no data)
7     Busy

```

1F402017h S command params (W)

Parameters must be sent BEFORE the S command is sent.

1F402018h S command result (R)

When an S command has finished executing, read the result here. Some S commands may require multiple reads.

CDVD N Commands

Parameters are in units of bytes and are little-endian. All N commands raise IRQ2 (bit 1 of CDVD I_STAT).

00h NOP**01h NOPsync**

Params: None.

These commands do nothing? They do raise an IRQ upon their "completion".

02h Standby

Params: None.

Returns the read position to sector 0 and sets the drive status to PAUSED.

Possibly also spins the drive if it's not spinning already?

03h Stop

Params: None.

Returns the read position to sector 0 and stops the drive.

Seems to have a 166 ms delay?

04h Pause

Params: None.

Unknown what effect this has, from the perspective of emulation.

05h Seek

Params:

```

0-3   Sector position

```

Moves the read position to the indicated parameter.

06h ReadCd

Params:

```

0-3   Sector position
4-7   Sectors to read
10    Block size (1=2328 bytes, 2=2340 bytes, all others=2048 bytes)

```

Performs a CD-style read. Seems to raise bit 0 of CDVD I_STAT upon completion?

08h ReadDvd

Params:

```

0-3   Sector position
4-7   Sectors to read

```

Performs a DVD-style read, with a block size of 2064 bytes. The format of the data is as follows:

```

0     1   Volume number + 0x20
1     3   Sector number - volume start + 0x30000, in big-endian.
4     8   ? (all zeroes)
12    2048 Raw sector data
2060  4   ? (all zeroes)

```

09h GetToc

Params: None? Fetches the ToC from the disk, with a block size of 2064 bytes.

CDVD Reads and Seeks

Seeking

If a read command is called, a seek must be performed. An IRQ is NOT raised when the seek finishes during a read command.

When a seek begins, if the drive is currently not spinning, it takes 333 ms for the drive to spin and finish the seek.

Otherwise, one of three seek modes is possible:

- Contiguous read: When the seek delta is very small or zero, seek time = block_timing * delta
- Fast seek: When seek delta < 14764 for DVD reads and < 4371 for CD reads, seek time = ~30 ms
- Full seek: Seek time = ~100 ms

block_timing (in IOP cycles) can be found by the following formula:

```

block_timing = (IOP_CLOCK * block_size) / read_speed

```

Where IOP_CLOCK is ~36,864,000 Hz. read_speed for CD reads is 24 * 153600. For DVD reads read_speed = 4 * 1382400.

Unknown what delta is needed for a contiguous read (8/16 is used for CD/DVD respectively in PCSX2).

Reads

The time needed to read a single sector is the block_timing formula above. Once one sector has been read, the CDVD DMA channel can store the data in memory and allow the CDVD drive to continue.

When all sectors have been read, a CDVD IRQ is raised. Successful reads seem to raise both bits 1 AND 0 of CDVD I_STAT?

CDVD S Commands

The results of an S command can be read one at a time from 1F402017h. All units are in bytes.

03h Subcommand

Params:
 0 Subcommand number

This is a prefix byte. Subcommands are listed below in the format of 03h:XXh.

03h:00h MechaconVersion

Params: None.

Result:
 0-3 MECHACON version

Unknown what the bytes mean. Newer versions of CDVDMAN do checks on them of some sort.

PCSX2 uses 00020603h as the version.

05h UpdateStickyFlags

Params: None.

Result:
 0 Update status (0=successful, 1=busy)

Sets 1F40200Bh to the value of 1F40200Ah. Used internally by CDVDMAN for detecting if a disk has been inserted.

08h ReadRTC

Params: None.

Result:
 0 Zero
 1 Second
 2 Minute
 3 Hour
 4 Zero
 5 Day
 6 Month
 7 Year

Returns the current time stored on the RTC, in BCD format.

The RTC is mostly just used by the BIOS. Notably, Metal Gear Solid 3 requires the RTC to boot. This is likely an anti-piracy feature.

09h WriteRTC

Params:
 0 Ignored
 1 Second
 2 Minute
 3 Hour
 4 Ignored
 5 Day
 6 Month
 7 Year

Overwrites the RTC's time in BCD format.

05h UpdateStickyFlags

Params: None.

Result:
 0 Update status (0=successful, 1=busy)

Sets 1F40200Bh to the value of 1F40200Ah. Used internally by CDVDMAN for detecting if a disk has been inserted.

15h ForbidDVD

Params: None.

Blocks access to the DVD drive. PCSX2 writes 0x5 to result

40h OpenConfig

Params: None.

Opens the DVD drive configuration. Dobiestation returns zero

41h ReadConfig

Params: None.

Reads the previously opened configuration file. Output is four 32bit words

43h CloseConfig

Params: None.

Closes the opened config file

Sound Processing Unit (SPU2)

[SPU2 AutoDMA](#)

SPU2 AutoDMA

1F9001B0h+(core*400h) SPU_ADMA_CTRL

0 On core0, 1=start ADMA write/ADMA busy. No effect on core1
 1 Same as above, but only works on core1
 2 Start ADMA read on this core (1=start ADMA read). Unknown how ADMA reads work

ADMA streams signed stereo 16-bit PCM audio to the MEMIN region, bypassing almost all SPU2 processing. In other words, ADMA allows games to stream raw audio without compression or special effects - this feature is usually used for music.

ADMA uses the normal DMA interface, and it is not possible to have both ADMA and a normal SPU DMA transfer active at the same time.

MEMIN and Data Format

2000h-20FFh Core0 left buffer 0
 2100h-21FFh Core0 left buffer 1
 2200h-22FFh Core0 right buffer 0

2300h-23FFh Core0 right buffer 1

2400h-24FFh Core1 left buffer 0

2500h-25FFh Core1 left buffer 1

2600h-26FFh Core1 right buffer 0

2700h-27FFh Core1 right buffer 1

A full ADMA block is 1024 bytes, where the first 512 bytes are left channel samples, and the other half is the right channel. MEMIN is double-buffered, allowing the mixer hardware to read from one buffer while ADMA writes to another free buffer. Some games do upload blocks smaller than 1024 bytes, which partially fills the free buffer.

PS2 Serial Port (SIO2)

SIO2 provides access to controllers and memory cards in PS2 mode, as well as other peripherals. While following a similar principle as the PSX's serial port (called SIO0 here), SIO2 is completely different in design.

SIO2 Reference

[SIO2 Registers](#)

[SIO2 PS2 Memcards](#)

[SIO2 PS2 Memcard Filesystem](#)

[SIO2 Controller Commands](#)

[SIO2 Standard Controllers](#)

SIO2 Registers

A good portion of SIO2 is a mystery. Having no documentation whatsoever, its behavior has to be inferred from how programs use it.

1F808200h-1F80823Fh SIO2_SEND3 - Command Parameters

0-1 Port

8-16 Fake command length, not used by SIO2

18-24 Real command length (counting peripheral byte)

Other Unknown

SEND3 is an array of up to 16 different SIO2 commands. A command consists of a peripheral byte that selects to what the command is being sent, a command sent to the peripheral, and an arbitrary amount of data.

Known peripheral bytes:

- 01h - Controller
- 21h - Multitap
- 61h - Infrared
- 81h - Memory card

1F808240h-1F80825Fh SIO2_SEND1/SEND2 - Port1/2 Control?

Unknown purpose.

When bit 2 of the address is set, SEND2 is accessed. Otherwise, SEND1 is accessed.

1F808260h SIO2_FIFOIN - Data Write

A one-byte register used to upload commands to SIO2. The SIO2in DMA channel also writes to this register.

1F808264h SIO2_FIFOOUT - Data Read

Used to read replies and data from SIO2 peripherals after a command is sent. SIO2out reads from this register.

1F808268h SIO2_CTRL - Control Register

Bit 0 seems to start the command transfer. An SIO2 interrupt is raised after touching this bit, presumably when the command has completed.

Bits 2 and 3 reset SIO2, preparing it for another transfer.

Rest of the bits are unknown. They probably control bandwidths and interrupt masking, at the very least.

The SIO2MAN module in the BIOS sets this register to 3BCh on a reset.

1F80826Ch SIO2_RECV1 - Response Status 1 (R)

Set after a transfer, indicating if the peripheral is connected.

If (RECV1 & F000h) == 1000h after a memory card command, the memory card is connected. Else, disconnected.

If (RECV1 & 2000h) == 0 after a pad command, the pad is connected.

Known value for a disconnected peripheral is 1D100h. Known value for a connected peripheral is 1100h.

1F808270h SIO2_RECV2 - Response Status 2 (R)

Read by PADMAN. Always equal to 0xF?

1F808274h SIO2_RECV3 - Response Status 3 (R)

Unknown.

1F808280h SIO2_ISTAT? - Interrupt Flags?

SIO2MAN's interrupt handler reads from this and writes to it the value read. Not known what any of the bits represent.

SIO2MAN Program Flow For Transfer

Write CTRL | 0Ch to CTRL.

Write data to SEND1 and SEND2, then write data to SEND3.

Write data to DATAIN if applicable, then start SIO2in and SIO2out DMA transfers if applicable.

Write CTRL | 01h to CTRL, then wait for an SIO2 interrupt.

After interrupt, read RECV1, RECV2, and RECV3, then read DATAOUT if applicable.

SIO2 PS2 Memcards

Basic Info

A PS2 memory card uses flash memory. The main advantages are that it is cheap and does not require any power to retain information. The downsides are that

accessing flash is much slower than RAM, and flash memory will go bad when written to too many times.

Flash also has some restrictions on how it can be written to. Depending on the card, individual bits can be changed from 0 to 1 or 1 to 0, but not both. In order to flip a bit in the opposite direction, the entire block must be erased.

Official memory cards are 8 MB, but third-party cards can be larger than this. Certain games do not work if the memory card is larger than expected, for unknown reasons.

Commands

11h - Probe

Command: 11 + 1 XX byte
Reply: 2B + Terminator
Length: 2 bytes

First command sent when trying to detect a card.

The terminator defaults to 55h on reset. Newer versions of MCMAN check for this.

12h - Unk12

Command: 12 + 1 XX byte
Reply: 2B + Terminator
Length: 2 bytes

Unknown purpose. Sent on write/erase commands. Maybe some sort of flush?

21h - Start Erase

22h - Start Write

23h - Start Read

Command: 21 + 4-byte sector address + 2 XX bytes
Reply: 5 XX bytes + 2Bh + Terminator
Length: 7 bytes

These commands prepare read/write/erase sector operations by providing a starting address in sectors.

26h - Get Specs

Command: 26 + 10 XX bytes
Reply: 2B + 2-byte sector size in bytes + 2-byte erase block size + 4-byte sector count + 1-byte checksum + Terminator
Length: 11 bytes

Retrieves the card's capabilities. The total size of the memory card is (sector size + 16) * sectors (the 16 is space for error correction).

The checksum is an XOR between the individual bytes of sector size, erase blocks, and sectors.

A standard Sony 8 MB memory card will report the following.

2B 00 02 10 00 00 40 00 00 52

That is to say, a sector size of 512 bytes, an erase block page count of 16 sectors, and a sector count of 16,384.

27h - Set Terminator

Command: 27 + 1-byte new terminator + 1 XX byte
Reply: XX + 2B + 0ld terminator
Length: 3 bytes

28h - Get Terminator

Command: 28 + 2 XX bytes
Reply: 2B + Terminator + 55
Length: 3 bytes

42h - Write Data

Command: 42 + 1-byte write size + 128 data bytes + 2 XX bytes
Reply: 2B + Terminator + 129 XX bytes + Terminator
Length: 132 bytes

Writes up to 128 bytes at the memory card address given by command 22h. 128 data bytes must be sent, though anything more than the given size is ignored.

43h - Read Data

Command: 42 + 1-byte read size + 130 XX bytes
Reply: 2B + Terminator + 128 data bytes + 1-byte XOR checksum + Terminator
Length: 132 bytes

Reads up to 128 bytes at the address given by command 23h. The checksum is applied to the amount of bytes read.

81h - Read/Write End

Command: 81 + 1 XX byte
Reply: 2B + Terminator
Length: 2 bytes

Sent at the end of a read or write. Unknown purpose.

82h - Erase Block

Command: 82 + 1 XX byte
Reply: 2B + Terminator
Length: 2 bytes

Erases data at the address given by command 21h by setting it all to FFh. On a standard 8 MB card, this would be (512 + 16) * 16 bytes of data.

BFh - UnkBF

Command: 82 + 2 XX bytes
Reply: 1 XX byte + 2B + Terminator
Length: 3 bytes

Used during card detection.

F0h - AuthXorF0

Command: F0 + 1-byte param + variable
Reply: 1 XX byte + 2B + Variable + Terminator
Length: 12 bytes

This strange command is used by SECRMAN while detecting the card and does some XOR checksumming.

(todo: better explanation)

F3h - AuthF3

Command: F3 + 2 XX bytes
Reply: 1 XX byte + 2B + Terminator
Length: 3 bytes

Sent by SECRMAN. Unknown purpose.

F7h - AuthF7

Command: F7 + 2 XX bytes
Reply: 1 XX byte + 2B + Terminator
Length: 3 bytes

Sent by SECRMAN. Unknown purpose.

SIO2 PS2 Memcard Filesystem

The PS2 uses a double-indirect indexed FAT filesystem to keep track of clusters. It also has support for ECC and bad block detection, depending on MCMAN version.

Superblock Format

Byte	Name
0-27	Magic string. Should contain "Sony PS2 Memory Card Format "
28-39	Version string. Has the format "1.x.0.0", where x is the minor version
40-41	Size in bytes of a page. Default 512
42-43	Pages per cluster. Default 2
44-45	Pages per erase block. Default 16
46-47	Set to -256. Doesn't seem to be used
48-51	Total clusters in the card. Default 8192
52-55	Offset of the first allocatable cluster, in cluster units. Immediately after the FAT
56-59	Offset of the cluster after the last allocatable cluster, relative to the first
60-63	Offset of the root directory cluster, relative to the first. Should be 0
64-67	Backup erase block. Should be the last block in the card, default 1023
68-71	Second backup block, should be the second-last block in the card
80-207	Array of 32 indirect FAT cluster indices. On a standard card, only one indirect cluster is used
208-335	Array of 32 bad blocks, which cannot be used. -1 indicates no entry
384	Memory card type. Should be 2, indicating a PS2 memory card
385	Card flags. Default 52h
	Bit 0=ECC support if set
	Bit 3=Card has bad blocks
	Bit 4=Erased blocks have bits set to 0 if set

The superblock is located in the first page in the memory card.

SIO2 Controller Commands

SIO2 communicates with the controller by sending sequences of bytes and receiving replies from the controller.

Note that the reply comes immediately after the appropriate byte has been sent, more specifically the reply bytes are not queued.

Packets have a three byte header followed by an additional 2, 6 or 18 bytes of additional command and controller data (like button states, vibration motor commands, button pressures, etc.).

Packet header

Command: 01 + command byte + 00
 Reply: FF + depends on command + 5A
 Length: 3

The first byte is the peripheral byte sent by SIO2 and is always 01 for controllers. The reply is always FF. The command byte tells the peripheral which command to execute.

All commands reply with F3 except from commands 42 and 43 which reply with the mode byte (more details below). The third byte is always 00 and its reply is always 5A

Below is a list of known controller commands.

41h - Query Masked Mode

Command: header + 5A + 5A + 5A + 5A + 5A + 5A

If in digital mode or a nonstandard controller:
 Reply: header reply + zero bytes

If in analog or DS2 mode:
 Reply: header reply + 3 mask bytes + 00 + 00 + 5A
 Length: 8 +---- (last byte is always 0x5A)

This command is used to find out what buttons are included in poll responses. The controller can be configured (through command 0x4F) to respond with more or less information about each button with each poll. If the controller is in digital mode (0x41) this commands returns zeroes.

Only works when the controller is already in configuration mode.

42h - Read Data

Command: header + variable zero bytes

If in digital mode or a nonstandard controller:
 Reply: header reply (returns mode byte) + 2 button state bytes
 Length: 4

If in analog or DS2 mode:
 Reply: header reply (returns mode byte) + 2 button state bytes + 4 analog button state bytes + (optional) 12 button pressure bytes
 Length: 8 or 20 depending if analog button pressures are requested.

The format of controller inputs varies based not only on what is connected, but also what its current mode is. See the respective controller pages for their input formats.

The high nibble of the mode byte indicates the mode (0x4 is digital, 0x7 is analog, 0xF config / escape?), while the lower nibble is how many 16 bit words follow the header.

Known mode bytes returned by header are presented below:

12h Mouse
 23h NeGcon
 41h DualShock Digital
 73h DualShock Analog
 79h DualShock 2 (analog+pressure)
 F3h In config mode

43h - Enter/Exit Config Mode

Outside config mode:
 Command: header + 1 config byte (1=enter config) + zero bytes
 Reply: header reply (with mode byte) + same as Read Data (but without pressure values in DS2 mode)
 Length: Same as Read Data

Inside config mode:
 Command: header + 1 config byte (0=exit config) + zero bytes

Reply: header reply (with mode byte) + 6 zero bytes
Length: 8

Config mode allows the controller state to be changed. When the controller is not in config mode, only commands 42h and 43h work.

When the controller is outside of config mode this command behaves the same as 42. If the command is in config mode is command can only be used to exit config mode.

45h - Query Model and Mode

Command: header + 5A + 5A + 5A + 5A + 5A + 5A
Reply: header reply + model + 02 + analog + 02 + 01 + 00
Length: 8

model=01h for Dualshock and 03 for Dualshock 2

analog=01h if the controller is in analog mode and 00h if in digital mode.

46h - Query Act

Command: header + index + 5A + 5A + 5A + 5A + 5A

If index=0

Reply: header reply + 00 + 00 + 01 + 02 + 00 + 0A

If index=1

Reply: header reply + 00 + 00 + 01 + 01 + 01 + 14

Length: 8

This command is always issued twice in a row, and appears to be retrieving a 10 byte constant of over those two calls.

It is always called in a sequence of 46h 46h 47h 4Ch 4Ch. Only works after the controller is in config mode.

47h - Query Comb

Command: header + 00 + 5A + 5A + 5A + 5A + 5A
Reply: header reply + 00 + 00 + 02 + 00 + 01 + 00
Length: 8

Unknown purpose.

4Ch - Query Mode

Command: header + index + 5A + 5A + 5A + 5A + 5A
Reply: header reply + 00 + 00 + 00 + val + 00 + 00
Length: 8

if index=0, val=4

if index=1, val=7

Unknown purpose.

4Dh - Vibration Toggle

Command: header + aa + bb + cc + dd + ee + ff
Reply: header reply + .. old rumble values ..
Length: 8

This command configures rumble, but it is unknown exactly how. Bytes aa-ff are used for this.

On reset, rumble values are all FFh.

SIO2 Command Sequence

The PS2 supports both DualShock and DualShock 2 controllers natively. However, certain games require DS2.

Input Format

```
Cmd      Reply
42h      ID_lo
??h      ID_hi
??h      Buttons_lo
??h      Buttons_hi
---- transfer stops here if digital ----
??h      Analog right joystick, y-axis (00h = upmost, 80h = centered, FFh = downmost)
??h      Analog right joystick, x-axis (00h = leftmost, 80h = centered, FFh = rightmost)
??h      Analog left joystick, y-axis (see above)
??h      Analog right joystick, x-axis (see above)
---- transfer stops here if not DS2 ----
??h      D-pad right pressure (00h = no pressure, FFh = max pressure)
??h      D-pad left pressure
??h      D-pad up pressure
??h      D-pad down pressure
??h      Triangle pressure
??h      Circle pressure
??h      Cross pressure
??h      Square pressure
??h      L1 pressure
??h      R1 pressure
??h      L2 pressure
??h      R2 pressure
```

Button bits (0=pressed, 1=released):

```
0      Select
1      L3
2      R3
3      Start
4      D-pad Up
5      D-pad Right
6      D-pad Down
7      D-pad Left
8      L2
9      R2
10     L1
11     R1
12     Triangle
13     Circle
14     Cross
15     Square
```

IOP Interrupts

Interrupt handling for the IOP is similar to its PSX counterpart. The main differences are an additional register (I_CTRL) and more interrupt lines.

1F801070h I_STAT - Interrupt status register (R=Status, W=Acknowledge)

1F801074h I_MASK - Interrupt mask register (R/W)

Status: Read I_STAT (1=IRQ raised)

Acknowledge: Write I_STAT (0=Clear bit 1=No effect)

Mask: Read/Write I_MASK (0=Disabled 1=Enabled)

0	IRQ0	VBLANK start
1	IRQ1	GPU (used in PSX mode)
2	IRQ2	CDVD Drive
3	IRQ3	DMA
4	IRQ4	Timer 0
5	IRQ5	Timer 1
6	IRQ6	Timer 2
7	IRQ7	SI00
8	IRQ8	SI01
9	IRQ9	SPU2
10	IRQ10	PIO
11	IRQ11	VBLANK end
12	IRQ12	DVD? (unknown purpose)
13	IRQ13	PCMCIA (related to DEV9 expansion slot)
14	IRQ14	Timer 3
15	IRQ15	Timer 4
16	IRQ16	Timer 5
17	IRQ17	SI02
18	IRQ18	HTR0? (unknown purpose)
19	IRQ19	HTR1?
20	IRQ20	HTR2?
21	IRQ21	HTR3?
22	IRQ22	USB
23	IRQ23	EXTR? (unknown purpose)
24	IRQ24	FWRE (related to FireWire)
25	IRQ25	FDMA? (FireWire DMA?)
26-31	Unused/garbage	

1F801078h I_CTRL - Global interrupt control (R=Status and Disable, W)

0	Enable all interrupts
1-31	Unused/garbage

When bit 0=1, all IOP interrupts are enabled. Reading this register returns bit 0 AND clears it, disabling interrupts. Writing can set or reset bit 0.

Raising Interrupts

If I_CTRL && (I_STAT & I_MASK), then COP0.Cause:8 is set. When COP0.Status:8 is also set when this occurs, COP0.Cause.Excode is set to 00h and the IOP jumps to 80000080h, where the interrupt will be processed.

IOP DMA

The IOP re-uses the same DMA channels found in the PSX, and it contains additional channels for new peripherals.

Channels

Old channels		
1F80108xh	0	MDECin
1F80109xh	1	MDECout
1F8010Axh	2	SIF2 (EE->IOP, GPU in PSX mode)
1F8010Bxh	3	CDVD (CDROM in PSX mode)
1F8010Cxh	4	SPU1
1F8010Dxh	5	PIO
1F8010Exh	6	OTC

New channels		
1F80150xh	7	SPU2
1F80151xh	8	DEV9 (expansion port)
1F80152xh	9	SIF0 (IOP->EE, uses TADR)
1F80153xh	10	SIF1 (EE->IOP)
1F80154xh	11	SI02in
1F80155xh	12	SI02out

1F801xx0h+N*10h Dn_MADR

0-23	Start address of transfer in RAM
24-31	Unused, zero

1F801xx4h+N*10h Dn_BCR

0-15	Block size in words (0=0x10000)
16-31	Block count

1F801xx8h+N*10h Dn_CHCR

0	Transfer direction (0=to RAM, 1=from RAM)
1	MADR increment per step (0=+4, 1=-4)
2-7	Unused
8	When 1: -Burst mode: enable "chopping" (cycle stealing by CPU) -Slice mode: Causes DMA to hang -Linked-list/Chain mode: Transfer header/tag before data
9-10	Transfer mode 0=Burst (transfer data all at once, only low 16 bits of BCR are valid) 1=Slice (transfer in units of BCR blocks and arbitrate) 2=Linked-list mode 3=Chain mode (uses TADR)
11	Unknown, used by iLink DMA
12-15	Unused
16-18	Chopping DMA window size (1 << N words)
19	Unused
20-22	Chopping CPU window size (1 << N cycles)
23	Unused
24	Start transfer (0=stopped/completed, 1=start/busy)

- 28 Force transfer start without waiting for DREQ
- 29 In forced-burst mode, pauses transfer while set.
In other modes, stops bit 28 from being cleared after a slice is transferred.
No effect when transfer was caused by a DREQ.
- 30 Perform bus snooping (allows DMA to read from IOP cache?)
- 31 iLink automatic response - unknown purpose

1F801xxCh+N*10h Dn_TADR

- 0-23 Start address of list of tags
- 24-31 Unused, zero

TADR is used in chain mode. At the start of a chain transfer, a tag will be read from TADR, and TADR will be incremented by the size of the tag, which is either 2 or 4 words.

Chain Mode Tag Format

- 0-23 Start address of transfer in RAM
- 30 IRQ - raises IQE interrupt in DICR2 when all words in this tag are transferred
- 31 End of transfer, raises transfer completion interrupt
- 32-55 Size of transfer in words

... Optionally 2 words to transfer after tag ...

If Dn_CHCR.8 is set, 2 words after the tag in memory (TADR + 8) will be transferred first. This is used by SIF0 to transfer a DMA tag to the EE's DMAC. Only the SIF0, SIF1, and SPU1 channels can use tag mode. SIF0 and SIF1 use it exclusively, but it is unknown whether anything uses chain mode on SPU1.

1F8010F0h DPCR - DMA Priority/Enable

- Each 4 bits correspond to a channel.
- Bits 0-2 are priority, where 0=highest and 7=lowest. If two channels have the same priority, priority is determined by channel number, priority.
- Bit 3 is a channel enable, 0=disable, 1=enable.
- 0-3 Channel 0 (MDECin)
- 4-7 Channel 1 (MDECout)
- 8-11 Channel 2 (SIF2)
- 12-15 Channel 3 (CDVD)
- 16-19 Channel 4 (SPU1)
- 20-23 Channel 5 (PIO)
- 24-27 Channel 6 (OTC)
- 28-31 Channel 'C' (CPU priority. Bit 31 has no effect)

Initial value on PS2 reset is 0x07777777. Initial value on PSX reset is 0x07654321.

1F801570h DPCR2 - DMA Priority/Enable 2

Works similarly to DPCR.

- 0-3 Channel 7 (SPU2)
- 4-7 Channel 8 (DEV9)
- 8-11 Channel 9 (SIF0)
- 12-15 Channel 10 (SIF1)
- 16-19 Channel 11 (SI02in)
- 20-23 Channel 12 (SI02out)
- 24-27 Channel 'U' (USB DMA, controlled through USB registers)
- 28-31 Unknown

1F8010F4h DICR - DMA Interrupt Register

- 0-6 Controls channel 0-6 completion interrupts in bits 24-30.
When 0, an interrupt only occurs when the entire transfer completes.
When 1, interrupts can occur for every slice and linked-list transfer.
No effect if the interrupt is masked by bits 16-22.
- 7-14 Unused
- 15 Bus error flag. Raised when transferring to/from an address outside of RAM. Forces bit 31 to be set.
- 16-22 Channel 0-6 interrupt mask. If enabled, channels cause interrupts as per bits 0-6.
- 23 Master channel interrupt enable.
- 24-30 Channel 0-6 interrupt flags. Writing 1 clears a flag.
IMPORTANT: The flag only gets raised if the interrupt is enabled! INTRMAN relies on this behavior.
- 31 Master interrupt flag. When set, IRQ 3 is sent to the IOP's INTC.
DICR.31 = DICR.15 | (DICR.23 && (DICR.24-30 || DICR2.24-29))

1F801574h DICR2 - DMA Interrupt Register 2

- 0-12 Interrupt on tag bit. Corresponds to channels 0-12, but only bits 4, 9, and 10 can be set - this corresponds to SPU1, SIF0, and S
- When set, an interrupt is raised on an IRQ tag when the tag transfer completes.
NOTE: This works independently of the channel's interrupt mask, so a tag interrupt can still happen even when the transfer interr
- 13-15 Unused
- 16-21 Channel 7-12 interrupt mask. If enabled, channels cause interrupts upon transfer completion.
- 22-23 Unused
- 24-29 Channel 7-12 interrupt flags. Works same as DICR interrupt flags, including being masked by DICR.23
- 30-31 Unused

1F801578h DMACEN - Global DMA Enable

- 0 0=All transfers disabled
- 1 1=All transfers enabled

This register can be safely accessed while DMA is ongoing.

1F80157Ch DMACINTEN - Global DMA Interrupt Control

- 0 When 0, all channel interrupts disabled. Master interrupt flag is 0 in all cases but bus error interrupts.
- 1 When 1, DMA interrupts disabled - IRQ 3 is never sent to INTC. Does not affect master interrupt flag.

IOP Timers

1F801100h+N*10h Timer 0..2 count (R/W)

- 0-15 Current value
- 16-31 Unused/garbage

1F801480h+(N-3)*10h Timer 3..5 count (R/W)

- 0-31 Current value

Timers 0..5 increment automatically. 0..2 are 16-bit, and 3..5 are 32-bit. Writes set the counter to the value written.

1F801104h+N*10h Timer 0..2 mode (R/W)**1F801484h+(N-3)*10h Timer 3..5 mode (R/W)**

```

0    Gate enable
1-2  Gate mode
3    Zero return - reset counter on interrupt
4    Compare interrupt enabled
5    Overflow interrupt enabled
6    Repeat interrupt - if unset, bit 10 is set to 0 after interrupt occurs.
7    LEVL - toggle bit 10 on IRQs if bit 6 is set.
8    Use external signal
    If set:
        Timer 0: pixel clock (13.5 MHz regardless of screen mode)
        Timer 1/3: HBLANK
        Others: sysclock (no effect)
9    Timer 2 prescaler
10   Interrupts enabled (R)
11   Compare interrupt raised (R)
12   Overflow interrupt raised (R)
13-14 Timer 4/5 prescaler
15-31 Unused/garbage

```

Writes to mode reset the count to zero and set bit 10 to 1. Reads from mode clear the two raised interrupt flags.

Prescalers adjust clockrate as follows:

```

0    normal
1    1/8 speed
2    1/16 speed
3    1/256 speed

```

1F801108h+N*10h Timer 0..2 target (R/W)

```

0-15  Value
16-31 Unused/garbage

```

1F801488h+(N-3)*10h Timer 3..5 target (R/W)

```

0-31  Value

```

When count == target, a compare interrupt is raised. This raises an IRQ in I_STAT if both mode.4 and mode.10 are enabled.

If mode.7 (LEVL) is not set, writes to target set mode.10 to 1.

IOP Console

The IOP uses multiple addresses when printing output to the console. Registers \$a1 and \$a2 are used to store the string address in memory and its size respectively.

```

if (PC == 0x12C48 || PC == 0x1420C || PC == 0x1430C)
{
    uint32_t pointer = gpr[5];
    uint32_t text_size = gpr[6];
    while (text_size)
    {
        auto c = (char)ram[pointer & 0xFFFFF];
        putc(c);

        pointer++;
        text_size--;
    }
}

```

Subsystem Interface (SIF)

The SIF is how the EE and IOP communicate with each other. SIF has some mailbox hardware registers that the CPUs can use to pass values to each other, which happens during SIF initialization. However, once both sides have booted, they use the SIF0 (IOP->EE) and SIF1 (EE->IOP) DMA channels to communicate.

[SIF Registers](#)

[SIF DMA](#)

[SIF RPC Basics](#)

[SIF RPC Structs and Definitions](#)

[SIF RPC Commands](#)

[SIF RPC System Servers](#)

SIF Registers

Note: EE base is at 1000F200h, IOP base is at 1D000000h.

1000F200h/1D000000h SIF_MSCOM (Only writable by EE)

1000F210h/1D000010h SIF_SCOM (Only writable by IOP)

```

0-31  Value

```

These registers could be used for generic communication, but in the PS2 they are used to tell the other side the address of its DMA receive buffer. For example, the EE places its SIF0 receive address in MSCOM so that the IOP knows where to transfer data to, and the opposite is true for SMCOM.

1000F220h/1D000020h SIF_MSFLG (IOP writes mask)

1000F230h/1D000030h SIF_SMFLG (EE writes mask)

```

0-31  Value

```

These registers are used like semaphores. The following values are used:

```

10000h: SIF DMA/hardware initialized
20000h: SIFCMD initialized
40000h: IOP has finished booting (sent by EESYNC)

```

If the EE sends 20000h to MSFLG, the IOP can read this and clear MSFLG by writing 20000h to it. The opposite is true for SMFLG.

1000F240h/1D000040h SIF_CTRL

```

1    Always 1?
8    Always 1 for EE, 0 for IOP?
28-31 Always 0xF?
Other Unknown

```

Very little is known about this register.

1000F260h/1D000060h SIF_BD6

```

0-31  Unknown

```

Nothing is known about this register, other than that it does get accessed during initialization.

SIF DMA

During SIF initialization and after bootup the EE and IOP can use DMA to send data to each other. This is achieved using a set of fifos (First In First Out) located in the SIF, referred to as SIF0 and SIF1.

Assume that the EE wants to send data to the IOP:

- The EE writes the data it wants to send to RAM, alongside with respective DMA tags
- It calls SifSetDma to initialize a SIF1 transfer to the IOP
- The DMAC starts writing qwords to the SIF1 fifo
- After the data is transferred, the IOP starts a SIF1 transfer and reads the data in the fifo
- Finally an interrupt is asserted on DICR/DICR2 and if that mask is also set, an IOP interrupt is triggered.

Note that if a SIF0/SIF1 transfer is started on either component and the fifo is empty the transfer continues running until the respective fifo starts filling with data. In addition this process relies heavily on interrupts and DICR flag writing to work.

SIF RPC Basics

Broadly, the SIF protocol can be split into three different layers of abstraction, from lowest to highest:

- SIF DMA: Accesses raw hardware registers to transfer data between the EE and IOP.
- SIF CMD: Manages interrupt handling, allowing programs to send commands to the other side.
- SIF RPC: Defines a server/client interface around SIF CMD, allowing programs to call functions on the other side as if they were part of the program. For example, from the game's perspective, sceMcWrite is like any other function, even though internally it has to send a request to the IOP to access the memory card.

Calling RPC functions

Assume that a game on the EE wants to open a file.

- The EE client sends a BIND request to the IOP. To do this, the game must know the ID of the server it wants to bind to. Since it is trying to open a file, it would use 0x80000001, the ID of FILEIO.
- The IOP receives the request and looks up the server struct associated with the ID.
- The IOP finds the server and sends an END packet to the EE. This contains a pointer to the IOP server, along with some other information. After receiving the END reply, the EE client can now call FILEIO functions.
- The EE sends a CALL request to the IOP. The IOP receives this and wakes up the RPC thread associated with the server, passing the function number, data buffer, and size of the data to the RPC thread.
- FILEIO executes the open command and returns a result. The IOP sends an END packet to the EE, and it can optionally directly DMA the result of the RPC command to a different buffer. In this case, it would be a file descriptor ID.

In more succinct terms, the EE first uses SifBindRpc to bind the server to the client, then it uses SifCallRpc to call an IOP function.

Registering an RPC server

Servers can be registered on both the EE and the IOP. In the vast majority of cases however, only IOP modules register servers.

- Create a new thread, which will be used by the server. All actions below are done on the new thread.
- Call SifSetRpcQueue first and then SifRegisterRpc to register the server.
- Finally, call SifRpcLoop. This puts the server thread to sleep until a new SIF RPC request arrives.

The above creates a dedicated server thread, which can only be used for SIF RPC. If non-blocking execution is desired for whatever reason, one can use SifGetNextRequest and SifExecRequest.

SIF RPC Structs and Definitions

Functions

```
typedef void (*SifCmdHandler)(void *data, void *harg);
typedef void* (*SifRpcFunc)(int fno, void *buff, int length);
typedef void (*SifRpcEndFunc)(void *end_param);
```

Packet Headers

```
struct SifCmdHeader
{
    uint psize:8; //Size of the command packet
    uint dsize:24; //Size of the payload, if any
    void *dest; //Destination of the payload, if any
    int cid; //Command ID
    uint opt;
}
struct SifRpcPktHeader
{
    struct SifCmdHeader sifcmd;
    int rec_id;
    void *pkt_addr;
    int rpc_id;
}
```

Client

```
struct SifRpcClientData
{
    struct SifRpcHeader hdr;
    u32 command;
    void *buff, *cbuf;
    SifRpcEndFunc end_function;
    void *end_param;
    struct SifRpcServerData *server;
}
```

Server

```

struct SifRpcServerData
{
    int sid;

    SifRpcFunc func;
    void *buff;
    int size;

    SifRpcFunc cfunc;
    void *cbuff;
    int size2;

    struct SifRpcClientData *client;
    void *pkt_addr;
    int rpc_number;

    void *receive;
    int rsize;
    int rmode;
    int rid;

    struct SifRpcServerData *link;
    struct SifRpcServerData *next;
    struct SifRpcDataQueue *base;
}

```

Server Data Queue

```

struct SifRpcDataQueue
{
    int thread_id, active;
    struct SifRpcServerData *link, *start, *end;
    struct SifRpcDataQueue *next;
}

```

DMA Packet

```

struct SifDmaTransfer
{
    void *src, *dest;
    int size;
    int attr;
}

```

SIF RPC Commands

80000000h Change SADDR

```

struct SifSaddrPkt
{
    struct SifCmdHeader header;
    void* buff;
}

```

This just changes the EE's receive buffer on the IOP side. Used when SifInitRpc is called more than once.

80000001h Set SREG

```

struct SifCmdSRegData
{
    SifCmdHeader header;
    int index;
    uint value;
}

```

Sets a software SIF register - that is, a variable in memory rather than an I/O register.

Used by the IOP during SIF RPC initialization to tell the EE what the IOP has stored for the EE's receive buffer.

80000002h SIFCMD Init

```

struct SifInitPkt
{
    struct SifCmdHeader header; //NOTE: The "opt" field is used by the IOP.
    void* buff;
}

```

This command is sent twice by the EE during SIFRPC initialization, the first with opt=0 and the second with opt=1.

When opt=0, buff is used as the EE's SIF0 receive address, and the IOP sets SMFLG to 20000h.

When opt=1, the IOP finishes SIFRPC initialization.

80000003h Reboot IOP

```

struct SifIopResetPkt
{
    struct SifCmdHeader header;
    int arglen; //Length of the command
    int mode; //Bit 31 enables debug logging, unknown what other values do
    char arg[80]; //The command to be passed to MODLOAD
}

```

See [BIOS IOP REBOOT](#) for details on the reboot procedure.

80000008h Request End

```

struct SifRpcRendPkt
{
    struct SifCmdHeader sifcmd;
    int rec_id;
    void *pkt_addr;
    int rpc_id;
    struct SifRpcClientData *client;
    uint cid; //ID of the command sent by the other side (e.g. the command that triggered a REND)
    struct SifRpcServerData *server;
    void *buff, *cbuff;
}

```

80000009h Bind

```
struct SifRpcBindPkt
{
    struct SifCmdHeader sifcmd;
    int rec_id;
    void *pkt_addr;
    int rpc_id;
    struct SifRpcClientData *client;
    int sid; //ID of the server
}
```

8000000Ah Call

```
struct SifRpcCallPkt
{
    struct SifCmdHeader sifcmd;
    int rec_id;
    void *pkt_addr;
    int rpc_id;
    struct SifRpcClientData *client;
    int rpc_number; //ID of the function to call on the server
    int send_size; //Size of data to send to the server
    void *receive; //Buffer to hold reply data from the server
    int rcv_size; //Size of reply buffer
    int rmode;
    struct SifRpcServerData *server;
}
```

8000000Ch Get other data

```
struct SifRpcOtherDataPkt
{
    struct SifCmdHeader sifcmd;
    int rec_id;
    void *pkt_addr;
    int rpc_id;

    struct SifRpcReceiveData *receive;
    void *src;
    void *dest;
    int size;
}
```

Used by a server to request more data from a client, which replies with an END packet.

SIF RPC System Servers

System server IDs have bit 31 set (e.g. 80000000h). Known system IDs are listed here. Note that games can install custom servers with any possible ID, as long as bit 31 is not set, so those are excluded from the list.

System Server IDs

80000001h - File I/O (FILEIO)
 80000003h - IOP Heap Allocation (FILEIO)
 80000006h - Module/ELF Loader (LOADFILE)
 80000100h - Pad (PADMAN)
 80000101h - Pad extension? (PADMAN)
 80000400h - Memory cards (MCSERV)
 80000592h - CDVD Init (CDVDFSV)
 80000593h - CDVD S commands (CDVDFSV)
 80000595h - CDVD N commands (CDVDFSV)
 80000597h - CDVD SearchFile (CDVDFSV)
 8000059Ah - CDVD Disk Ready (CDVDFSV)
 80000701h - LIBSD Remote (SDRDRV, not in BIOS)
 80000901h - MTAP Port Open (MTAPMAN, not in BIOS)
 80000902h - MTAP Port Close (MTAPMAN)
 80000903h - MTAP Get Connection (MTAPMAN)
 80000904h - MTAP Unknown (MTAPMAN)
 80000905h - MTAP Unknown (MTAPMAN)
 80001400h - EyeToy (EYETOY, not in BIOS)

BIOS

The PS2 BIOS resides in a 4 MB ROM. Its purpose is to initialize hardware into a usable state, provide EE services and IOP modules, and launch PS2/PSX games.

BIOS Reference

[BIOS File Structure](#)
[BIOS Boot Process](#)
[BIOS EE Threading](#)
[BIOS EE Syscalls](#)
[BIOS EE Patches](#)
[BIOS PlayStation Compatibility](#)

BIOS IOP Modules

Unlike the EE, the IOP does not have a monolithic kernel. Instead, the kernel and device drivers are split into many modules. These modules are represented as IRX files, which are custom relocatable ELF.

[BIOS IOP Module Linking](#)
[BIOS List of IOP Modules](#)

BIOS File Structure

The BIOS consists of dozens of separate files. These files are indexed and accessed through ROMDIR.

Each ROMDIR entry has the following format.

```
struct romdir_entry
{
    char name[10]; //File name, must be null terminated
    ushort ext_info_size; //Size of the file's extended info in EXTINFO
    uint file_size; //Size of the file itself
}
```

First four entries in an SCPH-39001 ROMDIR:

```
0x00002740: 52455345 54000000 00000C00 40270000    RESET
0x00002750: 524F4D44 49520000 00005400 D0050000    ROMDIR
0x00002760: 45585449 4E464F00 00000000 80070000    EXTINFO
0x00002770: 524F4D56 45520000 00000000 10000000    ROMVER
...
```

To find the start of ROMDIR, look for the first occurrence of "RESET" in the BIOS.

To find a specific file in the BIOS, find its entry in ROMDIR and then add the file sizes of all previous files to get the starting address.

BIOS Boot Process

Upon reset, both the EE and IOP begin executing at BFC00000h. The BIOS checks which CPU is executing by checking COP0.PRid (register 15). If PRid >= 59h, the EE boot code is executed. Else, the IOP boot code is executed.

EE boot process

- The EE clock speed is measured. (SCPH-10000 uses COP0.Count; SCPH-39001 counts HBLANKs using one of the timers. Unknown what other BIOSes use.)
- The memory controller is initialized, giving access to the 32 MB of RDRAM.
- The EE kernel is copied into RAM starting at virtual address 80000000h. The BIOS jumps to the entry point at 80001000h.
- The TLB and PGIF handler are initialized and various hardware components are reset.
- EENULL, an idle loop thread, is loaded into 00081FC0h.
- SIF DMA is initialized. The EE writes to an SIF register and waits for a reply from the IOP.
- The EELOAD module is loaded, which in turn loads OSDSYS.

OSDSYS, once loaded, is responsible for bringing up the "Sony Computer Entertainment" screen and the browser.

IOP boot process

- Various hardware registers are initialized, and IOPBOOT is loaded.
- IOPBOOT finds SYSMEM and LOADCORE, loads them into memory, and executes their entry points.
- LOADCORE boots all of the IOP modules in the BIOS, starting with EXCEPMAN.
- At some point, SIF is initialized. The IOP waits for the EE to send a message and replies over another SIF register.
- Once all modules have been loaded, the IOP enters an infinite loop, waiting for the EE to send commands.

BIOS EE Threading

The EE kernel uses a cooperative priority-based thread scheduler. It can support up to 256 threads and 256 semaphores. 128 priority levels are available.

Structs

```
struct ThreadParam //Used as argument for CreateThread, ReferThreadStatus
{
    int status;
    void *func; //function to execute when thread begins
    void *stack;
    int stack_size;
    void *gp_reg;
    int initial_priority;
    int current_priority;
    u32 attr;
    u32 option;
}
struct SemaParam //Used as argument for CreateSema
{
    int count, //used by WaitSema and SignalSema
        max_count,
        init_count, //initial value for count
        wait_threads; //number of threads associated with this semaphore
    u32 attr, //not used by kernel
        option; //not used by kernel
}
//Thread statuses
#define THS_RUN 0x01
#define THS_READY 0x02
#define THS_WAIT 0x04
#define THS_SUSPEND 0x08
#define THS_WAITSPEND 0x0C //THS_WAIT | THS_SUSPEND
#define THS_DORMANT 0x10
struct TCB //Internal thread structure
{
    struct TCB *prev;
    struct TCB *next;
    int status;
    void *func;
    void *current_stack;
    void *gp_reg;
    short current_priority;
    short init_priority;
    int wait_type; //0=not waiting, 1=sleeping, 2=waiting on semaphore
    int sema_id;
    int wakeup_count;
    int attr;
    int option;
    void *_func; //???
    int argc;
    char **argv;
    void *initial_stack;
    int stack_size;
}
```

```

    int *root; //function to return to when exiting thread?
    void *heap_base;
}
struct thread_context //Stack context layout
{
    u32 sa_reg; // Shift amount register
    u32 fcr_reg; // FCR[fs] (fp control register)
    u32 unkn;
    u32 unused;
    u128 at, v0, v1, a0, a1, a2, a3;
    u128 t0, t1, t2, t3, t4, t5, t6, t7;
    u128 s0, s1, s2, s3, s4, s5, s6, s7, t8, t9;
    u64 hi0, hi1, lo0, lo1;
    u128 gp, sp, fp, ra;
    u32 fp_regs[32];
}
struct sema //Internal semaphore structure
{
    struct sema *free; //pointer to empty slot for a new semaphore
    int count;
    int max_count;
    int attr;
    int option;
    int wait_threads;
    struct TCB *wait_next, *wait_prev;
}

```

Thread Scheduler

The kernel uses an array of 128 doubly-linked lists for managing thread priorities. The general algorithm for the scheduler is as follows:

```

void reschedule(uint32 EPC, uint32 stack)
    Set current thread entry function to EPC (instruction after exception)
    Set current thread stack base to stack
    Set current thread status to READY
    Loop through active thread priority list, starting from 0 (highest priority)
        If an active thread is found, set current thread to it and break
        If no active thread is found, print an error and call Exit(1)
    Set found thread's status to RUN and return its entry function and stack pointer

```

Scheduling is only invoked by syscalls and relies on absolute priority. Threads with lower priority will never run as long as there is an active thread with higher priority.

When the BIOS receives an interrupt, it will save the GPR registers into a buffer, so that the interrupt can be served without destroying the thread state. However, when a thread is paused (ie. yields the CPU) its CPU context is placed on the user stack using the context described above.

Semaphores

Threads that call WaitSema on a semaphore with a "count" of zero are placed in a WAIT state and removed from the active thread list.

Conversely, calling SignalSema re-adds a thread waiting on a semaphore to the active list.

Semaphores are useful for blocking execution of a thread until a task completes. Sony's official SIF protocol, for instance, use a semaphore to place the caller thread to sleep while the IOP processes a request. The caller thread is reactivated in the SIF interrupt handler.

BIOS EE Syscalls

When a SYSCALL instruction is executed, the EE jumps to 80000180h and sets COP0.Cause to 08h. The kernel syscall handler retrieves the syscall number from the v1 register.

Negative syscalls use the current thread's stack for internal operations and are meant to be used in interrupt handlers. Positive syscalls use the kernel stack, and some of them can trigger thread reschedules.

List of EE Syscalls

01h void ResetEE(int reset_flag)

Resets EE components depending on which bits are set in reset_flag:

```

0 - DMAC
1 - VU1
2 - VIF1
3 - GIF
4 - VU0
5 - VIF0
6 - IPU

```

NOTE: INTC is reset regardless of the flag's settings.

02h void SetGsCrt(bool interlaced, int display_mode, bool frame)

Initializes the PCRTC.

04h void Exit(int status)

Returns to the OSDSYS browser. Internally calls LoadExecPS2("rom0:OSDSYS", 1, "BootBrowser").

05h void _ExceptionEpilogue(RFU005)

Internal syscall used by kernel exception handlers. Returns to the user program.

06h void LoadExecPS2(const char* filename, int argc, char** argv)

Loads an ELF with the specified arguments and executes it. This function also clears all of the internal kernel state, such as threads and semaphores, before execution.

This function does not (and must not) return.

07h void ExecPS2(void* entry, void* gp, int argc, char** argv)

Clears all of the internal kernel state and creates a thread with priority 0 (main thread). The thread begins executing from the entry point.

This function does not (and must not) return.

10h int AddIntcHandler(int int_cause, int (*handler)(int), int next, void* arg, int flag)

Adds an interrupt handler with the specified cause to a queue. This function returns the id of the handler or -1 if the operation fails. int_cause corresponds to a bit in INTC_STAT/INTC_MASK. For example, int_cause=2 will register a VBLANK handler.

next is a previously registered handler id which this handler will be placed before. next==0 will place the handler in front of the queue, and next==-1 will place it at the back of the queue.

arg and flag are stored in the handler struct but don't seem to be used otherwise?

11h int RemoveIntcHandler(int int_cause, int handler_id)

Removes the handler associated with the interrupt cause. Returns -1 if the operation fails.

12h int AddDmacHandler(int dma_cause, int (*handler)(int), int next, void* arg, int flag)

Adds an interrupt handler for INT1 (DMAC) interrupts. Otherwise works the same as AddIntcHandler.

13h int RemoveDmacHandler(int dma_cause, int handler_id)

Works same as RemoveIntcHandler.

14h bool _EnableIntc(int cause_bit)

Enables the indicated bit in INTC_MASK. Returns true if this bit was set to 0 and false if it was already set to 1.

15h bool _DisableIntc(int cause_bit)

Disables the indicated bit in INTC_MASK. Returns true if this bit was set to 1 and false if it was already set to 0.

16h bool _EnableDmac(int cause_bit)

Same as _EnableIntc, but for D_STAT's mask.

17h bool _DisableDmac(int cause_bit)

Same as _DisableIntc, but for D_STAT's mask.

20h int CreateThread(ThreadParam* t)

Creates a thread with DORMANT status. The thread is not placed in the priority linked-list.

Only t->func, t->initial_priority, t->stack, t->stack_size, and t->gp_reg are used. The new thread re-uses the heap of its parent thread.

Returns the id of the newly created thread, or -1 if the function fails.

21h void DeleteThread(int thread_id)

Deletes a thread. The function only succeeds if the thread has DORMANT status and is not the current thread.

22h void StartThread(int thread_id, void* arg)

Adds a DORMANT thread to the active thread list and gives it READY status, forcing a thread reschedule. arg is intended to be used by the callee thread if it becomes the current thread.

The function fails if the thread is the current thread or is not DORMANT.

23h void ExitThread()

Removes the current thread from the active thread list, resets it, and gives it DORMANT status, forcing a thread reschedule.

24h void ExitDeleteThread()

Deletes the current thread, forcing a thread reschedule.

25h void TerminateThread(int thread_id)

Has different behavior depending on the thread's status:

No status (0), RUN, DORMANT - No effect.

READY - Removes thread from active thread list, resets it, and gives it DORMANT status, forcing a thread reschedule.

WAIT, WAITSPEND - Same as READY but also decrements the thread's semaphore's "wait_threads" by one.

All other statuses - Resets the thread and gives the thread DORMANT status, forcing a thread reschedule.

26h void iTerminateThread(int thread_id)

Same as TerminateThread, but does not force a thread reschedule. Used internally by TerminateThread.

Returns -1 if unsuccessful.

29h int ChangeThreadPriority(int thread_id, int priority)

Changes the thread's priority, forcing a thread reschedule. If thread_id==0, the current thread's id is used.

This function fails if the thread is DORMANT.

2Ah int iChangeThreadPriority(int thread_id, int priority)

Same as ChangeThreadPriority, but does not force a thread reschedule. Used internally by RotateThreadReadyQueue.

Returns -1 upon a failure, or the thread's old priority on a success.

2Bh void RotateThreadReadyQueue(int priority)

Rotates the threads in the given priority's linked list, forcing a thread reschedule. This allows different threads of the same priority level to execute.

2Ch int iRotateThreadReadyQueue(int priority)

Same as RotateThreadReadyQueue, but does not force a thread reschedule. Used internally by ChangeThreadPriority.

Returns -1 upon a failure, or the given priority on a success.

2Dh void ReleaseWaitThread(int thread_id)

If the thread has WAIT status, it is given READY status and added to the active thread list. If it has WAITSPEND status, it is given SPEND status.

Both (and only) these cases force a thread reschedule. The reference to the thread's semaphore is also removed.

2Eh int iReleaseWaitThread(int thread_id)

Same as ReleaseWaitThread, but does not force a thread reschedule. Internally used by ReleaseWaitThread.

Returns -1 if unsuccessful.

2Fh int GetThreadId()

Returns the current thread's id.

30h int ReferThreadStatus(int thread_id, ThreadParam* status)**31h int iReferThreadStatus(int thread_id, ThreadParam* status)**

Fills out the ThreadParam struct with the given thread's information. Returns -1 if unsuccessful.

The "i" variant performs exactly the same function.

32h void SleepThread()

If the current thread's "wakeup_count" is greater than zero, it is decremented. Else, if the thread status is RUN or READY, the current thread is removed from the active thread list and set to WAIT, forcing a thread reschedule.

33h void WakeupThread(int thread_id)

Has different behavior depending on the thread's status:

WAIT (sleeping) - Set to READY and re-added to active thread list, forcing a thread reschedule.
 WAITSUSPEND (sleeping) - Placed in SUSPEND status, forcing a thread reschedule.
 READY, SUSPEND, WAIT/WAITSUSPEND (semaphore) - Increments "wakeup_count", forcing a thread reschedule.
 Other statuses - No effect.

34h int iWakeupThread(int thread_id)

Same as WakeupThread, but does not force a thread reschedule. Internally used by WakeupThread.
 Returns -1 if unsuccessful.

35h int CancelWakeupThread(int thread_id)**36h int iCancelWakeupThread(int thread_id)**

Resets the thread's "wakeup_count" to zero. Returns -1 if unsuccessful, or the old wakeup_count otherwise.

37h int SuspendThread(int thread_id)**38h int iSuspendThread(int thread_id)**

Has different behavior depending on the thread's status:

READY, RUN - Removed from active thread list and placed in SUSPEND status.
 WAIT - Placed in WAITSUSPEND status.
 All other statuses - No effect.

Returns -1 on failure or thread_id otherwise.

BUG: This function does NOT force a thread reschedule! If the current thread is suspended, it will continue to run!

39h void ResumeThread(int thread_id)

Has different behavior depending on the thread's status:

SUSPEND - Placed in READY or RUN status and added to active thread list, forcing a thread reschedule.
 WAITSUSPEND - Placed in WAIT status, forcing a thread reschedule.
 All other statuses - No effect

3Ah int iResumeThread(int thread_id)

Same as ResumeThread, but does not force a thread reschedule. Internally used by ResumeThread.
 Returns -1 if unsuccessful or thread_id otherwise.

3Bh void JoinThread()**3Ch void* InitMainThread/RFU060(uint32 gp, void* stack, int stack_size, char* args, int root)**

Initializes the current thread. Returns the stack pointer of the thread.

If stack == -1, the stack pointer equals the end of RDRAM - stack_size. Else, it equals stack + stack_size.

This function should only be called before the program's main function.

3Dh void* InitHeap/RFU061(void* heap, int heap_size)

Initializes the current thread's heap. If heap == -1, the end of the heap resides at the thread's stack pointer. Else, the end of the heap is heap + heap_size.
 Returns the end of the thread's heap.

3Eh void* EndOfHeap()

Returns the current thread's heap base.

40h int CreateSema(SemaParam* s)

Creates a semaphore. Returns the semaphore's id if successful and -1 if not.

Only s->init_count and s->max_count need to be specified. s->attr and s->option may also be specified.

41h int DeleteSema(int sema_id)

Deletes the semaphore, forcing a thread reschedule. Threads waiting on the semaphore will either be released or suspended, depending on their status.

42h int SignalSema(int sema_id)

Signals a semaphore. If a thread has called WaitSema on this semaphore, this forces a thread rescheduling. Otherwise, the semaphore's count is incremented by one.
 Returns -1 if unsuccessful.

43h int iSignalSema(int sema_id)

Similar to SignalSema, except this does not reschedule threads. The semaphore's threads are simply re-added to the active thread list.
 This function is called internally by SignalSema. Returns -1 if unsuccessful and -2 if the thread is released from its wait state.

44h void WaitSema(int sema_id)

If the semaphore's "count" variable > 0, count is decremented. Else, a thread rescheduling occurs, changing the active thread. The caller thread's status is set to WAIT.

45h int PollSema(int sema_id)**46h int iPollSema(int sema_id)**

Decrements the semaphore's count variable. Returns -1 if unsuccessful.

64h void FlushCache(int mode)

Modes of operation
 mode=0: Flush data cache (invalidate+writeback dirty contents to memory)
 mode=1: Invalidate data cache
 mode=2: Invalidate instruction cache
 All other modes invalidate both caches.

70h uint64_t GsGetIMR()

Returns the value of the privileged register GS_IMR.

71h void GsPutIMR(uint64_t value)

Sets GS_IMR to the value.

73h void SetVSyncFlag(int* vsync_occurred, u64* csr_stat_on_vsync)

Sets two kernel pointers that are used in the default INTC interrupt handler when a VSYNC interrupt has occurred.

If vsync_occurred is not NULL, the handler sets the value pointed to by vsync_occurred to 1.

If both pointers are not NULL, the value pointed to by csr_stat_on_vsync is set to GS_CSR.

A good use of this syscall is to retrieve the even/odd interlacing field in GS_CSR as early as possible.

74h void SetSyscall/RFU116(int index, int address)

Replaces an entry on the syscall table with the specified address.
This function does not perform any bounds checking.

76h int SifDmaStat(unsigned int dma_id)

Returns a positive value if the SIF transfer is queued, 0 if the transfer is in progress, and a negative value if the transfer has completed.

77h unsigned int SifSetDma(SifDmaTransfer* trans, int len)

Low-level syscall for starting SIF1 transfers. The format of SifDmaTransfer is as follows.

```
struct SifDmaTransfer //Internal semaphore structure
{
    void* src; //EE source
    void* dest; //IOP destination
    int size; //Size in bytes
    int attr;
}
```

The len parameter is how large the SifDmaTransfer array is. Multiple SIF1 transfers are queued by the kernel.
This function returns the ID of the current transfer. It does not wait for the transfer to complete.

78h void SifSetDChain()

Initializes the SIF0 channel by resetting QWC and setting CHCR to 184h (channel busy, chain mode, TTE bit in tag enabled).

7Bh void ExecOSD(int argc, char argv)**

Shorthand for LoadExecPS2("rom0:OSDSYS", argc, argv).

7Dh void PSMMode()**7Eh int MachineType()****7Fh int GetMemorySize()**

Returns the amount of RDRAM on the console in bytes.

BIOS EE Patches

The EE kernel is full of bugs, so games have to work around them. Both the official SDK and PS2SDK patch the alarm functions and certain threading functions before the main function is executed.

Alarm Patches

Using SetSyscall, games first set syscall 5Ah to be a memcpy with kernel privileges. 5Ah is then used to copy the real patch and EENULL to 80076000h and 00082000h respectively.

The patch contains syscalls FCh-FFh, which are SetAlarm, iSetAlarm, ReleaseAlarm, and iReleaseAlarm respectively. It also patches the interrupt handler for timer 3 (INTC12) by setting "syscall" 12Ch, which is out of bounds.

Threading Patches

iWakeupThread and iSuspendThread do not work properly when used on the currently executing thread, or its priority in the case of iRotateThreadReadyQueue.

While not technically a patch, games resolve this by creating a "top thread" whose purpose is to redirect these interrupt syscalls to their non-interrupt variants, which forces thread reschedules.

BIOS PlayStation Compatibility

EE Side

In PSX mode, the EE and GS work together to emulate the PSX GPU. PS1DRV, a standard ELF, handles this emulation.

The main emulation loop of PS1DRV acts as a translation layer between PSX and GS commands. It polls PGIF registers to know when a command has arrived.

After OSDSYS detects a PSX disc, it loads PS1DRV and jumps to it. The setup for PSX mode is as follows.

- PS1DRV attempts to parse several hack databases. First, it parses an internal list of games stored in the executable. Next, if the PSX disc has a "PSD1.0.0" entry in SYSTEM.CNF, PS1DRV parses hacks from there. Finally, PS1DRV will attempt to open "BIDATA-SYSTEM/TITLE.DB" on both memory card ports. If one of them is opened successfully, it then parses the database. Unknown what the format of the SYSTEM.CNF and TITLE.DB hacks are.
- Some parameters are retrieved using GetOsdConfigParam and GetGsVParam. They are used to control factors like CDROM read speed and texture filtering.
- Various system registers are reset. PS1DRV also seems to remap SPU2 registers to their original PSX address base at 1F801C00h using SSBUS writes.
- Bit 19 is set in SIF_CTRL. This is probably to assert the reset line on the IOP, switching it to PSX mode. PS1DRV waits 1000000h ticks, then unsets bit 19, which probably deasserts the reset line, allowing the IOP to run. It then waits another 1000000h ticks.
- PS1DRV enters its emulation loop and waits for GPU commands to be sent.

IOP Side

TBIN is the PSX BIOS driver stored in the PS2 BIOS. It lacks the shell and jumps directly to the PS logo when loading a game. Unknown what other differences exist compared to real PSX models.

RESET will jump to TBIN if COP0.Priid is less than 0Fh or if I/O register 1F801570.3 is set. Unknown which one is true after the PS2->PSX transition, or if both are true after that.

Unknown what TBIN means. A string in it says "PS compatible mode by M.T.", which most likely refers to a person. If that is the case, TBIN could mean T's Binary, whoever T is.

The most significant change when transitioning to PSX mode is that the IOP is underclocked to 33.8688 MHz, the original speed of the PSX. SPU2 also outputs samples at 44.1 KHz, rather than 48 KHz.

Hacks

The overhead of translating GPU commands, combined with the GS being far, far faster than the PSX GPU, means that the emulator will have different timings from a real PSX. Some PSX games are exceptionally sensitive to timings, so the emulator is able to apply various hacks. All hacks are listed below. Note that the hacks aren't actually given names, so they are made up here.

- Render Polygon Delay
- Render Rectangle Delay
- Force CDROM speed: When greater than 2, this overwrites the speed setting set in OSDSYS.

- GP0 Increased Sync: Makes the emulator spend more time polling the GP0 FIFO directly, allowing it to respond faster to incoming data.
- VRAM Read Delay
- VBLANK Delay: Adds extra scanlines to emulated frame time, making emulated VBLANK interrupts take more time to occur.
- Timer Delay: Makes the emulator not do anything until timer 0's counter is equal to or greater than this value plus 4.
- Misc. Render Hack
 - Bit 0: Strange hack. Ignores VRAM->VRAM transfers if the command word is not 80000000h.
 - Bit 1: When set, makes the render polygon delay only apply if the polygon is textured.

BIOS IOP Module Linking

IOP modules are dynamically linked to each other, but Sony devised an ingenious way to handle this without exposing symbols.

Linking is accomplished through export tables and import tables. A module can have multiple export tables, which define the functions other modules may access. Import tables give a module access to another module's export table.

The format of an export table is as follows.

```
struct export_table
{
    uint magic; //Must equal 0x41C00000!
    export_table* next; //Internal data for LOADCORE
    ushort version; //0x101 would be version 1.01
    ushort mode; //Unknown what this does
    char name[8]; //Name of the module. Must include a NULL terminator.
    void* export[0]; //An arbitrarily sized array of function pointers.
}
```

The format of a import table is similar.

```
struct import_table
{
    uint magic; //Must equal 0x41E00000!
    import_table* next; //Internal data for LOADCORE
    ushort version;
    ushort mode;
    char name[8];
    void* import[0]; //An arbitrarily sized array of jr ra; addiu zero, zero, X instruction pairs. More on that below.
}
```

LOADCORE is responsible for linking modules to each other. When a new module is loaded, LOADCORE parses its import tables. When an import table matches an export table, it looks up functions by taking X from addiu zero, zero, X, which is used as an index in the export's function pointer array. Each jr ra is then replaced with j \$function. In other words, the return statement is replaced with a direct jump to an imported module's function.

Since functions are only linked by their IDs, rather than their names, in theory this keeps others from learning the names of the functions. In practice, game developers often created custom IOP modules which they forgot to strip, which is how the symbols in Sony's IOP modules are known.

BIOS List of IOP Modules

The below IOP modules are loaded on reset.

[BIOS IOP IOPBOOT - Kernel Bootstrap](#)
[BIOS IOP SYSMEM - Memory Management](#)
[BIOS IOP LOADCORE - Kernel Loader and Linker](#)
[BIOS IOP EXCEPMAN - Exception Manager](#)
[BIOS IOP INTRMAN - Interrupt Manager](#)
[BIOS IOP SSBUSC - Subsystem Bus Controller](#)
[BIOS IOP DMACMAN - DMAC Manager](#)
[BIOS IOP TIMRMAN - Timer Manager](#)
[BIOS IOP SYSCLIB - Standard C Library](#)
[BIOS IOP HEAPLIB - Heap Allocation Library](#)
[BIOS IOP EECONF - EE Configuration](#)
[BIOS IOP THREADMAN - Thread Manager](#)
[BIOS IOP VBLANK - VBLANK Interrupt Manager](#)
[BIOS IOP IOMAN - File Input/Output Manager](#)
[BIOS IOP MODLOAD - Module Loader](#)
[BIOS IOP ROMDRV - ROM File Driver](#)
[BIOS IOP STDIO - C Standard Input/Output](#)
[BIOS IOP SIFMAN - SIF Low-level Manager](#)
[BIOS IOP IGREETING - Boot Info Display](#)
[BIOS IOP SIFCMD - SIF RPC Manager](#)
[BIOS IOP REBOOT - SIF Reboot Server](#)
[BIOS IOP LOADFILE - Module Loader RPC Server](#)
[BIOS IOP CDVDMAN - CDVD Manager](#)
[BIOS IOP CDVDFSV - CDVD RPC Server](#)
[BIOS IOP SIFINIT - SIF Initialization](#)
[BIOS IOP FILEIO - File Input/Output RPC Server](#)
[BIOS IOP SECRMAN - Security Manager](#)
[BIOS IOP EESYNC - Boot Finish Messenger](#)

Below modules are not loaded at startup, though they are in the BIOS. Instead, the EE must manually load them.

[BIOS IOP LIBSD - Low-level Sound Library](#)
[BIOS IOP SIO2MAN - SIO2 Manager](#)
[BIOS IOP MCMAN - Memory Card Manager](#)
[BIOS IOP MCSERV - Memory Card RPC Server](#)
[BIOS IOP PADMAN - Pad Input Manager](#)

BIOS IOP IOPBOOT - Kernel Bootstrap

IOPBOOT is technically not an IOP module, but rather, a raw binary file. It is responsible for parsing IOPBTCONF, loading SYSMEM and LOADCORE into memory, and passing execution to LOADCORE.

_entry(int ram_size, int boot_info, char *udnl_cmd, int unk)

The entry point is located at the start of IOPBOOT, BFC4A000h on SCPH-39001. ram_size is the total size of IOP RAM in megabytes. boot_info determines how IOPBOOT will find IOPBTCONF, more on that below. udnl_cmd is the full command string used to load UDNL and an IOPRP image. unk is passed but not used anywhere.

IOPBTCONF Format and Parsing

IOPBTCONF is the full list of modules that are loaded in the boot process, including SYSMEM and LOADCORE.

When IOPBOOT runs, it will first try to parse IOPBTCONx (where x is boot_info above), and if it can't find this, then it tries IOPBTCONF. On retail BIOSes, the only other file is IOPBTCON2, which is used to load modules after an IOP reboot. The regular IOPBTCONF is used for resets and the final phase of IOP reboot.

Each line in IOPBTCONF takes one of the following formats.

1. **@(START ADDR)**
The starting address of the kernel. IOPBOOT will place the first module at this address, and all other modules will be placed afterwards on 256-byte boundaries.
2. **!addr (FUNCTION ADDR)**
The function address here is executed while the modules are being loaded.
3. **!include (NAME)**
Acts like header file declarations, allowing another IOPBTCONF file to be parsed inside of the current one.
NOTE: The parsing for this is buggy. Files can only be included at the start or end of the list, and only one file can ever be included.
4. **#(COMMENT)**
Lines starting with # are treated as comments and ignored.
5. **(MODULE NAME)**
An IOP module to be loaded during boot.

Example IOPBTCONF

This excerpt was pulled from an SCPH-39001 BIOS.

```
@800
SYSMEM
LOADCORE
EXCEPMAN
INTRMANP
INTRMANI
SSBUSC
DMACMAN
TIMEMANP
TIMEMANI
...
```

BIOS IOP SYSMEM - Memory Management

SYSMEM is the first module to be loaded. It partitions memory as a linked list of 31 blocks, each block being an arbitrarily-sized multiple of 256 bytes. When too many blocks are allocated, SYSMEM will expand its block list by an additional 31 blocks; it can also free block lists that are no longer in use.

Exports

00h void* _start(int mem_size)

Initializes the memory allocator using the amount of memory given in bytes. Returns a pointer to allocated memory if successful, NULL otherwise.

04h void* AllocSysMemory(int mode, int size, void* ptr)

Allocates memory, returning a pointer to the memory if successful, NULL otherwise. size is rounded up to the nearest 256-byte boundary.

If mode==0, SYSMEM will allocate the first block in the list that meets the requirements.

If mode==1, SYSMEM will instead allocate the last free block.

If mode==2, SYSMEM will attempt to allocate memory at the given address in ptr. ptr is not used in any other mode.

05h int FreeSysMemory(void* ptr)

Frees memory, returning 0 if successful.

06h uint QueryMemSize()

Returns the total size of RAM in bytes.

07h uint QueryMaxFreeMemSize()

Returns the size of the largest free block in bytes - due to memory fragmentation, this is not necessarily the size of all free memory.

08h uint QueryTotalFreeMemSize()

Returns the amount of free memory in bytes.

09h void* QueryBlockTopAddress(void* addr)

Returns the starting address of the block that the given address is in. If a block is found but free, bit 31 of the result is set. Returns NULL if no block is found.

For example, if 0x150 is an allocated address, this function will return 0x100, as blocks must be allocated on a 256-byte boundary.

0Ah int QueryBlockSize(void* addr)

Returns the size of the block that the given address is in. If a block is found but free, bit 31 of the result is set. Returns NULL if no block is found.

0Eh int Kprintf(const char* format,...)

Wrapper function for the handler passed by KprintfSet. Does nothing if no handler is set.

0Fh void KprintfSet(KprintfHandler* handler, void* context)

Provides a handler function for Kprintf.

BIOS IOP LOADCORE - Kernel Loader and Linker

LOADCORE is the second module to be loaded. It loads and executes all necessary boot modules while also linking module imports to registered export tables.

Exports

00h void ((noreturn)) _start(lc_params* param)

```

struct lc_params
{
    uint ram_size;
    int boot_mode; //Determines what kind of reset has occurred, used by MODLOAD to start UDNL when rebooting the IOP.
    char* udnl_str; //Full argument list passed to UDNL
    u32* system_start;
    void* img_pos; //Pointer to memory that should be allocated. Used by UDNL to contain the updated modules.
    int img_buff_size;
    int module_count;
    u32** module_addr_list; //List of all modules to load. This includes SYMEM and LOADCORE
}

```

Boots all given modules, executes any post-boot callbacks, then enters an infinite loop.

04h void FlushIcache()

Flushes the icache. This should be used when executing code that is freshly loaded in memory, though the kernel handles this when new modules are loaded.

05h void FlushDcache()

Flushes the dcache. This should be used when passing data to and from peripherals, such as DMA.

06h int RegisterLibraryEntries(export_table* t)

Registers an export table, returning 0 if successful.

If an export table under the given name is already registered, several things can happen:

- If the major versions differ, the new export table is also registered, being treated as separate.
- If the new table's minor version is higher, the old table is destroyed, and any imports connected to the old table are updated to the new table.
- If the new table's minor version is lower or the same, the function fails.

07h int ReleaseLibraryEntries(export_table* t)

Unregisters an export table, returning 0 if successful.

0Ah int RegisterNonAutoLinkEntries(export_table* t)

Registers an export table that imports cannot link to, returning 0 if successful.

0Bh export_table* QueryLibraryEntryTable(iop_library* library)

Returns the export table associated with a registered library.

0Ch int* QueryBootMode(int mode)

Returns a registered "boot mode".

0Dh void RegisterBootMode(int* b)

Registers a boot mode, intended to give information to modules that LOADCORE executes.

Boot mode 4, for example, is used by MODLOAD to determine if UDNL should execute.

14h int RegisterPostBootCallback(BootupCallback func, int priority, int *stat)

Registers a function to be called after LOADCORE has finished loading all modules.

BIOS IOP EXCEPMAN - Exception Manager

EXCEPMAN installs exception vectors at RAM addresses 0x40 and 0x80, but they do not do anything useful by themselves. Other modules will use EXCEPMAN to install their own handlers.

Exports

04h int RegisterExceptionHandler(int exception, ExceptionHandler handler)

Alias for RegisterPriorityExceptionHandler(exception, 2, handler).

05h int RegisterPriorityExceptionHandler(int exception, int priority, ExceptionHandler handler)

06h int RegisterDefaultExceptionHandler(ExceptionHandler handler)

07h int ReleaseExceptionHandler(int exception, ExceptionHandler handler)

08h int ReleaseDefaultExceptionHandler(ExceptionHandler handler)

BIOS IOP INTRMAN - Interrupt Manager

INTRMAN installs the default interrupt and syscall vectors, though like EXCEPMAN, it is up to other modules to install useful handlers.

Note: the actual module name is INTRMANI - "intrman" is the export table name.

There is also an INTRMANP variant, possibly for PSX compatibility?

Exports

04h int RegisterIntrHandler(int irq, int mode, int (*handler)(void *), void *arg)

05h int ReleaseIntrHandler(int irq)

06h int EnableIntr(int irq)

07h int DisableIntr(int irq)

08h int CpuDisableIntr()

09h int CpuEnableIntr()

0Eh int CpuInvokeInKmode(void *function, ...)

0Fh void DisableDispatchIntr(int irq)

10h void EnableDispatchIntr(int irq)

```
11h int CpuSuspendIntr(int *state)
12h int CpuResumeIntr(int state)
17h int QueryIntrContext()
18h int QueryIntrStack(void *sp)
19h int iCatchMultiIntr(void)
```

BIOS IOP SSBUSC - Subsystem Bus Controller

Not entirely sure what SSBUSC does. It seems to involve memory mapping and access times for the IOP?

Exports

```
04h int SetDelay(int device, uint value)
05h int GetDelay(int device)
06h int SetBaseAddress(int device, uint value)
07h int GetBaseAddress(int device)
08h int SetRecoveryTime(uint value)
09h int GetRecoveryTime()
0Ah int SetHoldTime(uint value)
0Bh int GetHoldTime()
0Ch int SetFloatTime(uint value)
0Dh int GetFloatTime()
0Eh int SetStrobeTime(uint value)
0Fh int GetStrobeTime()
10h int SetCommonDelay(uint value)
11h int GetCommonDelay()
```

BIOS IOP DMACMAN - DMA Manager

Note: The names below are made up, because no known modules expose symbols for DMACMAN.

Exports

```
04h void DmaSetMadr(int chan, uint value)
05h uint DmaGetMadr(int chan)
06h void DmaSetBcr(int chan, uint value)
07h uint DmaGetBcr(int chan)
08h void DmaSetChcr(int chan, uint value)
09h uint DmaGetChcr(int chan)
0Ah void DmaSetTadr(int chan, uint value)
0Bh uint DmaGetTadr(int chan)
0Eh void DmaSetDpcer(uint value)
0Fh uint DmaGetDpcer()
10h void DmaSetDpcer2(uint value)
11h uint DmaGetDpcer2()
12h void DmaSetDpcer3(uint value)
13h uint DmaGetDpcer3()
14h void DmaSetDicer(uint value)
15h uint DmaGetDicer()
16h void DmaSetDicer2(uint value)
17h uint DmaGetDicer2()
```

1Ch int DmaRequestTransfer(uint chan, void* addr, uint size, uint count, int dir)

20h void DmaStartTransfer(uint chan)

21h void DmaSetChanPrio(uint chan, uint val)

22h void DmaEnableChan(uint chan)

23h void DmaDisableChan(uint chan)

BIOS IOP TIMRMAN - Timer Manager

Note: Like for INTRMAN, the actual module name is TIMEMANI - "timrman" is the export table name.
There is also a TIMEMANP variant, possibly for PSX mode?

Exports

04h int AllocHardTimer(int source, int size, int prescale)

05h int ReferHardTimer(int source, int size, int mode, int modemask)

06h int FreeHardTimer(int timid)

07h void SetTimerMode(int timid, int mode)

08h uint GetTimerStatus(int timid)

09h void SetTimerCounter(int timid, uint count)

0Ah uint GetTimerCounter(int timid)

0Bh void SetTimerCompare(int timid, uint compare)

0Ch uint GetTimerCompare(int timid)

0Dh void SetHoldMode(int holdnum, int mode)

0Eh int GetHoldMode(int holdnum)

0Fh uint GetHoldReg(int holdnum)

10h int GetHardTimerIntrCode(int timid)

Below are functions only found in newer TIMRMAN modules. (i.e., not in the BIOS)

14h int SetTimerHandler(int timid, ulong compare, uint (*handler)(void*), void *common)

15h int SetOverflowHandler(int timid, uint (*handler)(void*), void *common)

16h int SetupHardTimer(int timid, int source, int mode, int prescale)

17h int StartHardTimer(int timid)

18h int StopHardTimer(int timid)

BIOS IOP SYSCLIB - Standard C Library

Exports

04h int setjmp(jmp_buf env)

05h void longjmp(jmp_buf env)

06h char _toupper(char c)
Note: non-standard, returns char instead of int.

07h char _tolower(char c)
Note: non-standard, returns char instead of int.

08h uchar look_ctype_table(char character)

09h void* get_ctype_table()

0Ah void* memchr(const void *s, int c, size_t n)

0Bh int memcmp(const void *p, const void *q, size_t size)

0Ch void* memcpy(void *dest, const void *src, size_t size)

0Dh void* memmove(void *dest, const void *src, size_t size)

0Eh void* memset(void *ptr, int c, size_t size)

0Fh int bcmp(const void *s1, const void *s2, size_t size)

10h void bcopy(const void *src, void *dest, size_t size)

```

11h void bzero(void *ptr, size_t size)
12h int prnt(PrintCallback func, void *context, const char *format, va_list ap)
13h int sprintf(char *str, const char *format, ...)
14h char* strcat(char *dest, const char *src)
15h char* strchr(const char *s, int c)
16h int strcmp(const char *p, const char *q)
17h char* strcpy(char *dest, const char *src)
18h size_t strcspn(const char *s, const char *reject)
19h char* index(const char *s, int c)
1Ah char* rindex(const char *s, int c)
1Bh size_t strlen(const char *s)
1Ch char* strncat(char *dest, const char *src, size_t size)
1Dh int strncmp(const char *p, const char *q, size_t size)
1Eh char* strncpy(char *dest, const char *src, size_t size)
1Fh char* strpbrk(const char *s, const char *accept)
20h char* strrchr(const char *s, int c)
21h size_t strspn(const char *s, const char *accept)
22h char* strstr(const char *haystack, const char *needle)
23h char* strtok(char *s, const char *delim)
24h long strtol(const char *s, char **endptr, int base)
25h char* atob(char *s, int *i)
26h ulong strtoul(const char *s, char **endptr, int base)
28h void* wmemcpy(uint *dest, const uint *src, size_t size)
29h void* wmemset(uint *dest, uint c, size_t size)
2Ah int vsprintf(char *str, const char *format, va_list ap)
2Bh char* strtok_r(char *s, const char *delim, char **lasts)

```

BIOS IOP HEAPLIB - Heap Allocation Library

Exports

```

04h void* CreateHeap(int size, int flag)
05h void DeleteHeap(void* heap)
06h void* AllocHeapMemory(void *heap, size_t nbytes)
07h int FreeHeapMemory(void *heap, void *ptr)
08h int HeapTotalFreeSize(void *heap)

```

BIOS IOP EECONF - EE Configuration

Not sure what this does. It seems to clear out NVRAM for configuration settings and initialize some things after a reboot? Needs more research. No exports.

BIOS IOP THREADMAN - Thread Manager

This is a relatively large module that handles everything related to threading and scheduling.

The scheduler algorithm is currently unknown, though it is likely similar to the EE kernel's scheduler - that is to say, cooperatively threaded. The IOP kernel has more functionality though, such as event flags and message boxes.

THREADMAN contains multiple export tables. To make things more organized, they are split here into separate sections.

[BIOS IOP THBASE - Basic Threading](#)

[BIOS IOP THEVENT - Event Flags](#)

[BIOS IOP THSEMAP - Semaphores](#)

[BIOS IOP THMSGBOX - Message Boxes](#)

[BIOS IOP THFPOOL - Fixed-length Memory Pools](#)

BIOS IOP THBASE - Basic Threading

Exports

04h int CreateThread(ThreadParam *t)
05h int DeleteThread(int tid)
06h int StartThread(int tid, void *arg)
07h int StartThreadArgs(int tid, int args, void *argp)
08h int ExitThread()
09h int ExitDeleteThread()
0Ah int TerminateThread(int tid)
0Bh int iTerminateThread(int tid)
0Eh int ChangeThreadPriority(int tid, int priority)
0Fh int iChangeThreadPriority(int tid, int priority)
10h int RotateThreadReadyQueue(int priority)
11h int iRotateThreadReadyQueue(int priority)
12h int ReleaseWaitThread(int tid)
13h int iReleaseWaitThread(int tid)
14h int GetThreadId()
15h int CheckThreadStack()
16h int ReferThreadStatus(int tid, ThreadInfo *info)
17h int iReferThreadStatus(int tid, ThreadInfo *info)
18h int SleepThread()
19h int WakeupThread(int tid)
1Ah int iWakeupThread(int tid)
1Bh int CancelWakeupThread(int tid)
1Ch int iCancelWakeupThread(int tid)
21h int DelayThread(int usec)
22h int GetSystemTime(SysClock* clock)
23h int SetAlarm(SysClock* clock, uint (*alarm_cb)(void *), void* arg)
24h int iSetAlarm(SysClock* clock, uint (*alarm_cb)(void *), void* arg)
25h int CancelAlarm(uint (*alarm_cb)(void *), void* arg)
26h int iCancelAlarm(uint (*alarm_cb)(void *), void* arg)
27h void USec2SysClock(uint usec, SysClock* clock)
28h void SysClock2USec(SysClock* clock, uint* sec, uint* usec)
29h int GetSystemStatusFlag()

BIOS IOP THEVENT - Event Flags

Exports

04h int CreateEventFlag(EventParam* e)
05h int DeleteEventFlag(int ef)
06h int SetEventFlag(int ef, uint bits)
07h int iSetEventFlag(int ef, uint bits)
08h int ClearEventFlag(int ef, uint bits)
09h int iClearEventFlag(int ef, uint bits)

0Ah int WaitEventFlag(int ef, uint bits, int mode, uint *resbits)

0Bh int PollEventFlag(int ef, uint bits, int mode, uint *resbits)

0Dh int ReferEventFlagStatus(int ef, EventInfo *info)

0Eh int iReferEventFlagStatus(int ef, EventInfo *info)

BIOS IOP THSEMAP - Semaphores

Exports

04h int CreateSema(SemaParam *s)

05h int DeleteSema(int sid)

06h int SignalSema(int sid)

07h int iSignalSema(int sid)

08h int WaitSema(int sid)

09h int PollSema(int sid)

0Bh int ReferSemaStatus(int sid, SemaInfo *info)

0Ch int iReferSemaStatus(int sid, SemaInfo *info)

BIOS IOP THMSG BX - Message Boxes

Exports

04h int CreateMbx(MbxParam* m)

05h int DeleteMbx(int mid)

06h int SendMbx(int mid, void* msg)

07h int iSendMbx(int mid, void* msg)

08h int ReceiveMbx(void msg, int mid)**

09h int PollMbx(void msg, int mid)**

0Bh int ReferMbxStatus(int mid, MbxInfo* info)

0Ch int iReferMbxStatus(int mid, MbxInfo* info)

BIOS IOP THFPOOL - Fixed-length Memory Pools

Exports

04h int CreateFpl(FPoolParam* fp)

05h int DeleteFpl(int fpid)

06h void* AllocateFpl(int fpid)

07h void* pAllocateFpl(int fpid)

08h void* ipAllocateFpl(int fpid)

09h int FreeFpl(int fpid, void *mem)

0Bh int ReferFplStatus(int fpid, FPoolInfo *info)

0Ch int iReferFplStatus(int fpid, FPoolInfo *info)

BIOS IOP THVPOOL - Variable-length Memory Pools

Exports

04h int CreateVpl(VPoolParam* vp)

05h int DeleteVpl(int vpid)

06h void* AllocateVpl(int vpid, int size)

07h void* pAllocateVpl(int vpid, int size)

08h void* ipAllocateVpl(int vpid, int size)

09h int FreeVpl(int vpid, void *mem)

0Bh int ReferVplStatus(int vpid, VPoolInfo *info)

0Ch int iReferVplStatus(int vpid, VPoolInfo *info)

BIOS IOP VBLANK - VBLANK Interrupt Manager

Exports

04h void WaitVblankStart()

05h void WaitVblankEnd()

06h void WaitVblank()

07h void WaitNonVblank()

08h int RegisterVblankHandler(int end, int priority, int (*handler)(void *), void *arg)

09h int ReleaseVblankHandler(int end, int (*handler)(void *))

BIOS IOP IOMAN - File Input/Output Manager

IOMAN provides a C-style interface for file operations - other modules must install their own drivers for IOMAN to be effective.

IOMAN does install a TTY driver intended for use with STDIO. On retail BIOSes, the TTY functions are all stubbed. Unknown if other IOMAN modules in other BIOSes install proper drivers, or if the drivers are re-installed by something else.

Exports

04h int open(const char *name, int mode)

05h int close(int fd)

06h int read(int fd, void *ptr, size_t size)

07h int write(int fd, void *ptr, size_t size)

08h int lseek(int fd, int pos, int whence)

09h int ioctl(int fd, int command, void *arg)

0Ah int remove(const char *name)

0Bh int mkdir(const char *path)

0Ch int rmdir(const char *path)

0Dh int dopen(const char *path, int mode)

0Eh int dclose(int fd)

0Fh int dread(int fd, void *buf)

10h int getstat(const char *name, IOStat *stat)

11h int chstat(const char *name, IOStat *stat, uint statmask)

12h int format(const char *dev)

14h int AddDrv(IODevice *device)

15h int DelDrv(const char *name)

Below are functions only found in newer IOMAN modules.

17h void StdioInit(int mode)

19h int rename(const char *old, const char *new)

1Ah int chdir(const char *name)

1Bh int sync(const char *dev, int flag)

1Ch int mount(const char *fsname, const char *dev, int flag, void *arg, int arglen)

1Dh int umount(const char *fsname)

1Eh long long lseek64(int fd, long long offset, int whence)

1Fh int devctl(const char *name, int cmd, void *arg, uint arglen, void *buf, uint buflen)

20h int symlink(const char *old, const char *new)

21h int readlink(const char *path, char *buf, unsigned int buflen)

22h int ioctl2(int fd, int cmd, void *arg, uint arglen, void *buf, uint buflen)

BIOS IOP MODLOAD - Module Loader

Exports

```
04h int ReBootStart(const char *command, uint flags)

05h int LoadModuleAddress(const char *name, void* addr, int offs)

06h int LoadModule(const char *name)

07h int LoadStartModule(const char *name, int arglen, const char *args, int *result)

08h int StartModule(int id, const char *name, int arglen, const char *args, int *result)

09h int LoadModuleBufferAddress(void *buffer, void* addr, int offs)

0Ah int LoadModuleBuffer(void *buffer)

0Ch void SetSecrmanCallbacks(void *CardBootFunc, void *DiskBootFunc, void *LoadfileFunc)

0Dh void SetCheckKelfPathCallback(void *CheckKelfPathFunc)
```

BIOS IOP ROMDRV - ROM File Driver

This module has no exports. It simply installs an IOMAN driver for rom0 and rom1, the boot ROM and DVD ROM respectively.

BIOS IOP STDIO - C Standard Input/Output

Exports

```
04h int printf(const char *format, ...)

05h int getchar()

06h int putchar(int c)

07h int puts(const char *s)

08h char *gets(char *s)

09h int fdprintf(int fd, const char *format, ...)

0Ah int fdgetc(int fd)

0Bh int fdputc(int c, int fd)

0Ch int fdputs(const char *s, int fd)

0Dh char *fdgets(char *buf, int fd)

0Eh int vfdprintf(int fd, const char *format, va_list ap)
```

BIOS IOP SIFMAN - SIF Low-level Manager

Exports

```
02h void sceSifExit()

04h void sceSifDma2Init()

05h void sceSifInit()

06h void sceSifSetDChain()

07h int sceSifSetDma(SifDmaTransfer *trans, int len)

08h int sceSifDmaStat(int trid)

09h void sceSifSetOneDma(SifDmaTransfer dmat)

0Ch void sceSifDma0Transfer(void *addr, int size, int mode)

0Dh void sceSifDma0Sync()

0Eh int sceSifDma0Sending()

0Fh void sceSifDma1Transfer(void *addr, int size, int mode)

10h void sceSifDma1Sync()

11h int sceSifDma1Sending()

12h void sceSifDma2Transfer(void *addr, int size, int mode)

13h void sceSifDma2Sync()
```

```

14h int sceSifDma2Sending()
15h uint sceSifGetMSFlag()
16h uint sceSifSetMSFlag(uint value)
17h uint sceSifGetSMFlag()
18h uint sceSifSetSMFlag(uint value)
19h uint sceSifGetMainAddr()
1Ah uint sceSifGetSubAddr()
1Bh uint sceSifSetSubAddr(uint value)
1Ch void sceSifIntrMain()
1Dh int sceSifCheckInit()
1Eh void sceSifSetDmaIntrHandler(void (*handler)(void *), void *arg)
1Fh void sceSifResetDmaIntrHandler()
20h uint sceSifSetDmaIntr(SifDmaTransfer *trans, int len, void (*func)(), void *data)

```

BIOS IOP IGREETING - Boot Info Display

No exports. This module outputs console information to the TTY, as well as what kind of boot has occurred (hard reset, update reboot, etc).

Example output:

```

PlayStation 2 ===== Hard reset boot
ROMGEN=2002-0207, IOP info (CPUID=1f, CACH_CONFIG=0, 2MB, IOP mode)
<20020207-164243,ROMconf,PS20160AC20020207.bin:11552>

```

BIOS IOP SIFCMD - SIF RPC Manager

SIFCMD provides a server/client abstraction around SIFMAN, allowing modules to create their own RPC servers that listen to incoming requests from the EE.

Exports

```

02h int sceSifExitRpc()

04h int sceSifInitCmd()

05h void sceSifExitCmd()

06h uint sceSifGetSreg(int index)

07h void sceSifSetSreg(int index, uint value)

08h SifCmdHandlerData* sceSifSetCmdBuffer(SifCmdHandlerData *cmdBuffer, int size)

09h SifCmdHandlerData* sceSifSetSysCmdBuffer(SifCmdHandlerData *sysCmdBuffer, int size)

0Ah void sceSifAddCmdHandler(int cid, SifCmdHandler_t handler, void *harg)

0Bh void sceSifRemoveCmdHandler(int cid)

0Ch uint sceSifSendCmd(int cmd, void *packet, int packet_size, void *src_extra, void *dest_extra, int size_extra)

0Dh uint isceSifSendCmd(int cmd, void *packet, int packet_size, void *src_extra, void *dest_extra, int size_extra)

0Eh void sceSifInitRpc(int mode)

0Fh int sceSifBindRpc(SifRpcClientData *client, int rpc_number, int mode)

10h int sceSifCallRpc(SifRpcClientData *client, int rpc_number, int mode, void *send, int ssize, void *receive, int rsize, SifRpcEndFunc end_func, void *end_param)

11h void sceSifRegisterRpc(SifRpcServerData *sd, int sid, SifRpcFunc func, void *buf, SifRpcFunc cfunc, void *cbuf, SifRpcDataQueue *qd)

12h int sceSifCheckStatRpc(SifRpcClientData *cd)

13h SifRpcDataQueue* sceSifSetRpcQueue(SifRpcDataQueue *q, int thread_id)

14h SifRpcServerData* sceSifGetNextRequest(SifRpcDataQueue *qd)

15h void sceSifExecRequest(SifRpcServerData *srv)

16h void sceSifRpcLoop(SifRpcDataQueue *qd)

17h int sceSifGetOtherData(SifRpcReceiveData *rd, void *src, void *dest, int size, int mode)

```

18h SifRpcServerData* sceSifRemoveRpc(SifRpcServerData *sd, SifRpcDataQueue *qd)

19h SifRpcDataQueue* sceSifRemoveRpcQueue(SifRpcDataQueue_t *qd)

1Ah void sceSifSetSifICB(void *func, int param)

1Bh void sceSifClearSifICB()

BIOS IOP REBOOT - SIF Reboot Server

REBOOT installs a custom SIF command handler for 80000003h and waits for this command to be sent.

IOP Reboot Procedure

- The EE sends SIF command 80000003h with an argument of "rom0:UDNL [last image file]..[first image file] [-v]". An example image path is "cdrom0:\MODULES\IOPRP243.IMG;1" (sent by Atelier Iris: Eternal Mana). The -v flag is recognized by UDNL but doesn't seem to have any effect.
- REBOOT calls MODLOAD.ReBootStart. MODLOAD cleans up all module libraries and jumps to IOPBOOT.
- IOPBOOT bootstraps SYMEM and LOADCORE. LOADCORE loads all modules in IOPBTCON2.
- When MODLOAD is reloaded, it installs a post-boot callback in LOADCORE. This callback jumps to the UDNL module.
- UDNL scans all provided image files and finds the newest modules in each of them, then triggers yet another reset by bootstrapping SYMEM and LOADCORE.
- LOADCORE reads from IOPBTCONF and loads the newest modules in the image files or the ones in the boot ROM. The reboot is now complete.

Image File Format

Like the BIOS, an image file contains a ROMDIR filesystem. An image file usually contains a RESET file of zero bytes, ROMDIR, EXTINFO, and a list of newer IOP modules to load.

BIOS IOP LOADFILE - Module Loader RPC Server

The first real SIF RPC server that is loaded. Mainly responsible for loading new IOP modules and EE ELF's.

NOTE: Newer LOADFILE modules seem to have a different interface. Needs more research.

LOADFILE RPC (0x80000006)

00h SifLoadModule

01h SifLoadElf

02h SifIopSetAddr

03h SifIopGetAddr

04h SifLoadModuleEncrypted

05h SifLoadElfEncrypted

BIOS IOP CDVDMAN - CDVD Manager

For hardware details of the CDVD drive, see [CDVD Drive](#).

CDVDMAN installs an IOMAN driver for "cdrom".

Exports

04h int sceCdInit(int mode)

05h int sceCdStandby()

06h int sceCdRead(uint lbn, uint sectors, void* buff, sceCdRMode *mode)

07h int sceCdSeek(uint lbn)

08h int sceCdGetError()

09h int sceCdGetToc(void* toc)

0Ah int sceCdSearchFile(sceCdIFILE *file, const char *name)

0Bh int sceCdSync(int mode)

0Ch int sceCdGetDiskType()

0Dh int sceCdDiskReady(int mode)

0Eh int sceCdTrayReq(int param, uint *tray_status)

0Fh int sceCdStop()

10h uint sceCdPosToInt(sceCdILOCCD *p)

11h sceCdILOCCD* sceCdIntToPos(uint sector, sceCdILOCCD *p)

13h int sceCdGetToc2(void* buff, char unk)

14h int sceCdReadDVDV(u32 lbn, u32 sectors, void *buff, sceCdRMode *mode)

```
15h int sceCdCheckCmd()

16h int sceCdRI(void *buff, uint *result)

17h int sceCdWI(void *buff, uint *result)

18h int sceCdReadClock(sceCdCLOCK *clock)

19h int sceCdWriteClock(const sceCdCLOCK *clock)

1Ch int sceCdStatus()

1Dh int sceCdApplySCmd(int cmd, void *input, ushort size, void *output)

1Eh int sceCdSetHDMMode(uint mode)

1Fh int sceCdOpenConfig(int block, int mode, int block_count, uint *result)

20h int sceCdCloseConfig(uint *result)

21h int sceCdReadConfig(void *buff, uint *result)

22h int sceCdWriteConfig(void *buff, uint *result)

23h int sceCdReadKey(char unk1, char unk2, uint cmd, void *key)

24h int sceCdDecSet(char unk1, char unk2, char shift)

25h int sceCdCallback(void (*cb_func)(int reason))

26h int sceCdPause()

27h int sceCdBreak()

28h int sceCdReadCDDA(u32 lbn, u32 sectors, void *buff, sceCdRMode *mode)

29h int sceCdReadConsoleId(void *buff, uint *result)

2Ah int sceCdWriteConsoleId(void *buff, uint *result)

2Bh int sceCdMV(void *buff, uint *result)

2Ch int sceCdGetReadPos()

2Dh int sceCdCtrlADout(int unk, uint *result)

2Eh int sceCdNop

2Fh void* sceGetFsvRbuf()

30h int sceCdstm0Cb(void (*cb)(int))

31h int sceCdstm1Cb(void (*cb)(int))

32h int sceCdSC(int code, uint *error)

33h int sceCdRC(sceCdCLOCK *clock)

34h int sceCdForbidDVDP(uint *result)

35h int sceCdReadSUBQ(void *buff, uint *result)

36h int sceCdApplyNCmd(uint cmd, void *input, ushort size, void *output)

37h int sceCdAutoAdjustCtrl(int mode, uint *result)

38h int sceCdStInit(uint buff_size, uint banks, void *buff)

39h int sceCdStRead(uint sectors, uint *buff, uint mode, uint *result)

3Ah int sceCdStSeek(uint lbn)

3Bh int sceCdStStart(uint lbn, uint *result)

3Ch int sceCdStStat()

3Dh int sceCdStStop()
```

Below are exports only found in newer CDVDMAN modules.

```
3Eh int sceCdRead0(uint lbn, uint sectors, void *buff, sceCdRMode *mode, int csec, void *cb_func)

40h int sceCdRM(void *buff, uint *result)

41h int sceCdWM(void *buff, uint *result)

42h int sceCdReadChain(sceCdRChain *chain, sceCdRMode *mode)
```

```

43h int sceCdStPause()

44h int sceCdStResume()

45h int sceCdForbidRead(uint *result)

46h int sceCdBootCertify(const void *name)

47h int sceCdSpinCtrlIOP(uint speed)

48h int sceCdBlueLEDCtl(uchar value, uint *result)

49h int sceCdCancelPOffRdy(uint *result)

4Ah int sceCdPowerOff(uint *result)

4Bh int sceCdMmode(int media)

4Ch int sceCdReadFull(ulonglong lbn, ulonglong sectors, void *buff, sceCdRMode *mode)

4Dh int sceCdStSeekF(ulonglong lbn)

4Eh void* sceCdPOffCallback(void (*func)(void *),void *addr)

4Fh int sceCdReadDiskID(uint *id)

50h int sceCdReadGUID(ulonglong *guid)

51h int sceCdSetTimeout(int unk, int timeout)

52h int sceCdReadModelID(ulonglong *id)

53h int sceCdReadDvdDualInfo(int *is_dual, ulonglong *layer1_start)

54h int sceCdLayerSearchFile(sceCdIFile *fp, const char *path, int layer)

5Ah int sceCdStatus2()

70h int sceCdApplySCmd2(uchar cmd, const void *input, ulonglong size, void *output)

72h int sceCdRE(ulonglong lbn, ulonglong sectors, void *buff, sceCdRMode *mode)

73h int sceCdRcBypassCtl(int mode, uint *result)

```

BIOS IOP CDVDFSV - CDVD RPC Server

CDVDFSV creates several servers that allow the EE to interface with CDVDMAN. Some of these servers only perform a single function... not sure why Sony organized it like this.

Note: newer CDVDFSV modules seem to have a different interface. Needs research.

CdInit RPC (0x80000592)

CdSCmd RPC (0x80000593)

CdNCmd RPC (0x80000595)

CdSearchFile RPC (0x80000597)

CdDiskReady RPC (0x8000059A)

BIOS IOP SIFINIT - SIF Initialization

A rather strange module... it only exists to call SIFMAN.sceSifInit. CDVDFSV already does this though, so the usefulness of SIFINIT is questionable. Perhaps it had another purpose at one point during development?

BIOS IOP FILEIO - File Input/Output RPC Server

FILEIO creates two RPC servers: one for allocating memory on the IOP and one for interfacing with IOMAN.

Note: newer FILEIO modules have a different interface. Needs research.

IOPHEAP RPC (0x80000001)

FILEIO RPC (0x80000003)

BIOS IOP SECRMAN - Security Manager

SECRMAN handles decrypting MagicGate-protected executables/data and authenticating PS2 memory cards. The bulk of this work is handled by the CDVD Drive, which contains the actual MagicGate keys.

Exports

```
04h void SecrSetMcCommandHandler(int (*func)(int port, int slot, Sio2Transfer *sio2_trans))
```

```

05h void SecrSetMcDevIDHandler(int (*func)(int port, int slot))

06h int SecrAuthCard(int port, int slot, int cnum)

07h void SecrResetAuthCard(int port, int slot, int cnum)

08h int SecrCardBootHeader(int port, int slot, void *buff, SecrBitTable *BitTable, int *pSize)

09h int SecrCardBootBlock(void *src, void *dest, uint size)

0Ah void* SecrCardBootFile(int port, int slot, void *buff)

0Bh int SecrDiskBootHeader(void *buff, SecrBitTable *BitTable, int *pSize)

0Ch int SecrDiskBootBlock(void *src, void *dst, uint size)

0Dh void* SecrDiskBootFile(void *buff)

```

BIOS IOP EESYNC - Boot Finish Messenger

This module installs a post-boot callback in LOADCORE that calls SIFMAN.sceSifSetSMFlag(0x40000).
 Not sure if anything on the EE side looks for this value, but the write does indicate that the IOP has finished booting.

BIOS IOP LIBSD - Low-level Sound Library

Exports

```

02h int sceSdQuit()

04h int sceSdInit(int flag)

05h void sceSdSetParam(ushort entry, ushort value)

06h ushort sceSdGetParam(ushort entry)

07h void sceSdSetSwitch(ushort entry, uint value)

08h uint sceSdGetSwitch(ushort entry)

09h void sceSdSetAddr(ushort entry, uint value)

0Ah uint sceSdGetAddr(ushort entry)

0Bh void sceSdSetCoreAttr(ushort entry, ushort value)

0Ch ushort sceSdGetCoreAttr(ushort entry)

0Dh ushort sceSdNote2Pitch(ushort center_note, ushort center_fine, ushort note, short fine)

0Eh ushort sceSdPitch2Note(ushort center_note, ushort center_fine, ushort pitch)

0Fh int sceSdProcBatch(sceSdBatch *batch, uint *result, uint num)

10h int sceSdProcBatchEx(sceSdBatch *batch, uint *result, uint num, uint voice)

11h int sceSdVoiceTrans(short chan, ushort mode, void *addr, void *spuaddr, uint size)

12h int sceSdBlockTrans(short chan, ushort mode, void *addr, uint size, ...)

13h uint sceSdVoiceTransStatus(short chan, short flag)

14h uint sceSdBlockTransStatus(short chan, short flag)

15h SdIntrCallback sceSdSetTransCallback(int core, SdIntrCallback cb)

16h SdIntrCallback sceSdSetIRQCallback(SdIntrCallback cb)

17h int sceSdSetEffectAttr(int core, sceSdEffectAttr *attr)

18h void sceSdGetEffectAttr(int core, sceSdEffectAttr *attr)

19h int sceSdClearEffectWorkArea(int core, int channel, int effect_mode)

1Ah sceSdTransIntrHandler sceSdSetTransIntrHandler(int chan, sceSdTransIntrHandler func, void *arg)

1Bh sceSdSpu2IntrHandler sceSdSetSpu2IntrHandler(sceSdSpu2IntrHandler func, void *arg)

1Ch void* sceSdGetTransIntrHandlerArgument(int arg)

1Dh void* sceSdGetSpu2IntrHandlerArgument()

1Eh int sceSdStopTrans(int chan)

1Fh int sceSdCleanEffectWorkArea(int core, int chan, int effect_mode)

```

20h int sceSdSetEffectMode(int core, sceSdEffectAttr *param)

21h int sceSdSetEffectModeParams(int core, sceSdEffectAttr *attr)

BIOS IOP SIO2MAN - SIO2 Manager

For details on SIO2 hardware, see [SIO2](#).

Exports

02h void Sio2Exit()

04h void Sio2SetCtrl(uint value)

05h uint Sio2GetCtrl()

06h uint Sio2GetRecv1()

07h void Sio2SetPortCtrl1(int index, uint value)

08h uint Sio2GetPortCtrl1(int index)

09h void Sio2SetPortCtrl2(int index, uint value)

0Ah uint Sio2GetPortCtrl2(int index)

0Bh uint Sio2GetRecv2()

0Ch void Sio2SetReg(int index, uint value)

0Dh uint Sio2GetReg(int index)

0Eh uint Sio2GetRecv3()

0Fh void Sio2SetUnkReg78(uint value)

10h uint Sio2GetUnkReg78()

11h void Sio2SetUnkReg7C(uint value)

12h uint Sio2GetUnkReg7C()

13h void Sio2WriteDataFifo(uchar value)

14h uchar Sio2ReadDataFifo()

15h void Sio2SetIstat(uint value)

16h uint Sio2GetIstat()

17h void Sio2PadInitTransfer()

18h void Sio2McInitTransfer()

19h int Sio2StartTransfer(Sio2Transfer *trans)

The following exports are only available in newer SIO2MAN modules.

1Ah void Sio2TransferReset()

30h void Sio2MtapInitTransfer()

31h void Sio2RmInitTransfer()

32h void Sio2UnkInitTransfer()

33h int Sio2StartTransfer2(Sio2Transfer *trans)

34h void Sio2ResetTransfer2()

35h void Sio2MtapChangeSlotSet(void *func)

36h void Sio2MtapGetSlotMaxSet(void *func)

37h void Sio2MtapGetSlotMax2Set(void *func)

38h void Sio2MtapUpdateSlotsSet(void *func)

39h int Sio2MtapChangeSlot(int* arg)

3Ah int Sio2MtapGetSlotMax(int port)

3Bh int Sio2MtapGetSlotMax2(int port)

3Ch void Sio2MtapUpdateSlots()

BIOS IOP MCMAN - Memory Card Manager

Exports

05h int McDetectCard(int port, int slot)
06h int McOpen(int port, int slot, char *path, int flags)
07h int McClose(int fd)
08h int McRead(int fd, void *buff, int size)
09h int McWrite(int fd, void *buff, int size)
0Ah int McSeek(int fd, int offs, int whence)
0Bh int McFormat(int port, int slot)
0Ch int McGetDir(int port, int slot, char *dir, int flags, int maxent, sceMcTblGetDir *info)
0Dh int McDelete(int port, int slot, char *path, int flags)
0Eh int McFlush(int fd)
0Fh int McChDir(int port, int slot, char *new_dir, char *cur_dir)
10h int McSetFileInfo(int port, int slot, char *path, sceMcTblGetDir *info, int flags)
11h int McEraseBlock(int port, int block, void **page, void *ecc) (only in older modules)
12h int McReadPage(int port, int slot, int page, void *buff)
13h int McWritePage(int port, int slot, int page, void *buff, void *ecc)
14h void McDataChecksum(void *buff, void *ecc)
1Dh int McReadPS1PDACard(int port, int slot, int page, void *buff)
1Eh int McWritePS1PDACard(int port, int slot, int page, void *buff)
24h int McUnformat(int port, int slot)
26h int McGetFreeClusters(int port, int slot)
27h int McGetMcType(int port, int slot)
28h void McSetPS1CardFlag(int flag)

Below are exports only found in newer MCMAN modules.

11h int McEraseBlock2(int port, int slot, int block, void **page, void *ecc) (replaces McEraseBlock!)
15h int McDetectCard2(int port, int slot)
16h int McGetFormat(int port, int slot)
17h int McGetEntSpace(int port, int slot, char *dir)
18h int McReplaceBadBlock()
19h int McCloseAll()
2Ah void* McGetModuleInfo()
2Bh int McGetCardSpec(int port, int slot, short *page_size, ushort *block_size, int *total_size, uchar *flags)
2Ch int McGetFATentry(int port, int slot, int fat_index, int *fat_entry)
2Dh int McCheckBlock(int port, int slot, int block)
2Eh int McSetFATentry(int port, int slot, int fat_index, int fat_entry)
2Fh int McReadDirEntry(int port, int slot, int cluster, int index, McFsEntry **pfse)
30h void Mc1stCacheEntSetWrFlagOff()
31h int McCreateDirentry(int port, int slot, int parent_cluster, int entries, int cluster, sceMcStDateTime *time)
32h int McReadCluster(int port, int slot, int cluster, McCacheEntry **pmce)
33h int McFlushCache(int port, int slot)
34h int McSetDirEntryState(int port, int slot, int cluster, int fsindex, int flags)

BIOS IOP MCSERV - Memory Card RPC Server

(todo)

BIOS IOP PADMAN - Pad Input Manager

(todo)